VBACKUP — утилита наборов сохранения для Linux

Руководство пользователя

VBACKUP-UG-0125-RUS

VBACKUP Version X01-25 *** 27-OCT-2026

Copyright © 2026 StarLet Squad and Ruslan R. Laishev.

VBACKUP распространяется на условиях Apache License, Version 2.0 («Лицензия»); использовать его можно только в соответствии с Лицензией. Текст Лицензии — в файле LICENSE комплекта и по адресу http://www.apache.org/licenses/LICENSE-2.0.

Если иное не требуется применимым правом и не согласовано письменно, VBACKUP распространяется по Лицензии «КАК ЕСТЬ», БЕЗ КАКИХ-ЛИБО ГАРАНТИЙ И УСЛОВИЙ, явных или подразумеваемых. Права и ограничения по Лицензии — в её тексте.

Содержание

Предисловие **4**
Кому адресовано . . . 4
Зачем нужен VBACKUP . . . 5
Структура документа . . . 6
Сопутствующие документы . . . 7
Благодарности . . . 8
Лицензия . . . 9
Товарные знаки . . . 10
Соглашения . . . 11

Глава 1. Введение **12**
1.1 Что делает VBACKUP . . . 12
1.2 Что даёт saveset . . . 13
1.3 Чего VBACKUP не делает . . . 14
1.4 Программы комплекта . . . 15

Глава 2. Установка **16**
2.1 Требования . . . 16
2.2 Сборка и установка . . . 17
2.3 Установка в Windows . . . 18
2.4 Установка в другое место . . . 19
2.5 Проверка установки . . . 20
2.6 Удаление . . . 21

Глава 3. Основные понятия **22**
3.1 Saveset-ы . . . 23
3.2 Операции . . . 24
3.3 Структура saveset-а . . . 25
3.4 Спецификации файлов и шаблоны . . . 27
3.5 Инкрементное сохранение и журнал . . . 28
3.6 Физические операции и операции над образом . . . 29
3.7 Сжатие . . . 30
3.8 Шифрование . . . 33
3.9 Каналы . . . 35
3.10 Saveset-ы на других узлах . . . 36
3.11 Кэш страниц . . . 38
3.12 Потоки . . . 39
3.13 Автономный распаковщик и распаковщики последней надежды . . . 40
3.14 Модули файловых менеджеров . . . 41
3.15 Расписание и ротация . . . 42
3.16 Saveset-ы OpenVMS BACKUP . . . 43
3.17 VBACKUP в Windows . . . 44

Глава 4. Примеры **46**

Глава 5. Словарь команд **50**
5.1 Команда VBACKUP . . . 51
5.2 Квалификаторы . . . 55

Глава 6. Сообщения **76**
6.1 Формат сообщений . . . 76
6.2 Уровни серьёзности и код завершения . . . 77
6.3 Сообщения . . . 78

Глава 7. Устранение неполадок **96**
7.1 Сверка падает на занятых файлах . . . 97
7.2 Не восстанавливаются владельцы и атрибуты . . . 98
7.3 Saveset испорчен . . . 99
7.4 Тома нет или он от другого saveset-a . . . 100
7.5 Пароль спрашивается, не подходит или отсутствует . . . 101
7.6 Saveset на другом узле недоступен . . . 102
7.7 Инкрементное восстановление отвергнуто . . . 103
7.8 Устройство отвергнуто . . . 104
7.9 HELP показывает только сводку . . . 105
7.10 Файловый менеджер не открывает saveset . . . 106

Приложение A. Файлы **107**
A.1 Устанавливаемые файлы . . . 107
A.2 Используемые файлы . . . 108

Приложение B. Сводка формата saveset-a **109**
B.1 Соглашения . . . 110
B.2 Тома и блоки . . . 111
B.3 Заголовок блока (64 байта) . . . 112
B.4 Типы блоков . . . 113
B.5 Записи . . . 114
B.6 Теги . . . 115
B.7 Шифрование (format.md, 6.10) . . . 116
B.8 Журнал . . . 117
B.9 Saveset-ы OpenVMS BACKUP . . . 118

Приложение C. Автономный распаковщик vbkx **119**
C.1 Описание . . . 119
C.2 Формат . . . 120
C.3 Команды . . . 121
C.4 Ключи . . . 122
C.5 Что восстанавливается . . . 123
C.6 Диагностика . . . 124
C.7 Коды завершения . . . 125
C.8 vbkx.exe в Windows . . . 126
C.9 Распаковщики последней надежды: vbkx-go, vbkx-rs, vbkx-pl . . . 127

Приложение D. Модули файловых менеджеров **129**
D.1 Общие сведения . . . 129
D.2 Midnight Commander . . . 130
D.3 far2l и Far Manager 3 . . . 131
D.4 Total Commander и Double Commander . . . 132

Предисловие

Кому адресовано

Руководство предназначено системному администратору и оператору, которые сохраняют и восстанавливают файлы систем Linux. Предполагается знакомство с файловыми системами Linux, блочными устройствами, владельцами и правами доступа к файлам и с командным интерпретатором. Знание утилиты BACKUP системы OpenVMS полезно, но не обязательно.

Тому, кто хочет знать, из каких байтов состоит saveset, нужен документ `doc/format.md`; здесь дана только его сводка (приложение B).

Зачем нужен VBACKUP

Сорок лет утилита BACKUP системы OpenVMS показывала, как делается резервная копия: одна команда, два параметра, квалификатор на каждый выбор, контрольная сумма на каждом блоке, группа блоков, которая восстанавливает испорченный, сообщение на каждое состояние. Лента, записанная ею в одно десятилетие, читалась в следующем. Машины, на которых она работала, уходят из машинных залов; этот способ работы уходить вместе с ними не должен.

VBACKUP переносит его на Linux. Saveset-ы у него свои — под диски, файловые системы и атрибуты Linux, — но команда у него та же, что у BACKUP, сообщения имеют вид и уровни серьёзности OpenVMS, есть библиотека HELP, версии — это IDENT-ы, а это руководство устроено как руководства OpenVMS. Он читает и saveset-ы OpenVMS BACKUP, так что сохранённое в OpenVMS можно забрать в Linux.

Структура документа

Руководство состоит из следующих глав и приложений:

Часть	Содержание
Глава 1	Знакомит с VBACKUP: что он делает, что даёт saveset, чего он не делает, программы комплекта
Глава 2	Установка VBACKUP в Linux и в Windows
Глава 3	Описывает saveset и механизмы VBACKUP: структура saveset-а, спецификации файлов, инкрементное сохранение и журнал, `/PHYSICAL` и `/IMAGE`, сжатие, шифрование, каналы, saveset-ы на других узлах, кэш страниц, потоки, распаковщики, модули, расписание, saveset-ы OpenVMS BACKUP, VBACKUP в Windows
Глава 4	Содержит примеры операций VBACKUP
Глава 5	Содержит словарь команд: команда `vbackup`, её параметры, определение операции, код завершения, ограничения и все квалификаторы
Глава 6	Описывает вид сообщения и перечисляет все сообщения с тем, что с ними делать
Глава 7	Устранение неполадок: признак, на что смотреть и что делать
Приложение A	Перечисляет устанавливаемые и используемые файлы
Приложение B	Кратко излагает формат saveset-а
Приложение C	Описывает автономный распаковщик vbkx и распаковщики последней надежды
Приложение D	Описывает модули файловых менеджеров

Тому, кто ставит VBACKUP впервые, нужны главы 1, 2 и 4 — в этом порядке. Тому, у кого на руках сообщение, — глава 6, а у кого признак неполадки — глава 7.

Сопутствующие документы

- *vBACKUP saveset format, version 1* (`doc/format.md`) — определяющее описание байтов saveset-а и журнала. При расхождении настоящего руководства с этим документом в части байтов на носителе преимущество имеет `format.md`.
- *OpenVMS BACKUP savesets, as VBACKUP reads them* (`doc/vmsbackup.md`).
- *Проектный документ vBACKUP* (`doc/DESIGN.md`) — причины, стоящие за форматом и программой.
- Библиотека справки VBACKUP: `vbackup /HELP [тема ...]`; её исходные тексты — `doc/vbackuplib.md` (английский) и `doc/vbackuplib_ru.md` (русский).
- Страницы руководства `vbackup(1)` и `vbkx(1)`.
- Простые руководства, задача за задачей: `doc/simple/vbackup-ru.md`, `doc/simple/vbackup-en.md`, `doc/simple/vbackup-es.md`.
- Заголовки исходных текстов `tools/go/main.go`, `tools/rust/src/main.rs`, `tools/perl/vbkx.pl` и `plugins/wcx/vbkwcx.c` — это руководства этих программ.
- *OpenVMS System Management Utilities Reference Manual*, глава о BACKUP — утилите, за которой следует VBACKUP.

Благодарности

Авторы благодарят Claude — ИИ-модель компании Anthropic, — которая вместе с ними прошла через формат, код, тесты, замеры и эти руководства.

Авторы благодарят соратников из сообщества NaPi Lab, оплативших подписку на Claude, — благодаря им почти пенсионер смог продолжать эту работу.

Лицензия

VBACKUP — свободная программа на условиях Apache License, Version 2.0: её можно использовать, копировать, изменять и передавать другим, в том числе в составе других программ и за деньги, при условии, что вместе с ней передаются копия Лицензии и файл `NOTICE`, а изменённый файл говорит о том, что он изменён. Условия — в файле `LICENSE`, уведомления — в файле `NOTICE`; оба устанавливаются вместе с документами (приложение А).

Образы для Windows (`vbackup.exe`, `vbkx.exe`, модуль WCX) содержат, кроме VBACKUP:

Часть	Лицензия
StarLet (`utility_routines`), вкомпилированный в `vbackup.exe`	Apache License 2.0
winpthreads из MinGW-w64, скомпонованный статически	лицензия expat (проект mingw-w64) и лицензия BSD из 3 пунктов (Lockless Inc.)

Тексты этих лицензий — в `win/COPYING.MinGW-w64`, который передаётся с каждой копией образов. Сжатие (блоки LZ4, Deflate, LZMA) и криптография (ChaCha20, HMAC-SHA256, PBKDF2) — собственные реализации, написанные по опубликованным описаниям; кода других проектов в них нет.

Товарные знаки

OpenVMS, VMS, Alpha, Integrity, Digital и DEC — товарные знаки Hewlett Packard Enterprise Development LP или VMS Software, Inc. Linux — зарегистрированный товарный знак Линуса Торвальдса в США и других странах. Microsoft и Windows — товарные знаки группы компаний Microsoft. Total Commander — товарный знак Кристиана Гислера. Far Manager, Double Commander, Midnight Commander, far2l и Perl — названия их проектов. Go — товарный знак Google LLC. Rust — товарный знак Rust Foundation. Arm и Cortex — товарные знаки Arm Limited. Debian — зарегистрированный товарный знак Software in the Public Interest, Inc. GitHub — товарный знак GitHub, Inc. NaPi — название плат команды NaPi Lab. Claude — товарный знак Anthropic, PBC. Все прочие названия продуктов и компаний — товарные знаки их владельцев.

VBACKUP — независимая реализация. В нём нет кода OpenVMS, и он никак не связан с названными здесь владельцами, не одобрен и не поддерживается ими; их названия упомянуты только для того, чтобы сказать, с чем VBACKUP работает или за чем следует. VBACKUP не имеет отношения ни к пакету Debian `vbackup`, ни к программе VBackup для Windows — это другие продукты с тем же названием.

Соглашения

В руководстве приняты следующие соглашения:

Соглашение	Значение
ПРОПИСНЫЕ	Слова прописными буквами в формате — имена квалификаторов и ключевые слова. Их можно вводить прописными или строчными буквами и сокращать, пока сокращение однозначно.
курсив	Слова курсивом в формате и в текстах сообщений обозначают значение, которое задаёт пользователь или выводит система.
[ ]	В формате квадратные скобки заключают необязательную часть. Скобки не вводятся. В записи /[NO]VERIFY они показывают, что квалификатор допускает отрицание.
(,)	В формате круглые скобки заключают список значений через запятую. При нескольких значениях скобки обязательны.
...	В формате многоточие означает, что предыдущий элемент может повторяться. В спецификации файла ... — шаблон для любого числа уровней каталогов.
моноширинный	Моноширинным шрифтом показано то, что вводит пользователь и что выводит система.
$	Приглашение командного интерпретатора обычного пользователя.
#	Приглашение командного интерпретатора root: команде нужны привилегии root, либо root подразумевается.
Ctrl/*x*	Удерживая клавишу Ctrl, нажать клавишу *x*.
%VBACKUP-s-IDENT	Сообщение средства VBACKUP; *s* — его уровень серьёзности (S, I, W, E, F).

Каждое сообщение утилиты начинается с даты, времени и номера процесса:

```
05-10-2026 14:19:25.787 2214682 %VBACKUP-I-STARTED, Operation: save, Input: /home/rrl, Output: rrl.bck -
started
```

Примечание

> В примерах руководства дата, время и номер процесса опущены, а длинные имена каталогов системы, на которой выполнялись примеры, сокращены до имён вида /home, /backup и /mnt/usb. Числа и тексты приведены так, как их выводит VBACKUP X01-11 и более поздние.

Числа десятичные, если не оговорено иное. Размеры с суффиксами K, M, G и T двоичные: 1K — 1024 байта.

Глава 1. Введение

1.1 Что делает VBACKUP

VBACKUP сохраняет файлы и каталоги файловых систем Linux в *saveset*, восстанавливает их из него, выводит его листинг и сравнивает его с диском. Он также копирует файлы с диска на диск, копирует блочное устройство целиком, блок за блоком, сохраняет и заново создаёт файловую систему целиком и ведёт журнал для инкрементного сохранения. Он следует утилите BACKUP системы OpenVMS: одна команда, два параметра, и операция определяется тем, что это за параметры.

```
$ vbackup /home/rrl /backup/rrl.bck /VERIFY
$ vbackup /backup/rrl.bck /LIST
$ vbackup /backup/rrl.bck /tmp/restore
```

Первая команда сохраняет каталог `/home/rrl` и сверяет saveset с диском; вторая выводит его листинг; третья восстанавливает его в `/tmp/restore`.

1.2 Что даёт saveset

- Файлы с владельцами, правами, временами с точностью до наносекунды, расширенными атрибутами, ACL POSIX, capabilities и флагами chattr; жёсткие и символические ссылки, разреженные файлы, FIFO и устройства.
- Контрольную сумму каждого блока и каждого файла: порча всегда обнаруживается.
- Блоки XOR (`/GROUP_SIZE`): один испорченный блок группы восстанавливается; с `/PARITY=`*m* (Рида — Соломона) — любые *m* из них. После потери чтение подхватывает поток со следующего целого блока, и страдают только файлы, лежавшие в потерянных блоках.
- Тома (`/VOLUME_SIZE=4G`), каждый из которых опознаёт себя сам, — на диске, через канал или сразу на другом узле (`vbackup /home host::/backup/home.bck`).
- Каталог в конце: `/LIST` и `/EXTRACT` мгновенны.
- Инкрементные сохранения: `/RECORD` ведёт журнал, `/SINCE=BACKUP` сохраняет изменённое с тех пор, а цепочка, восстановленная с `/INCREMENTAL`, возвращается без удалённых файлов.
- Сжатие на всех ядрах: LZ4, Deflate или LZMA, мелкие файлы упакованы вместе (SOLID), каждая запись проверяется до записи.
- Шифрование (`/ENCRYPT`): ChaCha20 и HMAC-SHA256 на каждый блок, ключи через PBKDF2; имена файлов тоже скрыты, а порча исправляется и без пароля.
- Saveset, который можно прочитать там, где VBACKUP не установлен: статически собранным `vbkx` и тремя распаковщиками последней надежды на Go, Rust и Perl.

1.3 Чего VBACKUP не делает

- Он не ведёт расписание и не ротирует saveset-ы. Это работа пакетной очереди — BATCH или cron, — которая запускает командную процедуру, запускающую VBACKUP (раздел 3.15).
- Он никогда не изменяет saveset на месте: в существующий saveset нельзя ничего добавить и из него нельзя ничего удалить.
- Saveset-ы OpenVMS BACKUP он читает, но не пишет, и не читает зашифрованные BACKUP-ом (DES) и данные их `/IMAGE` как тома (раздел 3.16).
- Он не хранилище дедуплицированных кусков: каждый saveset содержит все свои данные и читается сам по себе.
- Он не замораживает файловую систему. О файле, изменившемся во время сохранения, сообщается (FILCHANGED); для согласованной копии занятой системы сохраняйте её снимок.

1.4 Программы комплекта

Программа	Что это
`vbackup`	Утилита: все операции этого руководства
`vbkx`	Автономный распаковщик: листинг, извлечение и проверка saveset-а; собран статически (приложение C)
`vbkx-go`, `vbkx-rs`, `vbkx-pl`	Распаковщики последней надежды на Go, Rust и Perl, собираются из исходных текстов в `tools/` (приложение C.9)
`vbackup.exe`, `vbkx.exe`	Утилита и распаковщик в Windows (раздел 3.17)
Модули	Midnight Commander, far2l и Far Manager, Total Commander и Double Commander открывают saveset как архив (приложение D)

Глава 2. Установка

2.1 Требования

Что	Требование
Операционная система	Linux, x86_64 или aarch64
Компилятор	Компилятор C11 (GCC)
Сборка	CMake 3.16 или новее, make
Библиотеки	StarLet 1.6 или новее (обязательно); HELP 1.0 или новее (необязательно: `/HELP` и библиотеки справки)
Необязательно	Go и Rust — для `vbkx-go` и `vbkx-rs`; MinGW-w64 и wine — для `vbkx.exe` и `vbackup.exe`; Midnight Commander или far2l — для их модулей

Во время работы кроме библиотеки C ничего не нужно: VBACKUP не использует ни библиотек сжатия, ни криптографических библиотек.

2.2 Сборка и установка

Сначала установите StarLet (и HELP) из их комплектов. Затем:

```
# tar xzf vbackup-X01-25.tar.gz
# cd vbackup-X01-25
# cmake -S . -B build
# cmake --build build -j
# (cd build && ctest)
# cmake --install build
```

Все тесты должны пройти; упавший тест — повод не устанавливать. При `TAP=1` тесты говорят на языке TAP — для prove и систем CI.

`make kit` в дереве сборки делает `vbackup-`*ident*`.tar.gz`; в дереве git он делается через git archive от HEAD, так что в него попадает только закоммиченное.

Параметры сборки:

Параметр	По умолчанию	Действие
`VBACKUP$M_HELPAPI`	ON	Отвечать на `/HELP` из библиотеки справки, если найден HELP
`VBACKUP$M_HELPLIB`	ON	Транслировать и устанавливать библиотеки справки
`VBACKUP$M_INSTALL_DOC`	ON	Устанавливать страницы руководства и документы
`VBACKUP$M_PLUGINS`	ON	Устанавливать модули там, где найдены Midnight Commander и far2l
`VBACKUP$M_VBKXSTATIC`	ON	Собирать vbkx статически, если система это позволяет
`VBACKUP$M_GEEKS`	ON	Собирать vbkx-go и vbkx-rs, если найдены Go и Rust
`VBACKUP$M_WINVBKX`	ON	Собирать vbkx.exe с MinGW-w64 и проверять его под wine, если они найдены
`VBACKUP$M_PDF`	OFF	Делать PDF этого руководства (нужны pandoc и XeLaTeX)
`VBACKUP$M_STRICT`	OFF	Считать предупреждения компилятора ошибками

2.3 Установка в Windows

Образы для Windows собирает GitHub Actions из каждого тега и публикует с выпуском: `vbackup-`*тег*`-win64.zip` содержит `vbackup.exe`, `vbkx.exe`, модуль Total Commander (`vbackup.wcx64`, `vbackup.wcx`, `pluginst.inf`), `LICENSE`, `NOTICE` и `COPYING.MinGW-w64`, а `SHA256SUMS` перечисляет контрольные суммы. Распакуйте архив в каталог из PATH; больше ничего не устанавливается. Те же файлы вручную делает `tools/mkrelease.sh`.

2.4 Установка в другое место

Префикс по умолчанию — `/usr/local`. Другой задаётся CMake:

```
# cmake -S . -B build -DCMAKE_INSTALL_PREFIX=/opt/vbackup
```

2.5 Проверка установки

```
$ vbackup /HELP
$ vbackup /usr/local/share/doc/vbackup /tmp/doc.bck /VERIFY
$ vbkx t /tmp/doc.bck
```

Первая команда показывает темы библиотеки справки; собранный без HELP, VBACKUP показывает вместо них сводку, встроенную в образ. Вторая сохраняет установленные документы и сверяет saveset с диском: она должна закончиться `%VBACKUP-I-COMPLETED` и кодом завершения 0. Третья проверяет тот же saveset автономным распаковщиком.

2.6 Удаление

```
# cmake --build build --target uninstall
```

удаляет все файлы, поставленные установкой, и точно, между метками, вынимает разделы модулей из файлов Midnight Commander и far2l. Saveset-ы и журналы не затрагиваются никогда.

Глава 3. Основные понятия

Утилита VBACKUP сохраняет файлы и каталоги файловых систем Linux в *saveset*, восстанавливает их из него, выводит его содержимое и сравнивает его с диском. Кроме того, она копирует файлы с диска на диск, копирует блочное устройство поблочно, сохраняет и заново создаёт целую файловую систему и ведёт журнал для инкрементного сохранения. VBACKUP следует утилите BACKUP системы OpenVMS: одна команда, два параметра, и операция определяется тем, что представляют собой параметры.

VBACKUP не требует в системе ничего, кроме библиотеки C; saveset-ы можно прочитать и без VBACKUP — автономным распаковщиком vbkx и распаковщиками последней надежды (приложение C).

3.1 Saveset-ы

Saveset — это файл или набор файлов, называемых *томами*, содержащий копии файлов вместе со всем, что нужно для их возврата:

- данные обычных файлов, разреженные файлы с их дырами;
- каталоги, символические ссылки, жёсткие ссылки, FIFO, файлы символьных и блочных устройств;
- владелец и группа — по номеру и по имени;
- режим доступа, включая биты setuid, setgid и sticky;
- времена изменения, доступа, смены атрибутов и создания, с наносекундами;
- расширенные атрибуты, списки POSIX ACL, возможности (capabilities) файлов и метка SELinux;
- флаги файловой системы `chattr` (FS_IOC_GETFLAGS).

Помимо файлов, saveset содержит сводку сохранения (продукт, узел, пользователь, команда, время, базы входных спецификаций, размеры блока и группы, операционная система), контрольную сумму каждого блока и каждого файла, блоки XOR для починки и каталог своих файлов.

На входной стороне VBACKUP распознаёт saveset по содержимому, а не по имени. На выходной стороне сохранения saveset распознаётся по имени (оканчивается на `.bck` или `.sav`), по квалификатору `/SAVE_SET` или по имени `-` (стандартный вывод). Saveset на другом узле, *узел*::*файл*, — saveset на любой стороне (раздел 3.10).

3.2 Операции

Операция определяется параметрами и несколькими квалификаторами (см. раздел 5.1):

Вход	Выход	Операция
файлы	saveset	**Сохранение**: файлы записываются в новый saveset.
saveset[,...]	каталог	**Восстановление**: файлы saveset-а создаются в каталоге.
saveset	— (с `/ORIGINAL`)	**Восстановление** каждого файла в каталог, из которого он был сохранён.
saveset	— (с `/LIST`)	**Листинг**: выводятся сводка и файлы saveset-а.
saveset	каталог (с `/COMPARE`)	**Сравнение**: каждый файл saveset-а сравнивается с диском.
saveset	[файл] (с `/EXTRACT`)	**Извлечение**: один файл записывается в файл или на стандартный вывод.
файлы	каталог (не saveset)	**Копирование**: файлы копируются с диска на диск.
saveset[,...]	— (с `/RECORD`)	**Перестройка журнала**: журнал создаётся заново по каталогам saveset-ов.
—	— (с `/JOURNAL /LIST`)	**Листинг журнала**: известные журналу saveset-ы и файлы.
saveset	saveset	**Копирование saveset-а**: блоки копируются как есть, том за томом (`/TRANSFER`).

Сохранение может также проверить новый saveset (`/VERIFY`), вывести его листинг (`/LIST`), записать его в журнал (`/RECORD`) и удалить сохранённые и проверенные файлы (`/DELETE`).

Saveset сохранения, восстановления, листинга, сравнения, извлечения или копирования saveset-а может быть также `-`, каналом (раздел 3.9), или *узел*`::`*файл*, saveset-ом на другом узле (раздел 3.10).

3.3 Структура saveset-а

Блоки

Saveset — последовательность блоков одного фиксированного размера, *размера блока* (`/BLOCK_SIZE`, от 8192 до 1048576 байт, кратен 512; по умолчанию 65536). Каждый блок начинается 64-байтным заголовком, который содержит идентификатор saveset-а (случайный UUID), номер блока, номер тома, тип блока и CRC-32 заголовка и полезной нагрузки. Блоки нумеруются с 0 непрерывно через все тома. Первый блок каждого тома — *заголовок тома* (VHDR) со сводкой saveset-а, так что каждый том сам указывает, что он такое и к какому saveset-у относится.

Полезные нагрузки блоков данных, взятые по порядку, образуют один поток записей: запись SUMMARY, затем для каждого файла запись FILE, его записи DATA (или сжатые DATAZ) и запись FEND с размером, CRC данных и состоянием файла; затем записи CATALOG и запись END. Запись может пересекать границы блоков и томов.

CRC

Каждый блок несёт CRC-32 (IEEE) своего заголовка и нагрузки; каждый файл несёт CRC-32 своих данных в записи FEND и в элементе каталога. Повреждённый блок при чтении saveset-а обнаруживается всегда; файл, данные которого не совпадают с его CRC, сообщается (CRCERR) и никогда молча не восстанавливается как исправный.

Группы XOR и починка

После каждых *n* блоков данных (*n* — *размер группы*, `/GROUP_SIZE`, от 0 до 100, по умолчанию 10) VBACKUP пишет один блок XOR — побайтовое XOR нагрузок блоков данных группы. Если при чтении saveset-а в группе испорчен ровно один блок данных, он восстанавливается из остальных блоков группы и блока XOR и выводится сообщение BLKFIXED; ничего не теряется. Если испорчены два блока группы или больше, о них сообщается (BLKLOST); чтение продолжается с первой записи, начинающейся в следующем исправном блоке, и страдают только файлы, данные которых лежали в потерянных блоках (FILDAMAGED, FILLOST). `/GROUP_SIZE=0` не пишет блоков XOR, и ничего нельзя починить. Блоки XOR при размере группы по умолчанию увеличивают saveset примерно на 10%.

Тома и их имена

`/VOLUME_SIZE` разрезает saveset на тома фиксированного размера. Том 1 имеет имя, заданное в команде; том *k* (*k* >= 2) — это имя, точку и *k* в десятичной записи, не менее трёх цифр:

```
home.bck  home.bck.002  home.bck.003  ...
```

Размер тома округляется вниз до кратного размеру блока; каждый том, кроме последнего, имеет ровно этот размер. Группа блоков никогда не пересекает границу тома. Чтобы прочитать saveset, укажите имя его первого тома; остальные VBACKUP найдёт рядом. Об отсутствующем томе сообщается (MISSVOL); файлы, целиком лежащие в остальных томах, всё равно восстанавливаются. Файл с именем тома, принадлежащий другому saveset-у, пропускается (WRONGVOL).

Saveset, записываемый в канал, несёт свои тома подряд, каждый со своим VHDR, а номера блоков идут через них сквозь (раздел 3.9); копирование такого saveset-а в файлы раскладывает его по файлам томов с указанными выше именами.

Каталог

В конце потока VBACKUP пишет каталог: по элементу на каждый файл, с его именем, типом, владельцем, режимом, размером, временами, целью ссылки, CRC, состоянием и местом (том, блок, смещение), где начинается его запись FILE. `/LIST` читает только каталог, поэтому даже очень большой saveset выводится сразу; `/EXTRACT` добирается до одного файла по его месту в каталоге, не читая ничего другого. Каталог инкрементного saveset-а перечисляет и файлы, которые им охвачены, но не сохранены, с состоянием PRESENT (см. раздел 3.5).

Ничего не пропадает молча. Начиная с X01-22 восстановление, `/COMPARE`, `/VERIFY` и команды `x` и `t` vbkx всегда сверяют каталог с встреченными записями FILE — а не только после потерянных блоков. Файл каталога, записи которого так и не пришли, называется: FILLOST, если блоки были потеряны, NOTINSTREAM, если не был потерян ни один (запись, которую читающий не знает, неправильно сделанный saveset); `/COMPARE` и `/VERIFY` считают его различием (COMPARERR). Из канала записи каталога берутся по мере поступления.

TRAILER

Последний блок saveset-а — TRAILER. В нём итоги сохранения (файлы, байты, ошибки, блоки, тома, элементы каталога) и место начала каталога. Saveset без TRAILER — сохранение прервано, диск переполнился или отсутствует последний том — всё равно читается последовательно до последнего исправного блока; VBACKUP сообщает NOTRAILER, а `/LIST` читает весь saveset (NOCATALOG).

3.4 Спецификации файлов и шаблоны

Входная спецификация сохранения или копирования — имя файла или каталога, возможно с шаблонами. VBACKUP раскрывает шаблоны сам; заключайте их в кавычки, чтобы их не раскрыл командный интерпретатор:

Шаблон	Соответствует
`*`	любые символы в пределах одного компонента имени
`% ?`	ровно один символ
`...`	любое число уровней каталогов, в том числе ни одного (как целый компонент)

База спецификации — часть до первого компонента, содержащего шаблон или `...`; без шаблонов база — каталог, содержащий названный файл или каталог. Имена, хранимые в saveset-е, — *хранимые имена* — задаются относительно базы. Поэтому каталог, названный в спецификации, хранится со своим именем впереди:

Спецификация	База	Хранимые имена
`/home/rrl`	`/home`	`rrl`, `rrl/...`
`/home/rrl/src/.../*.c`	`/home/rrl/src`	`a.c`, `lib/b.c`
`/etc/*.conf`	`/etc`	`host.conf`, …
`/mnt/data/.`	`/mnt/data/.`	содержимое `/mnt/data` под собственными именами

Каталог, выбранный спецификацией, берётся со всем содержимым. Скрытые файлы (имена, начинающиеся с точки) тоже подходят под шаблоны. Элементы каталога обходятся в байтовом порядке их имён, так что два сохранения одного дерева дают одинаковое расположение saveset-а. Базы записываются в сводку как абсолютные имена (realpath); ими пользуются `/ORIGINAL` и `/COMPARE` без выходного параметра.

Шаблоны `/SELECT` и `/EXCLUDE` сопоставляются с хранимыми именами. В шаблоне `*` соответствует любым символам, включая `/`, а `%` и `?` — одному символу.

Слово команды, начинающееся с косой черты, является квалификатором, только если оно называет квалификатор; `/etc` и `/home` — имена файлов. Спецификация с шаблоном никогда не принимается за saveset.

3.5 Инкрементное сохранение и журнал

VBACKUP различает в saveset-е два множества файлов (`format.md`, 6.6):

- *охваченное* множество: все элементы, выбранные входными спецификациями и фильтрами имён — `/SELECT`, `/EXCLUDE`, флагом nodump, `/NOCROSS_DEVICE` – и квалификатором `/BY_OWNER`;
- *сохранённое* множество: та часть охваченного, данные которой есть в saveset-е. В *полном* saveset-е это само охваченное множество. В *инкрементном* его выбирают фильтры времени — `/SINCE`, `/BEFORE`, `/SINCE=BACKUP`.

Любой фильтр времени делает saveset инкрементным. Его каталог перечисляет всё охваченное множество: сохранённые файлы с их состоянием, остальные – как PRESENT. Тем самым каталог описывает дерево таким, каким оно было в момент сохранения, и восстановление `/INCREMENTAL` может удалить из каталога то, чего в нём уже не было. Каталоги сохраняются всегда.

Журнал — отдельный файл вне любого saveset-а. Сохранение с `/RECORD` записывает в него saveset и, для каждого файла, сохранённого с состоянием OK, его абсолютное имя, inode, размер, время изменения и время смены атрибутов. Сохранение с `/SINCE=BACKUP` сохраняет файл, если журнал его не знает или если его inode, время смены атрибутов, время изменения или размер отличаются от записанных; номер устройства записывается, но не сравнивается. Файл, сохранённый в состоянии CHANGED или с ошибкой чтения, не записывается, поэтому следующее `/SINCE=BACKUP` сохранит его снова. При `/VERIFY` журнал обновляется, только если проверка не нашла различий.

Журнал — `/var/lib/vbackup/vbackup.jnl` для root и `~/.vbackup/vbackup.jnl` для остальных пользователей; `/JOURNAL=файл` задаёт другой. Он переписывается целиком под исключительной блокировкой (`журнал.lock`) через `журнал.tmp`. Потерянный или испорченный журнал можно перестроить по каталогам saveset-ов (`saveset[,...] /RECORD`); журнал — удобство, а не единственная копия чего бы то ни было.

Цепочка — полный saveset, затем инкрементные в порядке их создания – восстанавливается с `/INCREMENTAL`. Заметьте: OpenVMS BACKUP восстанавливает инкрементную цепочку начиная с новейшего saveset-а; VBACKUP — начиная со старейшего.

3.6 Физические операции и операции над образом

/PHYSICAL

Сохранение `/PHYSICAL` копирует каждый блок одного блочного устройства – раздела или целого диска — или файла образа, что бы на нём ни было: любую файловую систему, зашифрованный том, загрузочные области. Saveset содержит один обычный файл — образ устройства, названный по узлу устройства; куски по 65536 байт, состоящие из одних нулей, не записываются. VBACKUP отвергает устройство, смонтированное на чтение и запись (или раздел которого смонтирован так), и устройство, занятое (удерживаемое LVM, RAID, dm-crypt или используемое как swap). Устройство, смонтированное только на чтение, принимается.

Восстановление `/PHYSICAL` записывает образ на устройство или в файл образа. На устройство оно требует `/REPLACE`, отвергает смонтированное или занятое устройство и устройство меньше сохранённого, пишет нули в промежутки и на терминале спрашивает `YES`. На устройство большего размера записывается начальная часть, остальное остаётся как было. Копия несёт метки и UUID оригинала (PHYSUUID): никогда не монтируйте оба одновременно.

Saveset `/PHYSICAL`, восстановленный без `/PHYSICAL`, — это разреженный файл образа в выходном каталоге; так его восстанавливает любой распаковщик.

/IMAGE

Сохранение `/IMAGE` сохраняет каждый файл одной смонтированной файловой системы — ничего из файловых систем, смонтированных ниже, флаги nodump игнорируются — под именами относительно её корня и записывает то, что делает том этим томом: тип, метку, UUID, занятое место, параметры монтирования и владельца, режим, времена и ACL её корневого каталога. Вход – точка монтирования или устройство, если оно смонтировано (достаточно только на чтение).

Восстановление `/IMAGE` на устройство создаёт новую файловую систему сохранённого типа, с той же меткой и UUID, запуская `mkfs.ext2`, `mkfs.ext3`, `mkfs.ext4`, `mkfs.xfs`, `mkfs.btrfs` или `mkfs.vfat` (программа должна быть установлена), монтирует её во временный каталог с `nosuid,nodev,noexec`, восстанавливает в неё файлы, придаёт её корню сохранённые атрибуты и размонтирует. Защиты — те же, что у `/PHYSICAL`; устройство должно вмещать занятое место плюс 5% плюс 16 МБ и может быть меньше или больше сохранённого. `/IMAGE` не копирует ни загрузочные сектора, ни таблицу разделов, ни номера inode; для этого используйте `/PHYSICAL` целого диска.

Saveset `/IMAGE`, восстановленный без `/IMAGE`, — обычное дерево файлов.

3.7 Сжатие

`/DATA_FORMAT=COMPRESSED` хранит данные файлов в записях DATAZ; `/LEVEL=n` задаёт, насколько сильно, и сжимает сам по себе:

/LEVEL	Кодек	Текст (запись 1 МБ)	Двоичные	Сохранение, одно ядро	Восстановление, одно ядро
1 (по умолчанию)	LZ4	2.1x	1.5x	130 МБ/с	400 МБ/с
2 .. 5	Deflate	3.2x .. 3.7x	1.9x .. 2.1x	33 .. 11 МБ/с	110 .. 230 МБ/с
6, 7	LZMA, быстрый разбор	4.0x .. 4.2x	2.3x	12 .. 8 МБ/с	30 .. 60 МБ/с
8, 9	LZMA, оптимальный разбор	4.8x .. 4.85x	2.4x	1.9 .. 1.7 МБ/с	30 .. 65 МБ/с

Степени сжатия измерены на этой машине на записях по 1 МБ из дерева исходников vBACKUP и из двоичных программ; скорость — одного ядра, вместе с проверкой каждой записи: сжатие идёт в нескольких потоках (раздел 3.12), и saveset от их числа не зависит. Восстановление быстрое на любом уровне: для LZ4 и Deflate быстрее большинства дисков, для LZMA – десятки МБ/с. Уровни 8 и 9 выбирают самое дешёвое кодирование каждого участка данных по ценам модели (оптимальный разбор, как в xz) и сжимают как `xz -6` на записи 1 МБ; сохранение на них медленное, как у xz, а восстановление такое же быстрое, как на 6 и 7.

Мелкие файлы: записи SOLID

Мелкие файлы по одному сжимаются плохо: каждый начинается с пустого словаря, а его записи FILE и FEND не сжимаются вовсе. Поэтому начиная с X01-21 сжимающее сохранение собирает мелкие обычные файлы — каждый не больше 256 КБ и не второе имя жёсткой ссылки — в группы до 256 КБ (в X01-21 — до 1 МБ), в порядке обхода, и сжимает каждую группу одной записью SOLID (format.md 6.12): записи FILE, DATA и FEND её файлов вместе, так что словарь общий, и метаданные тоже сжимаются. Каталог, ссылка или больший файл сначала закрывают открытую группу. Так делается по умолчанию всякий раз, когда сохранение сжимает; `/NOSOLID` пишет каждый файл своими записями, как раньше. Замеры на этой машине (размеры saveset-ов):

Дерево	/LEVEL	/NOSOLID	/SOLID, X01-21 (группы 1 МБ)	/SOLID, X01-22 (256 КБ, по умолчанию)
`/usr/include` (210 МБ, 22 444 файла)	1	81 МБ	62 МБ	62 МБ
`/usr/include`	2	60 МБ	44 МБ	45 МБ
`/usr/include`	5	55 МБ	39 МБ	39 МБ
`/usr/include`	6	56 МБ	36 МБ	37 МБ
`/usr/include`	9	53 МБ	32 МБ	33 МБ
`/usr/lib/python3` (512 МБ)	1	269 МБ	254 МБ	255 МБ
`/usr/lib/python3`	2	200 МБ	186 МБ	187 МБ
`/usr/lib/python3`	5	187 МБ	172 МБ	173 МБ

Дерево	/LEVEL	/NOSOLID	/SOLID, X01-21 (группы 1 МБ)	/SOLID, X01-22 (256 КБ, по умолчанию)
`/usr/lib/python3`	6	172 МБ	146 МБ	152 МБ
`/usr/lib/python3`	9	163 МБ	135 МБ	141 МБ

(– не замерялось.) Скорость сохранения и восстановления в обоих случаях примерно одна и та же. Большие файлы сжимаются по записям, как раньше. X01-22 делает группы вчетверо меньше, чем X01-21: saveset на несколько процентов больше, а в плохой день теряется намного меньше (см. ниже).

Чего это стоит в день восстановления. При чётности по умолчанию ничего не меняется: один плохой блок группы по-прежнему восстанавливается (любые *m* при `/PARITY=m`). Но блоки, потерянные без возможности починки — больше *m* плохих блоков в одной группе, — уносят больше, чем тот файл, в котором были: файлы разрезанной ими записи SOLID начиная с места повреждения и все файлы записи SOLID, которая в них начинается. Часть записи SOLID до разреза не теряется (начиная с X01-22): она распаковывается настолько, насколько это в точности верно, и файлы, целиком лежащие в ней, восстанавливаются, каждый проверяется по CRC своей записи FEND; файл на разрезе восстанавливается настолько, насколько дошёл (FILDAMAGED), файлы после него называются в FILLOST. То же относится к концу saveset-а, сохранение которого было прервано. Остальные файлы восстанавливаются. Файлы, теряемые на одну группу чётности, повреждённую без возможности починки, замер на `/usr/lib/python3/dist-packages`:

Saveset	/LEVEL=6	/LEVEL=1
`/NOSOLID`	19	11
`/SOLID`, X01-21 (группы 1 МБ, без спасения)	52	45
`/SOLID`, X01-22 (группы 256 КБ, со спасением)	24	22

То есть от нескольких до пары десятков мелких файлов, а не один. Спасение — дело читающего: VBACKUP X01-22 делает его и для saveset-ов X01-21. Для saveset-а, который должен пережить сильные повреждения, задайте `/PARITY=2` или больше (см. `/PARITY` в разделе 5.2) либо `/NOSOLID`.

Запись SOLID, которая не распаковывается вовсе или содержит бессмысленную запись, считается потерянными блоками (BADREC, затем FILLOST для её файлов) — никогда не неверными данными. Пишущий распаковывает каждую запись SOLID и сравнивает её до записи; группа, не вернувшаяся такой же (ZCHECK) или не уменьшившаяся, пишется обычными записями.

Записи и проверки

Каждая запись DATA большого файла сжимается отдельно (format.md 6.7): плохой блок стоит только этой записи, и ремонт по XOR и чётности работает как для несжатых данных — CRC файла считается по исходным байтам, CRC блока – по байтам, как они лежат в потоке. Для каждой записи VBACKUP сначала пробует первые 64 КБ и хранит запись несжатой, если она не уменьшается хотя бы на 3%; уже сжатые данные (фотографии, видео, `.gz`, `.zip`) почти не стоят времени. Запись DATAZ пишется только если она меньше, чем была бы запись DATA.

Восстановимо в день, когда понадобится. Каждая запись, которую VBACKUP сжал, тут же распаковывается и сравнивается с данными; запись, не вернувшаяся такой же, пишется несжатой, и об этом говорит ZCHECK. Saveset никогда не зависит от того, прав ли компрессор. Каждый кодек читают четыре независимых декодера — ядро VBACKUP (vbackup, vbkx, vbkx.exe, модуль WCX), vbkx-go, vbkx-rs, vbkx-pl,

— а форматы общедоступные (сырой Deflate RFC 1951, сырой LZMA1) и описаны в `format.md` достаточно, чтобы написать пятый; их читают zlib и liblzma.

VBACKUP и vbkx до X01-04 не читают сжатые данные, до X01-19 — уровни 2..9; они называют файлы повреждёнными (CRCERR, FILDAMAGED, BADREC) и никогда молча не пишут неверных данных. Сжатый saveset X01-21 и новее, если он сохранён не с `/NOSOLID`, имеет формат версии 3 в каждом блоке – даже если все группы случайно записались обычными записями; VBACKUP, vbkx и модули до X01-21 отвергают его целиком (NOTSAVESET) и никогда не читают неверно. Читайте его X01-21 или новее либо сохраняйте с `/NOSOLID` для старых читающих. Меньшие группы X01-22 — та же версия 3: X01-21 их читает, а X01-22 читает saveset-ы X01-21.

3.8 Шифрование

`/ENCRYPT` создаёт зашифрованный saveset (`format.md`, 6.10). Нагрузки блоков данных и TRAILER шифруются ChaCha20 и аутентифицируются HMAC-SHA256, каждый блок — 32-байтным тегом. Ключи выводятся из пароля функцией PBKDF2-HMAC-SHA256, по умолчанию 600000 итераций, со случайной 32-байтной солью, новой для каждого saveset-а. Криптографическая библиотека не используется: vbkx и распаковщики последней надежды читают зашифрованные saveset-ы без чего-либо ещё.

Ключи и KEYCHECK

Из пароля и соли VBACKUP выводит главный ключ, а из него — три ключа: для шифрования, для аутентификации и контрольное значение. Контрольное значение хранится в заголовке тома как KEYCHECK. Читающий выводит ключи и сравнивает контрольное значение, прежде чем читать что-либо ещё: несовпадение означает неверный пароль (WRONGKEY), а не повреждение saveset-а.

Что остаётся видимым

Заголовки блоков, CRC и блоки XOR вычисляются по шифротексту, поэтому повреждённый зашифрованный saveset проверяется и чинится без пароля. Без пароля видны: размер блока, размер группы, размер тома и число томов; длина потока и места начала записей в каждом блоке; время сохранения — лишь по временам файлов томов. Не видны: имена, размеры, атрибуты и содержимое файлов, узел, пользователь, команда и итоги TRAILER.

Блок с верным CRC, но неверным тегом изменён умышленно (или очень странным носителем): о нём сообщается (BLKFORGED), и он считается плохим блоком – чинится по группе, когда это возможно, иначе теряется.

Обращение с паролем

Пароль никогда не берётся из командной строки: команда хранится в saveset-е и видна другим пользователям через `ps`. Он получается в таком порядке:

1. из файла ключа, заданного `/KEY_FILE=файл`;
2. из файла ключа, названного переменной окружения `VBACKUP_KEY_FILE`;
3. с терминала (`/dev/tty`), без эха; при сохранении он запрашивается дважды, и ответы должны совпасть (иначе KEYMATCH, и ничего не сохраняется).

Если нет ни файла ключа, ни терминала (cron, канал, файловый менеджер) или задано `VBACKUP_NOPROMPT=1`, VBACKUP завершается с NOKEY; он никогда не ждёт. Пароль запрашивается один раз на команду и сохраняется для последующих saveset-ов (проверки после сохранения, цепочки восстановления `/INCREMENTAL`); при выходе и по Ctrl/C он стирается из памяти, и эхо терминала восстанавливается.

Файлы ключа

Пароль — первая строка файла ключа без признака конца строки (LF или CR LF), байт в байт; не длиннее 1024 байт и не пустая. Файл ключа должен быть обычным файлом, доступным на чтение и запись только владельцу (режим 600); иначе он отвергается (KEYFILE).

Пароль *и есть* ключ: saveset, пароль которого утерян, не откроет никто. Используйте длинный пароль — пять или более случайных слов. Журнал не шифруется: это файл сохраняющей системы.

Открытие зашифрованного saveset-а стоит вычисления PBKDF2: около 0,5 с на ПК, 1,6 с на небольшой плате ARM с инструкциями SHA, 6 с — без них. Эта цена намеренна: её платит и каждая попытка подбора.

VBACKUP и vbkx версий до X01-06 не читают зашифрованные saveset-ы; они сообщают о потере всех блоков и ничего не записывают.

3.9 Каналы

Имя - означает стандартный вывод как выход сохранения или копирования saveset-а и стандартный ввод как вход восстановления, /LIST, /COMPARE, /EXTRACT и копирования saveset-а:

```
$ vbackup /home - | ssh host 'vbackup - /backup/home.bck'
$ ssh host 'vbackup /home -' | vbackup - /restore
```

Поток из нескольких томов

Начиная с X01-11 saveset, записываемый в -, может состоять из нескольких томов (/VOLUME_SIZE): они идут на стандартный вывод подряд, каждый со своим VHDR, а номера блоков идут через них сквозь, как через файлы томов. О каждом томе после первого сообщается (CREATED, `Volume: (standard output)`). Читатель потока принимает VHDR следующего тома как конец текущего; о томе, пропущенном в потоке, сообщается (MISSVOL), и файлы, целиком лежащие в остальных томах, всё равно восстанавливаются.

Saveset, записываемый в -, после записи уже не прочитать: /VERIFY (а значит, и /DELETE) и /LIST с ним отвергаются (QUALUSE: `a saveset written to the standard output is gone once written: it cannot be read back here`).

Saveset из - читается один раз и только вперёд. Каталог в его конце бесполезен: /LIST выводит записи FILE по мере поступления, /EXTRACT ищет свой файл чтением, vbkx отвергает имена. Плохие блоки чинятся так же, как в файле. Канал, оборвавшийся до TRAILER, даёт NOTRAILER.

Приёмник

Если вход — saveset и выход — тоже saveset, VBACKUP копирует saveset блок за блоком (/TRANSFER). С - на одной из сторон это соединяет канал и файлы томов:

```
$ vbackup home.bck - | ...          тома подряд -- в канал
$ ... | vbackup - home.bck          поток -- в home.bck, home.bck.002, ...
```

Приёмник, `vbackup -`*saveset*, раскладывает поток по файлам томов на каждом VHDR, с именами из раздела 3.3: saveset на его диске — тот самый, что был отправлен, том в том и байт в байт. Каждый блок проверяется по прибытии. Плохой блок копируется как есть, и о нём сообщается (BLKCOPIED): его починит по группе восстановление; о потоке, кончившемся до своего TRAILER, сообщается (NOTRAILER). Записи никогда не разбираются, поэтому зашифрованный saveset проходит без пароля. Итоги даёт XFRSUMM.

3.10 Saveset-ы на других узлах

Saveset может лежать на другом узле; он называется так, как DECnet называл файл там: *узел*::*файл*. *узел* — то, что принимает ssh (`host` или `user@host`), без косой черты; *файл* — saveset (его первый том) на том узле.

```
$ vbackup /home backup-host::/backup/home.bck /VOLUME_SIZE=4G /VERIFY
$ vbackup backup-host::/backup/home.bck /restore
$ vbackup backup-host::/backup/home.bck /LIST
```

Механизм

VBACKUP запускает VBACKUP на другом узле через ssh — или через команду, названную переменной окружения `VBACKUP_RSH`, — и соединяет его с собой каналом; saveset идёт по каналу потоком (раздел 3.9). Другому узлу всегда даётся копирование saveset-а:

Этот узел	Другой узел
Сохранение или копирование saveset-а в *узел*::*файл*: saveset пишется в канал.	`vbackup -`*файл*`/TRANSFER`, с `/REPLACE`, если он задан здесь: поток раскладывается по файлам томов *файла*, каждый блок проверяется.
Восстановление, `/LIST`, `/COMPARE`, `/EXTRACT` или копирование saveset-а из *узел*::*файл*: saveset читается из канала.	`vbackup`*файл*`- /TRANSFER`: тома *файла* подряд — в канал.

Поэтому `/VOLUME_SIZE` сохранения на другой узел создаёт файлы томов там. Вход *узел*::*файл* должен быть единственным входом. Сообщения другой стороны приходят сюда на стандартный вывод ошибок как есть, вперемежку с сообщениями этой стороны и со своим номером процесса: её STARTED, CREATED, XFRSUMM и COMPLETED и всё, что там пошло не так.

/VERIFY

Сохранение на другой узел с `/VERIFY` проверяется, когда другая сторона завершилась: saveset читается оттуда обратно (второй ssh, `vbackup`*файл*`- /TRANSFER`), и каждый файл сравнивается с диском здесь (VERIFYING, CMPSUMM). Saveset в том виде, в каком он пришёл, — CRC каждого блока, TRAILER — проверяет приёмник там. `/DELETE` и `/LIST` с saveset-ом, создаваемым на другом узле, не принимаются (QUALUSE: `not with a saveset made on another node`); листинг — второй командой, которая прочитает его обратно. При `/RECORD` журнал записывает такой saveset как *узел*::*файл*.

Коды завершения

Код завершения другой стороны присоединяется к коду этой. 0 ничего не добавляет; 1 (предупреждения) делает код завершения не меньше 1; любой другой код или конец по сигналу (показывается как 128 + сигнал) сообщается REMOTEERR, и код завершения — 2. Код 127 чаще всего значит, что команду `VBACKUP_RSH` нельзя запустить здесь или `vbackup` не найден там; 255 — отказал сам ssh. Если другая сторона остановилась, пока эта пишет, эта сторона находит канал оборванным: WRITERR (errno 32), FATALSAVE и REMOTEERR следуют друг за другом — это одна неисправность, о которой говорят сообщения другой стороны выше них. `/EXTRACT` перестаёт читать, как только получил свой файл; оборванный канал, который тогда встречает другая сторона, ошибкой не является. REMOTE сообщает, что канал или процесс для ssh вообще нельзя создать.

Требования

- VBACKUP X01-11 или новее на другом узле, в PATH неинтерактивного сеанса ssh: `ssh`*узел*`vbackup` должен его запускать.
- ssh должен пускать этот узел по ключам, ничего не спрашивая: ключ пользователя установлен там (`ssh-copy-id`), ключ другого узла уже известен.
- Пользователь на другом узле должен иметь возможность создать или прочитать saveset там.

Зачем /TRANSFER

Команды, отдаваемые другому узлу, несут `/TRANSFER`. VBACKUP до X01-11 не знает этого квалификатора и отвергает команду; без него он принял бы `vbackup - /backup/home.bck` за восстановление потока в каталог `/backup/home.bck`. Поэтому старый VBACKUP на другом узле явно отказывает (REMOTEERR), а не делает не то.

3.11 Кэш страниц

VBACKUP старается не вытеснять рабочие данные системы из кэша страниц:

- saveset пишется окнами по 8 МБ: запись каждого окна на диск запускается сразу, её завершения ждут одним окном позже, после чего страницы выбрасываются; каждый том при закрытии синхронизируется;
- тома читаемого saveset-а и сохраняемые файлы читаются с указанием последовательного доступа; прочитанная группа saveset-а выбрасывается из кэша;
- сохраняемый файл выбрасывается из кэша вслед за чтением, только если его в кэше до того не было (ни одна страница его первого мегабайта не находилась в памяти); файл, который был в кэше, — чьи-то рабочие данные, он остаётся.

Поэтому после сохранения страниц saveset-а и «холодных» файлов в кэше нет; это сделано намеренно.

3.12 Потоки

Поток(и)	Работа	Число
Записи	Пишет блоки saveset-а, пока основной поток читает файлы и строит блоки.	1; `VBACKUP_PIPELINE=0` — запись без него.
Опережающего чтения	Открывает следующие файлы обхода и читает их первый мегабайт, чтобы сохранение или копирование нашло их в кэше; вперёд не более 128 файлов и 64 МБ.	8 по умолчанию; `VBACKUP_PREFETCH=n`, не более 64, 0 — нет.
Сжатия	Сжимают данные сохранения `/DATA_FORMAT=COMPRESSED`; основной поток пишет записи по порядку.	По числу процессоров, не более 8; `VBACKUP_ZTHREADS=n`, 1 — нет.
Распаковки	Восстановление, `/COMPARE` и `/VERIFY` (с X01-23): записи читаются вперёд, до 4 на поток и 64 МБ, DATAZ и SOLID среди них распаковываются параллельно и отдаются по порядку.	Потоки шифрования: `VBACKUP_CTHREADS=n`, 1 — по одной записи.
Шифрования	Запись: зашифрованные блоки данных, ждущие потока записи, запечатываются (ChaCha20 и тег) параллельно, каждый целиком, а поток записи по порядку вычисляет блоки XOR и CRC; без потока записи — ключевой поток ChaCha20 блока полосами. Чтение: проверка тегов блоков группы и их расшифровка параллельно.	По числу процессоров, не более 8; `VBACKUP_CTHREADS=n`, 1 — нет.

Ни один поток не меняет saveset: при любом числе потоков один и тот же вход даёт один и тот же saveset.

3.13 Автономный распаковщик и распаковщики последней надежды

vbkx — небольшая статически скомпонованная программа, которая выводит листинг, извлекает, печатает и проверяет saveset-ы на машине, где VBACKUP не установлен, — в системе восстановления, в другом дистрибутиве. Ей не нужны ни библиотека справки, ни средство сообщений. Она восстанавливает данные, дыры, режимы, времена, символические и жёсткие ссылки и FIFO, а под root — владельцев (по номеру) и файлы устройств; ACL, расширенные атрибуты и флаги chattr она не восстанавливает. `vbkx.exe` — та же программа для Windows.

Ещё три распаковщика — vbkx-go (Go), vbkx-rs (Rust) и vbkx-pl (Perl) – написаны на тот день, когда всё остальное умерло: по одному исходному файлу, только стандартная библиотека, просто, чтобы их можно было прочесть и исправить по `format.md`. Они читают сжатые и зашифрованные saveset-ы, чинят один плохой блок на группу и называют каждый повреждённый или недостающий файл. См. приложение C.

3.14 Модули файловых менеджеров

Saveset-ы можно просматривать как папки в Midnight Commander (сценарий extfs `uvbk`), far2l и Far Manager 3 (MultiArc, поверх vbkx), Total Commander и Double Commander (модуль упаковщика WCX). Все модули работают только на чтение: saveset никогда не изменяется. Листинг берётся из каталога, поэтому даже saveset в сотни гигабайт открывается сразу. Зашифрованный saveset открывается с файлом ключа, названным в `VBACKUP_KEY_FILE`; модули никогда не спрашивают пароль. См. приложение D.

3.15 Расписание и ротация

VBACKUP не планирует себя и не удаляет старые saveset-ы: это дело пакетной очереди, и BATCH (подсистема пакетных заданий, `batch submit`) делает это так, как принято в OpenVMS, — задание выполняет сохранение и ставит само себя в очередь на следующий день.

```
#!/bin/sh
# /opt/jobs/vbackup-daily.sh - полная копия в воскресенье, инкремент в остальные дни
T=/backup/home-$(date +%Y%m%d)
if [ "$(date +%u)" = 7 ]; then
    vbackup /home $T.bck /RECORD /VERIFY
else
    vbackup /home $T.bck /SINCE=BACKUP /RECORD /VERIFY
fi
# Ротация: хранятся пять недель saveset-ов
find /backup -name 'home-*.bck*' -mtime +35 -delete
# Снова завтра в 02:00 - задание ставит себя в очередь само
batch submit "$0" /NAME=vbackup-daily "/AFTER=$(date -d tomorrow +%d-%b-%Y:02:00 | tr a-z A-Z)"
```

```
$ batch submit /opt/jobs/vbackup-daily.sh /NAME=vbackup-daily /AFTER=TOMORROW
```

Задание, завершившееся с ошибкой, остаётся в очереди вместе с журналом (`batch show`), где есть сообщения VBACKUP и код завершения; ничего не теряется молча, как с cron.

3.16 Saveset-ы OpenVMS BACKUP

На вход принимается saveset, записанный OpenVMS BACKUP. Он узнаётся по заголовку первого блока (магической строки нет) и читается от первого блока до последнего; VBACKUP выводит VMSSAVESET с размером блока и группы. Копируйте его с VMS в двоичном режиме (FTP `binary`); saveset, снятый с ленты, должен сохранить размер блока.

```
$ vbackup USERS.BCK /LIST /FULL
$ vbackup USERS.BCK /restore/vms /LOG
$ vbackup USERS.BCK "/EXTRACT=[SMITH]LOGIN.COM;3" login.com
```

`/LIST` и `/LIST /FULL` выводят то же, что `BACKUP/LIST` и `BACKUP/LIST/FULL`, байт в байт. Восстановление кладёт файлы в выходной каталог под именами Linux: `[SMITH.WORK]NOTES.TXT;5` становится `SMITH/WORK/NOTES.TXT`; старшая версия получает простое имя, младшие сохраняют `;n`; экранирование ODS-5 снимается, 8-битные имена переводятся в UTF-8; файл каталога становится каталогом. `/SELECT`, `/EXCLUDE` и `/EXTRACT` принимают имена Linux; `/EXTRACT` ещё и имя с версией или спецификацию файла VMS.

Последовательные файлы с управлением кареткой становятся текстами с LF, как их делает FTP VMS: у записей переменной длины и VFC убираются счётчики и управляющие области, после фиксированных записей ставится LF, Stream превращает CR LF в LF, Stream_CR — CR в LF. Файлы без управления кареткой копируются как лежат на диске VMS; индексные и относительные – образом файла RMS (VMSRAW). Режим берётся из защиты (R — r, W — w, E – x только у каталогов), время изменения — из даты ревизии, понятой как местное время; владелец — тот, кто восстанавливает.

Плохие блоки восстанавливаются по XOR-блокам `/GROUP_SIZE` (BLKFIXED); о блоках, которые не восстановить, сообщается (BLKLOST), как и о файле, потерявшем данные (FILDAMAGED). О saveset-е, записанном с `/NOCRC`, сообщается один раз (VMSNOCRC). Saveset-ы, зашифрованные BACKUP, и LBN-данные saveset-ов `/IMAGE` и `/PHYSICAL` не читаются. `/COMPARE`, `/INCREMENTAL`, `/ORIGINAL`, `/IMAGE`, `/PHYSICAL` и `/TRANSFER` с таким saveset-ом недопустимы. Автономный распаковщик `vbkx` читает его так же. Формат в том виде, в каком его читает VBACKUP, описан в `doc/vmsbackup.md`.

3.17 VBACKUP в Windows

`vbackup.exe` — это VBACKUP, собранный для Windows с MinGW-w64: та же команда, те же квалификаторы, те же saveset-ы. Saveset, сделанный в Linux, листается, восстанавливается и сравнивается в Windows; сделанный в Windows читается в Linux, `vbkx` и каждым распаковщиком последней надежды. Собирается в Linux из того же дерева, с исходниками StarLet (StarLet, установленный для Linux, не годится):

```
$ cmake -S . -B build-win -DCMAKE_TOOLCHAIN_FILE=cmake/mingw-w64.cmake \
        -DVBACKUP_STARLET_SRC=/root/Works/starlet-1.6.8
$ cmake --build build-win
```

Сборка для Linux делает его сама (`build/winexe/vbackup.exe`), когда есть MinGW-w64 и комплект StarLet установленной версии (`../starlet-x.y.z` рядом с деревом исходников), и `test/winutil.sh` гоняет его под wine. Образ статический: скопируйте `vbackup.exe` куда угодно и запускайте.

```
C:\> vbackup C:\Users\ivan\Documents D:\docs.bck /LOG
C:\> vbackup D:\docs.bck /LIST /FULL
C:\> vbackup D:\docs.bck C:\restore
C:\> vbackup C:\data D:\data.bck /ENCRYPT /KEY_FILE=C:\Users\ivan\backup.key
```

Что сохраняется и восстанавливается

Имена — в Юникоде (в saveset-е UTF-8, как в Linux), пути любой длины (где нужно, берётся форма `\\?\`). Компоненты спецификации разделяют и `\`, и `/` — `\` где угодно в командной строке считается разделителем, в `/SELECT` и `/COMMENT` тоже; в saveset-е имена хранятся с `/`.

Сохраняются и возвращаются:

- данные; разреженный файл — по его занятым диапазонам, восстанавливается с дырами (файл становится разреженным, когда пишется дыра);
- время изменения, доступа и смены атрибутов и время создания (BTIME);
- атрибуты NTFS — только чтение, скрытый, системный, архивный, временный, не индексировать (тег WINATTR); `/LIST /FULL` показывает их как `Windows: HIDDEN ARCHIVE`. Режим выводится из “только чтение”: `0444` или `0644` у файла, `0755` у каталога;
- дескриптор безопасности — владелец, группа, DACL и SACL, если у образа есть привилегия безопасности (тег NTSD); владелец и группа показываются учётными записями, `ДОМЕН\пользователь` (UID и GID равны 0);
- альтернативные потоки данных — как расширенные атрибуты `user.<поток>` (так их показывает ntfs-3g в Linux), до 64 КБ каждый; о более длинном сообщается (XATTRSKIP);
- каталоги, жёсткие ссылки (по идентификатору файла и серийному номеру тома NTFS, как по inode и устройству в Linux), символьные ссылки и junction-ы (как символьные ссылки).

В Linux saveset из Windows восстанавливается как любой другой: потоки становятся атрибутами `user.`, атрибуты NTFS и дескриптор пропускаются; его читает каждый распаковщик.

Привилегии и владелец

Запущенный от имени администратора (с повышением), `vbackup.exe` включает привилегии оператора резервного копирования — SeBackupPrivilege, SeRestorePrivilege, SeSecurityPrivilege, SeCreateSymbolicLinkPrivilege, – если они есть у его учётной записи. Тогда он читает любой файл, каким бы ни был его ACL, создаёт символьные ссылки и по умолчанию берёт `/OWNER=ORIGINAL`, как root в Linux: дескриптор безопасности каждого файла возвращается — владелец, группа, DACL (защищённый

или наследующий, как был) и, с привилегией безопасности, SACL. Без привилегии восстановления по умолчанию `/OWNER=DEFAULT`: дескриптор не возвращается, и восстановленный файл получает ACL своего каталога — DACL с другой машины мог бы оставить восстанавливающего без доступа к файлам. `/OWNER=ORIGINAL` всё равно просит дескриптор; владелец, которого нельзя назначить, называется (ATTRERR … security). `/OWNER=`*пользователь* и `/BY_OWNER` в Windows отвергаются (QUALUSE): SID — не номер пользователя. Дескриптор, прочитанный из saveset-а, перед использованием проверяется — каждый SID и ACL в нём должны лежать внутри элемента, — и поддельный отвергается (ATTRERR … the descriptor in the saveset is not valid).

Имена, которых в Windows не бывает

В Linux-saveset-е могут быть имена, которые Windows не допускает или понимает иначе: с `:` (поток другого файла), `< > " | ? * \`, с управляющим символом, с точкой или пробелом в конце (Windows их отбрасывает — два имени, один файл), имена устройств (`CON`, `PRN`, `AUX`, `NUL`, `COM1`-`COM9`, `LPT1`-`LPT9`, и с расширением тоже). Такой файл не создаётся: OPENOUT называет его с errno 22 и причиной `not a valid name on Windows`, и восстановление идёт дальше. Два имени, отличающихся только регистром (`README`, `readme`), в Windows — один файл: второе не восстанавливается — FILEEXISTS, а с `/REPLACE` OPENOUT (`a name that differs in case only is there`), чтобы оно никогда не заменило первое. FIFO, сокет и устройство не создаются (UNSUPP). Из расширенных атрибутов Linux-saveset-а потоками становятся только `user.`; остальные (`security.`, `trusted.`, `system.`) пропускаются.

Отличия от Linux

- Saveset на другом узле (`node::file`) идёт через `ssh.exe` — клиент OpenSSH из Windows 10 и 11; другой называет `VBACKUP_RSH`.
- `/PHYSICAL` и `/IMAGE` отвергаются сразу (QUALUSE).
- Нет упреждающего чтения файлов (`VBACKUP_PREFETCH` ни на что не влияет) и теневых копий (VSS): файл, который другая программа держит открытым без совместного доступа, называется (OPENIN) и пропускается; файл, в который она пишет, сохраняется таким, каким прочитан (FEND CHANGED, если он тем временем изменился).
- Права файла ключа не проверяются: кто его читает, решает его ACL. Держите его в своём профиле, где читать его можете только вы и администраторы.
- Журнал — `%USERPROFILE%\.vbackup\vbackup.jnl` (HOME, если он задан).
- Пароль читается с консоли без эха, в UTF-8: saveset, зашифрованный в Linux, открывается в Windows тем же паролем.

Глава 4. Примеры

1. ```
   $ vbackup /home/rrl /backup/rrl.bck /VERIFY
    %VBACKUP-I-STARTED, Operation: save, Input: /home/rrl, Output: /backup/rrl.bck -
   started
    %VBACKUP-I-SAVESUMM, Files: 6, Bytes: 108902, Blocks: 5, Volumes: 1 - saved
    %VBACKUP-I-VERIFYING, Saveset: /backup/rrl.bck - verifying
    %VBACKUP-I-CMPSUMM, Files: 6, Differences: 0 - compared
    %VBACKUP-I-COMPLETED, Operation: save, Seconds: 0.00 - completed
   ```

   Полное сохранение каталога `/home/rrl` в saveset `rrl.bck`. Saveset распознан по имени. `/VERIFY` читает новый saveset обратно и сравнивает каждый файл с диском; различий не найдено, код завершения 0. Хранимые имена начинаются с `rrl/`: база — `/home`.

2. ```
   $ vbackup /home full.bck /RECORD /JOURNAL=/backup/home.jnl
    %VBACKUP-I-STARTED, Operation: save, Input: /home, Output: full.bck - started
    %VBACKUP-I-SAVESUMM, Files: 7, Bytes: 3108910, Blocks: 55, Volumes: 1 - saved
    %VBACKUP-I-RECORDED, Files: 5, Journal: /backup/home.jnl - recorded
    %VBACKUP-I-COMPLETED, Operation: save, Seconds: 0.01 - completed
    $ vbackup /home mon.bck /SINCE=BACKUP /RECORD /JOURNAL=/backup/home.jnl
    %VBACKUP-I-STARTED, Operation: save, Input: /home, Output: mon.bck - started
    %VBACKUP-I-SAVESUMM, Files: 3, Bytes: 4, Blocks: 4, Volumes: 1 - saved
    %VBACKUP-I-INCRSUMM, Files: 4 - unchanged, listed as present, not saved
    %VBACKUP-I-RECORDED, Files: 1, Journal: /backup/home.jnl - recorded
    %VBACKUP-I-COMPLETED, Operation: save, Seconds: 0.00 - completed
    $ vbackup full.bck,mon.bck /restore /INCREMENTAL /LOG
   %VBACKUP-I-STARTED, Operation: restore, Input: full.bck, Output: /restore - started
    %VBACKUP-I-RESTORED, File: /restore/rrl - restored
    ...
    %VBACKUP-I-RESTORED, File: /restore/rrl/new.txt - restored
    %VBACKUP-I-RESTORED, File: /restore/rrl/sub - restored
  %VBACKUP-I-DELETED, File: /restore/rrl/sub/c.h - deleted: it is not in the incremental saveset
    %VBACKUP-I-RESTSUMM, Files: 10, Bytes: 3108914 - restored
    %VBACKUP-I-COMPLETED, Operation: restore, Seconds: 0.01 - completed
   ```

   Инкрементная цепочка. Полное сохранение записывает свои файлы в журнал этого плана. В понедельник один файл создан и один удалён; `/SINCE=BACKUP` сохраняет новый файл и каталоги, а каталог saveset-а перечисляет четыре неизменных файла как присутствующие. Восстановление берёт сначала полный saveset, затем инкрементный; удалённый в понедельник файл удаляется из выхода. Каталоги учитываются в RESTSUMM при каждом своём восстановлении.

3. ```
   $ vbackup /home /mnt/usb/home.bck /VOLUME_SIZE=4G /DATA_FORMAT=COMPRESSED
    ...
    %VBACKUP-I-CREATED, Volume: /mnt/usb/home.bck.002 - created
    %VBACKUP-I-CREATED, Volume: /mnt/usb/home.bck.003 - created
   %VBACKUP-I-SAVESUMM, Files: 48211, Bytes: 10737418240, Blocks: 151216, Volumes: 3 -
   saved
    ...
   ```
   ```

```
$ vbackup /mnt/usb/home.bck /restore
```

Saveset разрезается на тома по 4 ГБ для диска FAT32, его данные сжимаются. О томах после первого сообщается по мере их создания. Для восстановления указывается первый том; остальные находятся рядом. Если бы том был утерян, MISSVOL назвал бы его, а файлы остальных томов всё равно были бы восстановлены.

4.
```
# umask 077; head -c 32 /dev/urandom | base64 > /root/backup.key
# vbackup /home /backup/home.bck /ENCRYPT /KEY_FILE=/root/backup.key
# crontab -l
30 2 * * * VBACKUP_KEY_FILE=/root/backup.key /usr/local/bin/vbackup /home /backup/home-
$(date +\%a).bck /ENCRYPT /REPLACE
```

Зашифрованное сохранение из cron. Файл ключа содержит пароль в первой строке и доступен на чтение только root; файл ключа, который могут читать другие, отвергается с KEYFILE. В crontab файл ключа задан через VBACKUP_KEY_FILE. У cron нет терминала: без файла ключа сохранение завершилось бы с NOKEY, а не ждало бы. Храните копию файла ключа в надёжном месте — без неё saveset-ы не открыть.

5.
```
$ vbackup /etc backup-host::/backup/etc.bck /VOLUME_SIZE=1M /VERIFY
%VBACKUP-I-STARTED, Operation: save, Input: /etc, Output: backup-host::/backup/etc.bck -
started
%VBACKUP-I-STARTED, Operation: copy of a saveset, Input: (standard input), Output: /backup/etc.bck -
started
%VBACKUP-I-CREATED, Volume: (standard output) - created
%VBACKUP-I-CREATED, Volume: /backup/etc.bck.002 - created
...
%VBACKUP-I-CREATED, Volume: /backup/etc.bck.005 - created
%VBACKUP-I-SAVESUMM, Files: 42, Bytes: 3662512, Blocks: 72, Volumes: 5 - saved
%VBACKUP-I-XFRSUMM, Blocks: 72, Volumes: 5, Bad: 0 - copied
%VBACKUP-I-COMPLETED, Operation: copy of a saveset, Seconds: 0.02 - completed
%VBACKUP-I-VERIFYING, Saveset: backup-host::/backup/etc.bck - verifying
%VBACKUP-I-STARTED, Operation: copy of a saveset, Input: /backup/etc.bck, Output: (standard output)
started
%VBACKUP-I-XFRSUMM, Blocks: 72, Volumes: 5, Bad: 0 - copied
%VBACKUP-I-COMPLETED, Operation: copy of a saveset, Seconds: 0.00 - completed
%VBACKUP-I-CMPSUMM, Files: 42, Differences: 0 - compared
%VBACKUP-I-COMPLETED, Operation: save, Seconds: 0.04 - completed
```

Сохранение на другой узел томами по 1 МБ, с проверкой. VBACKUP на backup-host, запущенный через ssh, принимает поток и создаёт там файлы томов /backup/etc.bck, /backup/etc.bck.002, …, проверяя каждый блок; его сообщения (копирование saveset-а) приходят сюда вперемежку с сообщениями сохранения. /VERIFY читает saveset оттуда обратно и сравнивает его с /etc. Если бы VBACKUP там завершился неудачно, REMOTEERR сообщил бы его код завершения, и код завершения был бы 2.

6.
```
$ vbackup backup-host::/backup/etc.bck /restore/etc
%VBACKUP-I-STARTED, Operation: restore, Input: backup-host::/backup/etc.bck, Output: /restore/etc -
started
%VBACKUP-I-STARTED, Operation: copy of a saveset, Input: /backup/etc.bck, Output: (standard output)
started
%VBACKUP-I-XFRSUMM, Blocks: 72, Volumes: 5, Bad: 0 - copied
%VBACKUP-I-COMPLETED, Operation: copy of a saveset, Seconds: 0.00 - completed
%VBACKUP-I-RESTSUMM, Files: 42, Bytes: 3662512 - restored
%VBACKUP-I-COMPLETED, Operation: restore, Seconds: 0.01 - completed
$ vbackup backup-host::/backup/etc.bck /LIST
$ vbackup backup-host::/backup/etc.bck /EXTRACT=etc/fstab | less
```

Восстановление с другого узла. VBACKUP там присылает тома подряд;

восстановление читает их как канал, один раз и только вперёд. Листинг читает весь поток (до каталога в его конце не добраться заранее); извлечение останавливается, как только получило свой файл.

7.
```
$ vbackup /backup/home.bck - > /tmp/home.stream
%VBACKUP-I-STARTED, Operation: copy of a saveset, Input: /backup/home.bck, Output: (standard output)
started
   %VBACKUP-I-XFRSUMM, Blocks: 72, Volumes: 5, Bad: 0 - copied
   %VBACKUP-I-COMPLETED, Operation: copy of a saveset, Seconds: 0.01 - completed
   $ vbackup - /mnt/usb/home.bck < /tmp/home.stream
%VBACKUP-I-STARTED, Operation: copy of a saveset, Input: (standard input), Output: /mnt/usb/home.bck
started
   %VBACKUP-I-CREATED, Volume: /mnt/usb/home.bck.002 - created
   ...
   %VBACKUP-I-CREATED, Volume: /mnt/usb/home.bck.005 - created
   %VBACKUP-I-XFRSUMM, Blocks: 72, Volumes: 5, Bad: 0 - copied
   %VBACKUP-I-COMPLETED, Operation: copy of a saveset, Seconds: 0.02 - completed
   $ cmp /backup/home.bck.003 /mnt/usb/home.bck.003
   $ vbackup /backup/secret.bck - | ssh vault 'vbackup - /archive/secret.bck'
   $ vbackup /backup/secret.bck vault::/archive/secret.bck
```

Saveset из пяти томов копируется в поток, здесь сохранённый в файле, и обратно в файлы томов: каждый том копии байт в байт совпадает с оригиналом. Последние две команды отправляют зашифрованный saveset на другую машину — через канал, заданный вручную, и как *узел*::*файл*: блоки копируются и никогда не расшифровываются, поэтому пароль не спрашивается ни на одной стороне. Если бы блок испортился по дороге, приёмник сообщил бы о нём (BLKCOPIED) и скопировал его как есть; восстановление чинит его по группе.

8.
```
# umount /dev/sdb1
# vbackup /dev/sdb1 /mnt/usb/sdb1.bck /PHYSICAL /DATA_FORMAT=COMPRESSED
# vbackup /mnt/usb/sdb1.bck /dev/sdc1 /PHYSICAL /REPLACE
Everything on /dev/sdc1 (21474836480 bytes) is to be overwritten with the device saved in /mnt/usb/sd
 Type YES to go on: YES
```

Раздел поблочно копируется на другой. Сначала он размонтируется (устройство, смонтированное на чтение и запись, отвергается с PHYSMOUNTED). Восстановление требует `/REPLACE` и, на терминале, `YES`. Копия несёт метки и UUID оригинала (PHYSUUID): не монтируйте оба одновременно.

9.
```
# vbackup /mnt/data /mnt/usb/data.bck /IMAGE /VERIFY
# vbackup /mnt/usb/data.bck /dev/sdc1 /IMAGE /REPLACE /LOG
```

Целая файловая система сохраняется из точки монтирования вместе с типом, меткой и UUID. Восстановление запускает программу mkfs её типа на `/dev/sdc1` (при `/LOG` команду показывает IMGCMD), восстанавливает файлы в новую файловую систему и размонтирует её (IMGSUMM). `/dev/sdc1` может быть меньше исходного устройства, если файлы помещаются.

10.
```
# vbackup /mnt/usb/home.bck /ORIGINAL      %VBACKUP-I-STARTED, Operation: restore,
Input: /mnt/usb/home.bck, Output: (where its files came from) - started    %VBACKUP-
I-ORIGTARGET, Saveset: /mnt/usb/home.bck, Target: /home - its files go back there
%VBACKUP-W-FILEEXISTS, File: /home/rrl/a.txt - already exists, not restored     ...
```

Каждый файл возвращается в каталог, из которого был сохранён. Существующие файлы сохраняются, и о них сообщается; добавьте `/REPLACE`, чтобы перезаписать их.

11.
```
$ vbackup /home/ivan/old /mnt/usb/old.bck /VERIFY /DELETE      ...      %VBACKUP-
I-CMPSUMM, Files: 213, Differences: 0 - compared       %VBACKUP-W-SRCKEPT, File:
/home/ivan/old/log.txt - not deleted: it changed after it was saved     %VBACKUP-I-
DELSUMM, Deleted: 197, Kept: 1
```

Файлы архивируются и затем удаляются с диска. Файл, в который писали после сохранения, остаётся. Каталоги остаются.

12. `$ vbackup /backup/rrl.bck /COMPARE` `$ vbackup /backup/rrl.bck /tmp/restore /COMPARE`

    Первая команда сравнивает saveset с местами, откуда были сохранены его файлы; вторая — с каталогом, в который он был восстановлен. О каждом различии сообщается COMPARERR; при наличии различий код завершения 2.

13. `$ vbackup /backup/rrl.bck /EXTRACT=rrl/notes.txt | less` `$ vbackup /backup/rrl.bck /tmp/notes.txt /EXTRACT=rrl/notes.txt`

    Один файл извлекается на стандартный вывод, затем в файл. Имя – хранимое имя, как его показывает `/LIST`.

14. `$ vbackup /backup/rrl.bck /LIST` `$ vbackup /backup/rrl.bck /LIST /FULL` `$ vbackup /backup/rrl.bck /LIST=/tmp/rrl.lis /FORMAT=LS`

    Три формы листинга: краткая (по умолчанию), полная и по строке на файл в форме `ls -l`, здесь — в файл.

15. `$ vbackup /JOURNAL=/backup/home.jnl /LIST /FULL '/SELECT=*/notes.txt'` `$ rm /backup/home.jnl` `$ vbackup /backup/full.bck,/backup/mon.bck /RECORD /JOURNAL=/backup/home.jnl`

    Выводится листинг журнала с известными ему файлами, имена которых оканчиваются на `/notes.txt`, и saveset-ом с последней копией каждого. Затем журнал перестраивается по каталогам saveset-ов, от старейшего.

16. `# vbkx t /mnt/usb/home.bck` `/mnt/usb/home.bck: all files read, all checksums match` `# vbkx l /mnt/usb/home.bck | less` `# vbkx x /mnt/usb/home.bck -C /mnt/root/home rrl/projects`

    В системе восстановления без VBACKUP статически скомпонованный vbkx проверяет saveset, выводит его листинг и извлекает один каталог на восстанавливаемый диск.

17. `C:\> set VBACKUP_KEY_FILE=C:\Users\rrl\backup.key` `C:\> vbkx.exe l E:\home.bck` `C:\> vbkx.exe x E:\home.bck -C C:\restore rrl/documents`

    В Windows vbkx.exe выводит листинг зашифрованного saveset-а на USB-диске и извлекает каталог. Имена, недопустимые в Windows, не извлекаются, и об этом сообщается.

18. `$ vbackup USERS.BCK /LIST /FULL` `$ vbackup USERS.BCK /restore/vms /SELECT=SMITH/*` `$ vbkx p USERS.BCK "[SMITH]LOGIN.COM;3"`

    Saveset OpenVMS BACKUP, скопированный с Alpha в двоичном режиме, выводится так, как его выводит `BACKUP/LIST/FULL`, файлы одного пользователя восстанавливаются под именами Linux, а одна из младших версий файла выводится через vbkx.

Глава 5. Словарь команд

Глава описывает команду `vbackup`: её формат и параметры, определение операции, код завершения и ограничения (раздел 5.1), а также все её квалификаторы в алфавитном порядке (раздел 5.2).

5.1 Команда VBACKUP

Утилита VBACKUP сохраняет файлы в saveset-ы, восстанавливает, выводит, сравнивает, извлекает и копирует их, здесь или на другом узле, копирует файлы с диска на диск, копирует блочные устройства и целые файловые системы и ведёт журнал инкрементного сохранения.

Формат

```
vbackup input-specifier[,...] output-specifier [/qualifiers]
vbackup input-specifier /LIST[=file] [/qualifiers]
vbackup input-specifier [output-specifier] /COMPARE [/qualifiers]
vbackup input-specifier [output-specifier] /EXTRACT=stored-name [/qualifiers]
vbackup input-specifier /ORIGINAL [/qualifiers]
vbackup input-saveset output-saveset [/TRANSFER] [/qualifiers]
vbackup saveset[,...] /RECORD [/JOURNAL=file]
vbackup /JOURNAL[=file] /LIST [/FULL] [/SELECT=(pattern[,...])]
vbackup /HELP [topic ...]
```

Параметры

input-specifier[,...]

Задаёт вход операции одним словом; несколько спецификаций разделяются запятыми без пробелов. Элемент в двойных кавычках берётся как есть.

- **Сохранение, копирование:** сохраняемые или копируемые файлы и каталоги, возможно с шаблонами (см. раздел 3.4). Каталог берётся со всем содержимым. Не более 64 спецификаций.
- **Сохранение /PHYSICAL:** одно блочное устройство или файл образа.
- **Сохранение /IMAGE:** одна точка монтирования или одно смонтированное блочное устройство.
- **Восстановление, листинг, сравнение, извлечение, перестройка журнала:** saveset, заданный именем первого тома; остальные тома ищутся рядом. Для восстановления и перестройки журнала можно задать несколько saveset-ов; они обрабатываются по очереди. Для восстановления `/INCREMENTAL` задайте сначала полный saveset, затем инкрементные в порядке их создания.
- **Копирование saveset-а:** один saveset, заданный именем первого тома.
- **`-`:** saveset со стандартного ввода (восстановление, `/LIST`, `/COMPARE`, `/EXTRACT`, копирование saveset-а).
- ***узел*`::`*файл*:** saveset на другом узле (раздел 3.10), читаемый по каналу так же, как `-`; он должен быть единственным входом. *узел* — имя хоста или `user@host`, без косой черты.

Saveset распознаётся по содержимому. Вход с шаблоном или `...` никогда не бывает saveset-ом.

output-specifier

Задаёт выход операции.

- **Сохранение:** создаваемый saveset: имя на `.bck` или `.sav`, любое имя с `/SAVE_SET`, `-` для стандартного вывода или *узел*`::`*файл* для saveset-а на другом узле. Если saveset существует, VBACKUP останавливается (OPENOUT, errno 17), если не задан `/REPLACE`.
- **Копирование saveset-а:** создаваемый saveset, названный как при сохранении, или любое имя с `/TRANSFER`. Если какой-то его том существует, VBACKUP останавливается (OPENOUT, errno 17), если не задан `/REPLACE`.
- **Восстановление:** каталог, в который ведётся восстановление; при необходимости он создаётся. Завершающие косые черты игнорируются. С `/ORIGINAL` не задаётся.

- **Восстановление /PHYSICAL:** выходное блочное устройство или файл образа.
- **Восстановление /IMAGE:** выходное блочное устройство.
- **Сравнение:** каталог, с которым ведётся сравнение. Если он не задан, каждый файл сравнивается с местом, откуда был сохранён (базы saveset-а).
- **Извлечение:** записываемый файл. Если он не задан или равен `-`, файл идёт на стандартный вывод.
- **Копирование:** каталог, в который ведётся копирование. Выход, не являющийся именем saveset-а, при входе, не являющемся saveset-ом, означает копирование.
- **Листинг, перестройка журнала, листинг журнала:** нет.

Принимаются только эти два параметра. Третье слово — ошибка (MAXPARM); чаще всего это квалификатор, набранный без косой черты.

Сводка по использованию

VBACKUP вызывается из командного интерпретатора. Квалификаторы можно ставить в любом месте командной строки — до, между или после параметров. Квалификатор можно сокращать, пока сокращение однозначно (`/VER` вместо `/VERIFY`); неоднозначное сокращение — ошибка. Квалификатор можно писать и по-юниксовому, `--VERIFY`; регистр букв не важен. Слово, начинающееся с косой черты, является квалификатором, только если называет квалификатор, поэтому `/etc` и `/home` — имена файлов. Одиночное `--` завершает квалификаторы: каждое слово после него — параметр. `-h` равносильно `/HELP`.

Квалификаторы можно «приклеивать» к параметру, как в DCL: `x.sav/sav/log` – это `x.sav /SAVE_SET /LOG`. VBACKUP сообщает об этом (GLUED), чтобы опечатка в имени файла не была незаметно принята за квалификатор. Слово, являющееся именем существующего файла, никогда не разрезается; квалификатор, значение которого содержит косую черту (`/JOURNAL=/var/...`), нужно писать отдельно.

Шаблоны раскрывает сам VBACKUP; заключайте их в кавычки, как и значения квалификаторов, содержащие скобки или *:

```
$ vbackup '/home/rrl/.../*.c' sources.bck '/EXCLUDE=(*.o,*/.cache)'
```

Как определяется операция

VBACKUP рассматривает параметры в следующем порядке:

1. Проверяется каждый вход. Вход с шаблоном — не saveset; любой другой вход является saveset-ом, если об этом говорит его содержимое. *узел*::*файл* на своей стороне означает `-`: как вход это saveset, как выход — имя saveset-а.
2. Задан `/EXTRACT` — извлечение. Задан `/COMPARE` — сравнение. Оба требуют ровно одного входа, и он должен быть saveset-ом (иначе NOTSAVESET).
3. Один вход, являющийся saveset-ом, выход — имя saveset-а (`.bck`, `.sav`, `/SAVE_SET` или `-`) или задан `/TRANSFER`, и нет `/LIST`: копирование saveset-а блок за блоком. Поэтому `vbackup x.bck y.bck` копирует saveset; чтобы восстановить, задайте каталог, имя которого не является именем saveset-а.
4. Все входы — saveset-ы: `/LIST` без выхода — листинг; `/RECORD` без выхода — перестройка журнала; иначе — восстановление.
5. Выход — имя saveset-а (`.bck`, `.sav`, `/SAVE_SET` или `-`): сохранение.
6. Выход задан и входы существуют: копирование.
7. Параметров нет совсем, заданы `/JOURNAL` и `/LIST`: листинг журнала.

`/TRANSFER` при любом другом исходе отвергается (QUALUSE).

В остальных случаях VBACKUP сообщает, чего не хватает: OPENIN, если вход не существует и выход не задан; NOPARAM, если не задан выход; IVOP, если вход не существует, а выход не является именем saveset-а. Без параметров и квалификаторов VBACKUP выводит краткую справку по использованию.

Начало и завершение

Каждая операция, кроме листинга, сообщает о своём начале (STARTED) и завершении (COMPLETED) с затраченным временем и итогом: `completed`, `completed with warnings` или `completed with errors`. Сохранение, восстановление, сравнение, копирование и копирование saveset-а сообщают также итоги (SAVESUMM, RESTSUMM, CMPSUMM, CPYSUMM, XFRSUMM) и без `/LOG`. `/LOG` добавляет сообщение о каждом файле.

Сообщения

Все сообщения пишутся на стандартный вывод ошибок, по одному в строке, в виде

```
dd-mm-yyyy hh:mm:ss.cc pid %VBACKUP-s-IDENT, text
```

На стандартный вывод идёт только то, что производит операция: листинг, данные `/EXTRACT`, saveset, записываемый в `-`, текст справки. См. главу 6. Сообщения VBACKUP на другом узле (раздел 3.10) тоже приходят на стандартный вывод ошибок, с номером процесса той стороны.

Код завершения

Значение	Смысл
0	Всё сделано. Ни одного предупреждения или ошибки.
1	Всё сделано, с предупреждениями: выдано хотя бы одно сообщение уровня W (например, файл изменился во время сохранения, файл существует и не восстановлен).
2	Что-то не сделано: выдано хотя бы одно сообщение уровня E или F (например, файл не удалось прочитать, `/COMPARE` или `/VERIFY` нашли различие, VBACKUP на другом узле не завершился успешно — REMOTEERR) либо команду не удалось разобрать.

Информационные сообщения и сообщения об успехе на код завершения не влияют.

Ограничения

Привилегии

- Привилегии root нужны, чтобы восстановить файлы с исходными владельцами (`/OWNER=ORIGINAL` — умолчание только для root), восстановить файлы устройств, установить некоторые расширенные атрибуты (`trusted.*`, `security.*`, возможности файлов) и некоторые флаги chattr (immutable, append-only), а также чтобы читать файлы, недоступные пользователю.
- Операции `/PHYSICAL` и `/IMAGE` над блочными устройствами требуют root: они открывают устройства, а восстановление `/IMAGE` запускает `mkfs` и монтирует новую файловую систему.
- Пользователь, не являющийся root, восстанавливает файлы как свои (`/OWNER=DEFAULT`); о владельцах и атрибутах, которые не удалось установить, сообщается (ATTRERR).

Пределы

Параметр	Предел
Размер блока (`/BLOCK_SIZE`)	от 8192 до 1048576 байт, кратно 512
Размер группы (`/GROUP_SIZE`)	от 0 до 100 блоков данных
Размер тома (`/VOLUME_SIZE`)	не менее (размер группы + 3) x размер блока; округляется вниз до кратного размеру блока
Тома читаемого saveset-а	до 9999; поиск прекращается после 16 отсутствующих имён подряд
Входные спецификации	64
Шаблоны `/SELECT`, `/EXCLUDE`	по 64
Глубина обхода каталогов	256 уровней
Спецификация файла	4095 байт
Текст `/COMMENT`	255 байт
Командная строка в сводке	4096 байт
Пароль	от 1 до 1024 байт
Итерации PBKDF2	не менее 1000 (по умолчанию 600000)
Данные в одной записи DATA или DATAZ	1048576 байт

Прочие ограничения

- Saveset, записываемый на стандартный вывод, несёт свои тома подряд; записывающая его команда не может его проверить (а значит, и удалить файлы после проверки) и вывести его листинг.
- Saveset со стандартного ввода читается один раз и только вперёд: поиска по каталогу нет; vbkx отвергает для него имена.
- Saveset, создаваемый на другом узле, не принимает ни `/DELETE`, ни `/LIST`; `/VERIFY` читает его оттуда обратно. Вход *узел*::*файл* должен быть единственным входом. На другом узле нужен VBACKUP X01-11 или новее, и он должен пускать этот узел по ключам ssh.
- Копирование saveset-а сохраняет размер блока, размер группы и тома входа: `/BLOCK_SIZE`, `/GROUP_SIZE` и `/VOLUME_SIZE` их не меняют.
- `/PHYSICAL` и `/IMAGE` принимают один вход при сохранении и один saveset при восстановлении.
- `/INCREMENTAL` отвергает saveset без каталога и saveset, созданный до X01-02 (NOTINCR).
- `/ORIGINAL` требует saveset, созданный X01-02 или позднее (ORIGNOBASE); с `/INCREMENTAL` — saveset с одной базой.
- `/IMAGE` восстанавливает только файловые системы типов ext2, ext3, ext4, xfs, btrfs, vfat и msdos.
- Saveset-ы никогда не изменяются на месте; в существующий saveset нельзя ни добавить, ни удалить из него что-либо.

5.2 Квалификаторы

В этом разделе квалификаторы команды VBACKUP описаны в алфавитном порядке; в конце описаны переменные окружения, влияющие на неё.

Сводка квалификаторов

В столбце *Применяется к*: S — сохранение, R — восстановление, L – листинг, C — сравнение, X — извлечение, P — копирование, J — перестройка и листинг журнала, T — копирование saveset-а.

Квалификатор	Класс	Применяется к	По умолчанию
`/BEFORE=time`	Выбор входных файлов	S, P	Любое время
`/BLOCK_SIZE=n`	Выходной saveset	S	65536
`/BRIEF`	Листинг	L	`/BRIEF`
`/BY_OWNER=user`	Выбор входных файлов	S, P	Любой владелец
`/CHANGED`	Выбор входных файлов	S, P	`/MODIFIED`
`/COMMENT="text"`	Выходной saveset	S	Нет
`/COMPARE`	Команда	C	—
`/[NO]CONFIRM`	Команда	S, R, P	`/NOCONFIRM`
`/CREATED`	Выбор входных файлов	S, P	`/MODIFIED`
`/[NO]CROSS_DEVICE`	Выбор входных файлов	S, P	`/NOCROSS_DEVICE`
`/DATA_FORMAT=keyword`	Выходной saveset	S	`UNCOMPRESSED`
`/DELETE`	Входной файл	S	Файлы сохраняются
`/ENCRYPT`	Выходной saveset	S	Без шифрования
`/EXCLUDE=(pattern[,...])`	Выбор входных файлов	S, R, C, P	Нет
`/EXTRACT=stored-name`	Команда	X	—
`/FORMAT=keyword`	Листинг	L	`VMS`
`/FULL`	Листинг	L, J	`/BRIEF`
`/GROUP_SIZE=n`	Выходной saveset	S	10
`/HELP [topic ...]`	Команда	—	—
`/IGNORE=NOBACKUP`	Выбор входных файлов	S, P	Файлы с nodump не берутся
`/IMAGE`	Команда	S, R	—
`/INCREMENTAL`	Выходной файл	R	—
`/JOURNAL[=file]`	Команда	S, J	См. текст
`/KEY_FILE=file`	Команда	S, R, L, C, X, J	`VBACKUP_KEY_FILE`, иначе терминал
`/LEVEL=n`	Выходной saveset	S	1 (с `/DATA_FORMAT=COMPRESSED`)
`/LIST[=file]`	Команда	L, S	—
`/[NO]LOG`	Команда	все	`/NOLOG`
`/MODIFIED`	Выбор входных файлов	S, P	`/MODIFIED`
`/ORIGINAL`	Выходной файл	R	—
`/OWNER=option`	Выходной файл	R, P	`ORIGINAL` для root, иначе `DEFAULT`
`/PARITY=m`	Выходной saveset	S	1
`/PHYSICAL`	Команда	S, R	—
`/[NO]RECORD`	Команда	S, J	`/NORECORD`

Квалификатор	Класс	Применяется к	По умолчанию
/[NO]REPLACE	Выходной файл, выходной saveset	S, R, X, T	/NOREPLACE
/SAVE_SET	Выходной saveset	S, T	По имени
/SELECT=(pattern[,...])	Выбор входных файлов	S, R, C, P, J	Все файлы
/SINCE=time	Выбор входных файлов	S, P	Любое время
/[NO]SOLID	Выходной saveset	S	/SOLID при сжатии
/TRANSFER	Команда	T	По имени
/[NO]VERIFY	Команда	S, P	/NOVERIFY
/VOLUME_SIZE=size	Выходной saveset	S	Один том
/[NO]XATTRS	Входной файл, выходной файл	S, R, P	/XATTRS

/BEFORE

Квалификатор выбора входных файлов.

Формат

```
/BEFORE=time
```

Описание

Выбирает для сохранения только файлы, время которых раньше заданного. Рассматривается время изменения, если не задан /CREATED или /CHANGED. Каталоги сохраняются всегда.

Время записывается так (допускается однозначное сокращение ключевого слова):

Форма	Смысл
dd-MMM-yyyy[hh:mm[:ss]]	Абсолютное время, например 3-OCT-2026 14:00
hh:mm[:ss]	Сегодня в это время
-[dd]hh:mm[:ss]	Интервал назад от текущего момента: "-1 0:0" — сутки назад
TODAY, YESTERDAY, TOMORROW	Полночь в начале этого дня
NOW	Текущий момент

/BEFORE делает saveset инкрементным: файлы, которые он не выбрал, перечисляются в каталоге как PRESENT (см. раздел 3.5). О неправильно записанном времени сообщается IVTIME. /BEFORE нельзя сочетать с /PHYSICAL и /DELETE. При копировании копируются только выбранные файлы.

Пример

```
$ vbackup /home/rrl old.bck "/BEFORE=1-JAN-2026"
```

Сохраняются файлы /home/rrl, в последний раз изменённые до 1 января 2026 г.; более новые файлы перечисляются в каталоге как присутствующие.

/BLOCK_SIZE

Квалификатор выходного saveset-а.

Формат

```
/BLOCK_SIZE=n
```

Описание

Задаёт размер блоков saveset-а в байтах. *n* должно быть кратно 512 и лежать в пределах от 8192 до 1048576; по умолчанию 65536. Большие блоки немного быстрее; маленькие теряют меньше данных при потере одного блока. Размер блока читаемого saveset-а берётся из самого saveset-а.

О недопустимом значении сообщается IVQUAL.

Пример

```
$ vbackup /home home.bck /BLOCK_SIZE=1000
%VBACKUP-E-IVQUAL, Value: 1000, Qualifier: /BLOCK_SIZE - is not valid
$ vbackup /home home.bck /BLOCK_SIZE=262144
```

/BRIEF

Квалификатор листинга.

Формат

```
/BRIEF
```

Описание

С `/LIST` выводит для каждого файла хранимое имя, размер и дату изменения; имена каталогов оканчиваются косой чертой. Это умолчание. `/BRIEF` и `/FULL` нельзя задавать вместе (CONFQUAL).

Пример

```
$ vbackup rrl.bck /LIST /BRIEF
Listing of save set(s)

Save set:          rrl.bck
Volumes:           1
Written with:      VBACKUP X01-11
Node name:         TTR-RTR
Written by:        root
Command:           vbackup /home/rrl rrl.bck /VERIFY
Date:               5-OCT-2026 14:19:25.00
Base:              /home
Kind:              full
Block size:        65536
Group size:        10
Operating system:  Linux 6.1.0-52-amd64 x86_64

rrl/
rrl/a.txt                                                 6   5-OCT-2026 14:19
rrl/link                                                  5   5-OCT-2026 14:19
rrl/sub/
rrl/sub/b.c                                          108894   5-OCT-2026 14:19
rrl/sub/c.h                                               2   5-OCT-2026 14:19

Total of 6 files, 108902 bytes
End of save set
```

/BY_OWNER

Квалификатор выбора входных файлов.

Формат

```
/BY_OWNER=user
```

Описание

Выбирает только файлы заданного пользователя — по имени или по числовому идентификатору. Каталоги берутся всегда. Файлы других владельцев не охватываются: они не сохраняются и не перечисляются как присутствующие. О неизвестном имени, не являющемся числом, сообщается IVQUAL. `/BY_OWNER` нельзя сочетать с `/PHYSICAL` и `/IMAGE`.

Пример

```
# vbackup /home ivan.bck /BY_OWNER=ivan
```

/CHANGED

Квалификатор выбора входных файлов.

Формат

```
/CHANGED
```

Описание

Заставляет `/SINCE` и `/BEFORE` рассматривать время смены атрибутов (ctime) файла. Это время меняется также при изменении прав, владельца или имени файла. Задать можно только один из `/MODIFIED`, `/CREATED` и `/CHANGED` (CONFQUAL).

Пример

```
$ vbackup /etc etc.bck /SINCE=YESTERDAY /CHANGED
```

/COMMENT

Квалификатор выходного saveset-а.

Формат

```
/COMMENT="text"
```

Описание

Сохраняет текст длиной не более 255 байт в сводке saveset-а. `/LIST` выводит его в строке `Comment:`.

Пример

```
$ vbackup /home/rrl rrl.bck /COMMENT="before the upgrade"
```

/COMPARE

Квалификатор команды.

Формат

```
input-specifier [output-specifier] /COMPARE
```

Описание

Читает saveset и сравнивает каждый файл с диском: тип, размер, содержимое и цель символической ссылки. Владельцы, режимы и времена не сравниваются. С выходным параметром файлы ищутся в этом каталоге по хранимым именам; без него — в базах, записанных в saveset-е (там, откуда они были сохранены). Первое различие каждого файла сообщается COMPARERR, итоги — CMPSUMM. Любое различие даёт код завершения 2.

Файл каталога, которого поток записей не дал, — тоже различие (начиная с X01-22): `not compared: its records were lost in bad blocks` после потерянных блоков, иначе `not compared: the catalog has it, the record stream does not`. Файл, которого нет в saveset-е, никогда не обходится молчанием.

/SELECT и /EXCLUDE ограничивают сравнение. Saveset может быть -. Вход должен быть ровно одним saveset-ом (иначе NOTSAVESET).

Пример

```
$ vbackup rrl.bck /COMPARE
%VBACKUP-I-STARTED, Operation: compare, Input: rrl.bck - started
%VBACKUP-E-COMPARERR, File: /home/rrl/a.txt - the size differs
%VBACKUP-I-CMPSUMM, Files: 6, Differences: 1 - compared
%VBACKUP-I-COMPLETED, Operation: compare, Seconds: 0.00 - completed with errors
```

/CONFIRM

Квалификатор команды.

Формат

/CONFIRM /NOCONFIRM (по умолчанию)

Описание

Спрашивает на терминале перед обработкой каждого файла:

Операция	Вопрос
Сохранение	Save *хранимое-имя* ? [N]:
Восстановление	Restore *файл* ? [N]:
Копирование	Copy *файл* ? [N]:
/DELETE, удаления /INCREMENTAL	Delete *файл* ? [N]:

Ответ — YES, NO, QUIT или ALL; достаточно первой буквы, пустой ответ означает NO. ALL обрабатывает остальное без вопросов. QUIT прекращает операцию; прерванное так сохранение всё же правильно закрывает saveset. Без терминала ответом считается QUIT. /CONFIRM отвергается при восстановлении /IMAGE (CONFQUAL).

Пример

```
$ vbackup /home/rrl rrl.bck /CONFIRM
Save rrl/a.txt ? [N]: Y
Save rrl/core ? [N]: N
Save rrl/sub/b.c ? [N]: ALL
```

/CREATED

Квалификатор выбора входных файлов.

Формат

/CREATED

Описание

Заставляет /SINCE и /BEFORE рассматривать время создания (рождения) файла. Linux хранит его на ext4, xfs и btrfs. В файловой системе без него времени создания нет, и её файлы не выбираются ни /SINCE, ни /BEFORE. Задать можно только один из /MODIFIED, /CREATED и /CHANGED.

Пример

```
$ vbackup /data new.bck /SINCE=TODAY /CREATED
```

/CROSS_DEVICE

Квалификатор выбора входных файлов.

Формат

`/CROSS_DEVICE` `/NOCROSS_DEVICE` (по умолчанию)

Описание

По умолчанию встреченная при обходе точка монтирования сохраняется как каталог, но смонтированное на ней — нет; поэтому сохранение `/` не сохраняет `/proc`, `/sys` или смонтированный USB-диск. `/CROSS_DEVICE` заходит и в другие файловые системы. Сохранение `/IMAGE` всегда работает как `/NOCROSS_DEVICE`.

Пример

```
# vbackup / root.bck /NOCROSS_DEVICE '/EXCLUDE=(tmp,var/tmp)'
```

/DATA_FORMAT

Квалификатор выходного saveset-а.

Формат

`/DATA_FORMAT=COMPRESSED` `/DATA_FORMAT=UNCOMPRESSED` (по умолчанию)

Описание

`COMPRESSED` хранит данные файлов сжатыми в блочном формате LZ4 (записи DATAZ); несжимаемые данные хранятся как есть. Сводка отмечает это, и `/LIST` выводит `Data format: compressed (LZ4)`. Сжатие выполняется в нескольких потоках (`VBACKUP_ZTHREADS`); saveset от их числа не зависит. Читающие распознают сжатые данные сами. `/DATA_FORMAT` работает и с `/PHYSICAL`, и с `/IMAGE`. Мелкие файлы сжимаются группами, записями SOLID, если не задан `/NOSOLID` (см. `/SOLID`). См. раздел 3.7.

Пример

```
$ vbackup /home /mnt/usb/home.bck /DATA_FORMAT=COMPRESSED
```

/DELETE

Квалификатор входного файла.

Формат

`/DELETE`

Описание

После сохранения удаляет с диска файлы, которые были сохранены и проверены. Требуется `/VERIFY` (иначе QUALUSE). Файл удаляется, только если проверка не нашла вообще никаких различий во всём saveset-е и файл не изменился с момента сохранения: тот же inode и устройство, а для всего, кроме дополнительного имени жёсткой ссылки, — тот же размер, время изменения и время смены атрибутов, что в его элементе каталога. Иначе файл остаётся, и SRCKEPT сообщает почему. Удаляются только обычные файлы, символические и жёсткие ссылки в состоянии OK; каталоги остаются.

`/CONFIRM` спрашивает перед каждым удалением; `/LOG` сообщает о каждом (SRCDELETED); DELSUMM даёт итоги. `/DELETE` нельзя сочетать с `/PHYSICAL`, `/IMAGE`, `/SINCE` и `/BEFORE` (CONFQUAL). Если saveset идёт на стандартный вывод, `/DELETE` отвергается (QUALUSE): проверять не с чем. С saveset-ом на другом узле он тоже отвергается (QUALUSE: `not with a saveset made on another node`).

Пример

```
$ vbackup /home/ivan/old /mnt/usb/old.bck /VERIFY /DELETE /LOG
...
%VBACKUP-I-CMPSUMM, Files: 3, Differences: 0 - compared
%VBACKUP-I-SRCDELETED, File: /home/ivan/old/x - deleted: it is in the saveset and verified
%VBACKUP-I-SRCDELETED, File: /home/ivan/old/y - deleted: it is in the saveset and verified
%VBACKUP-I-DELSUMM, Deleted: 2, Kept: 0
```

/ENCRYPT

Квалификатор выходного saveset-а.

Формат

```
/ENCRYPT
```

Описание

Создаёт зашифрованный saveset (см. раздел 3.8). Пароль читается из файла ключа `/KEY_FILE` или `VBACKUP_KEY_FILE`; иначе он дважды запрашивается на терминале без эха. Без файла ключа и терминала сохранение завершается с NOKEY; при несовпадении ответов — с KEYMATCH. С `/LOG` сообщение ENCRYPTED показывает число итераций.

`/ENCRYPT` задаётся только при сохранении. Зашифрованный saveset при чтении распознаётся сам: восстановление, `/LIST`, `/COMPARE`, `/EXTRACT` и перестройка журнала запрашивают пароль или берут `/KEY_FILE`. `/ENCRYPT`, заданный другой операции, отвергается (QUALUSE).

Пример

```
$ vbackup /home/me me.bck /ENCRYPT
Passphrase for me.bck:
The same passphrase again:
```

/EXCLUDE

Квалификатор выбора входных файлов.

Формат

```
/EXCLUDE=(pattern[,...])
```

Описание

Исключает файлы, хранимое имя которых соответствует одному из шаблонов. Подходящий каталог исключается со всем содержимым. В шаблоне `*` соответствует любым символам, включая `/`, а `%` и `?` — одному символу. Не более 64 шаблонов (TOOMANY).

При сохранении и копировании исключённые файлы не охватываются; при восстановлении и сравнении они пропускаются. Листинг saveset-а исключённые записи пропускает. Его нельзя сочетать с `/INCREMENTAL`, `/PHYSICAL` и `/IMAGE`.

Пример

```
$ vbackup /home/rrl rrl.bck '/EXCLUDE=(*.o,*/.cache)'
```

/EXTRACT

Квалификатор команды.

Формат

```
input-specifier [output-specifier] /EXTRACT=stored-name
```

Описание

Записывает содержимое одного обычного файла saveset-a в выходной файл или на стандартный вывод, если выход не задан или равен `-`. Имя точное, без шаблонов, как его показывает `/LIST`. Файл находится через каталог, поэтому это быстро даже в очень большом saveset-e; из канала или saveset-a без каталога он ищется чтением. Данные проверяются по CRC файла (CRCERR).

Имя, которого нет в saveset-e, даёт NOTFOUND; файл, не являющийся обычным, – UNSUPP. Существующий выходной файл сохраняется (FILEEXISTS), если не задан `/REPLACE`.

Пример

```
$ vbackup home.bck /EXTRACT=rrl/notes.txt | less
$ vbackup home.bck /tmp/notes.txt /EXTRACT=rrl/notes.txt
```

/FORMAT

Квалификатор листинга.

Формат

`/FORMAT=VMS` (по умолчанию) `/FORMAT=LS`

Описание

`VMS` — листинг, описанный в `/BRIEF` и `/FULL`. `LS` выводит без сводки по одной строке на файл, как `ls -l`: режим, число ссылок, владелец, группа, размер, дата и время (`mm-dd-yyyy hh:mm:ss`), полное хранимое имя и для символической ссылки `-> цель`. Он предназначен для программ; его читает модуль Midnight Commander.

Пример

```
$ vbackup rrl.bck /LIST /FORMAT=LS
drwxr-xr-x   3 root     root           4096 10-05-2026 14:19:25 rrl
-rw-r--r--   1 root     root              6 10-05-2026 14:19:25 rrl/a.txt
lrwxrwxrwx   1 root     root              5 10-05-2026 14:19:25 rrl/link -> a.txt
drwxr-xr-x   2 root     root           4096 10-05-2026 14:19:25 rrl/sub
-rw-r--r--   1 root     root         108894 10-05-2026 14:19:25 rrl/sub/b.c
-rw-r--r--   1 root     root              2 10-05-2026 14:19:25 rrl/sub/c.h
```

/FULL

Квалификатор листинга.

Формат

`/FULL`

Описание

С `/LIST` выводит для каждого файла тип, размер, владельца (имя и номер), защиту, время изменения, контрольную сумму и состояние, а также цель ссылки. Типы: `file`, `directory`, `symlink`, `hard link`, `character device`, `block device`, `FIFO` и `socket`; состояния: `OK`, `changed while saved` и `read error`.

С `/JOURNAL /LIST` добавляет известные журналу файлы — для каждого размер, время записи и saveset с его последней копией.

Пример

```
$ vbackup rrl.bck /LIST /FULL
...
rrl/a.txt
    Type: file         Size: 6             Owner: root:root (0,0)  Protection: -rw-r--r--
    Modified:  5-OCT-2026 14:19:25.00  Checksum: 363A3020  Status: OK
rrl/link
```

```
    Type: symlink      Size: 5             Owner: root:root (0,0)  Protection: lrwxrwxrwx
    Modified:  5-OCT-2026 14:19:25.00  Checksum: 00000000  Status: OK
    Link to: a.txt
...
```

/GROUP_SIZE

Квалификатор выходного saveset-а.

Формат

```
/GROUP_SIZE=n
```

Описание

Задаёт число блоков данных, после которых пишется блок XOR. *n* — от 0 до 100; по умолчанию 10, что увеличивает saveset примерно на 10%. Один плохой блок в группе восстанавливается при чтении saveset-а (BLKFIXED). Меньшее *n* чинит больше повреждений и стоит больше места. `/GROUP_SIZE=0` не пишет блоков XOR, и ничего нельзя починить.

Пример

```
$ vbackup /home /mnt/usb/home.bck /GROUP_SIZE=4
```

/HELP

Квалификатор команды.

Формат

```
/HELP [topic ...]
```

Описание

Выводит описание VBACKUP из библиотеки справки или описание заданной темы. Каждое слово после `/HELP` — часть темы, в том числе квалификатор: `vbackup /HELP /VERIFY`. Среди тем — квалификаторы, PARAMETERS, WILDCARDS, INCREMENTAL, TIME, PLUGINS, VBKX, EXAMPLES, TROUBLESHOOTING и MESSAGES. О несуществующей теме сообщается NOTOPIC. `-h` равносильно `/HELP`. Если VBACKUP собран без пакета HELP, вместо этого выводится краткая справка по использованию.

Библиотека ищется в файле, названном `VBACKUP_HELPLIB`, затем в `../share/help/vbackup.hlb` относительно образа программы, рядом с образом и там, куда продукт был установлен.

Пример

```
$ vbackup /HELP /VERIFY

  VBACKUP /VERIFY

    After the saveset is written, VBACKUP reads it again and compares
    every file with the disk. A difference is reported with
    %VBACKUP-E-COMPARERR.
```

/IGNORE

Квалификатор выбора входных файлов.

Формат

```
/IGNORE=NOBACKUP
```

Описание

Файлы и каталоги с флагом nodump (`chattr +d`) обычно не сохраняются; это флаг NOBACKUP системы OpenVMS. С `/LOG` о каждом сообщается (SKIPPED). `/IGNORE=NOBACKUP` сохраняет и их. Сохранение `/IMAGE` всегда игнорирует этот флаг.

Пример

```
$ vbackup /home/rrl rrl.bck /IGNORE=NOBACKUP
```

/IMAGE

Квалификатор команды.

Формат

```
mount-point|device saveset /IMAGE saveset device /IMAGE /REPLACE
```

Описание

При сохранении сохраняет каждый файл одной целой файловой системы и её идентичность (см. раздел 3.6). Вход — одна точка монтирования или одно смонтированное блочное устройство; подкаталог — не том (IMGNOTVOL), несмонтированное устройство отвергается (IMGNOTMNT). Если UUID файловой системы найти не удалось, сообщается IMGNOID, и новая файловая система получит новый UUID.

При восстановлении создаёт на выходном устройстве новую файловую систему сохранённого типа, с той же меткой и UUID, и восстанавливает в неё файлы. Требует `/REPLACE` и отвергает смонтированное или занятое устройство (PHYSMOUNTED, PHYSHELD), устройство, которое не вместит файлы (IMGSMALL), и тип, который VBACKUP не умеет создавать (IMGUNSUPP). На терминале VBACKUP спрашивает:

```
Everything on device (n bytes) is to be overwritten with the device saved in saveset.
Type YES to go on:
```

Любой ответ, кроме `YES`, оставляет устройство нетронутым (PHYSABORT). Если стандартный ввод — не терминал, ничего не спрашивается. `/LOG` сообщает команду mkfs (IMGCMD). Итоги сообщает IMGSUMM.

`/IMAGE` работает с `/RECORD`, `/SINCE`, `/VERIFY`, `/DATA_FORMAT` и `/ENCRYPT`; отвергает `/SELECT`, `/EXCLUDE`, `/BY_OWNER`, `/INCREMENTAL`, `/PHYSICAL`, `/ORIGINAL`, `/DELETE` и при восстановлении `/CONFIRM`. Сохранение `/IMAGE` принимает один вход, восстановление — один saveset (QUALUSE).

Пример

```
# vbackup /mnt/data /mnt/usb/data.bck /IMAGE
# vbackup /mnt/usb/data.bck /dev/sdc1 /IMAGE /REPLACE
```

/INCREMENTAL

Квалификатор выходного файла.

Формат

```
full-saveset,incremental-saveset[,...] directory /INCREMENTAL
```

Описание

Восстанавливает цепочку saveset-ов: полный saveset, затем каждый инкрементный, от старейшего к новейшему, и приводит каждый каталог saveset-ов к виду, который он имел при последнем сохранении. Прежде чем трогать файлы очередного saveset-а, читается его каталог; после восстановления saveset-а файл, находящийся в каталоге saveset-а, но не перечисленный в его каталоге, удаляется (DELETED). О файле, который каталог перечисляет как присутствующий, но который не принёс ни один предыдущий saveset, сообщается (MISSING). `/INCREMENTAL` подразумевает `/REPLACE`.

Очищаются только каталоги, которые есть в saveset-е, и никогда — сам выходной каталог. Saveset без каталога, с испорченным каталогом или созданный до X01-02 отвергается (NOTINCR), и на нём цепочка обрывается: ничего не удаляется на основании неполного списка. /CONFIRM спрашивает перед каждым удалением, /LOG сообщает о нём. /SELECT и /EXCLUDE нельзя задавать с /INCREMENTAL (CONFQUAL), как и /PHYSICAL и /IMAGE. С /ORIGINAL saveset должен иметь одну базу.

Пример

```
$ vbackup full.bck,mon.bck,tue.bck /home /INCREMENTAL
```

/JOURNAL

Квалификатор команды.

Формат

```
/JOURNAL[=file]
```

Описание

Задаёт журнал, используемый /RECORD и /SINCE=BACKUP. По умолчанию это /var/lib/vbackup/vbackup.jnl для root и $HOME/.vbackup/vbackup.jnl для остальных пользователей; каталог при необходимости создаётся. Обычное применение — отдельный журнал для каждого плана резервного копирования.

С /LIST и без параметров выводится листинг журнала: известные ему saveset-ы, для каждого — время, вид, число файлов и байтов, имя и фильтр. /FULL добавляет файлы; /SELECT выбирает файлы по их абсолютным именам. О журнале, который не удаётся прочитать или записать, сообщается JNLERR.

Пример

```
$ vbackup /JOURNAL=/backup/plan1.jnl /LIST
Journal:           /backup/plan1.jnl

 5-OCT-2026 14:20:37.00  full              7 files        3108910 bytes  /backup/full.bck
5-OCT-2026 14:20:38.00  incremental     3 files          4 bytes  /backup/mon.bck /SINCE=BACKUP

Total of 2 savesets, 6 files recorded
```

/KEY_FILE

Квалификатор команды.

Формат

```
/KEY_FILE=file
```

Описание

Задаёт файл ключа с паролем зашифрованного saveset-а: его первую строку без признака конца строки. Файл должен быть обычным файлом, читать и изменять который может только владелец (chmod 600); иначе он отвергается (KEYFILE). Без /KEY_FILE используется файл, названный VBACKUP_KEY_FILE; без того и другого пароль запрашивается на терминале. /KEY_FILE служит сохранению /ENCRYPT и любому чтению зашифрованного saveset-а; для незашифрованного saveset-а он ни на что не влияет.

Пример

```
$ chmod 600 /root/backup.key
# vbackup /home /backup/home.bck /ENCRYPT /KEY_FILE=/root/backup.key
```

/LEVEL

Квалификатор выходного saveset-а.

Формат

`/LEVEL=n` (1 .. 9; по умолчанию 1 с `/DATA_FORMAT=COMPRESSED`)

Описание

Насколько сильно сжимаются данные: 1 — LZ4, 2..5 — Deflate, 6..9 – LZMA, с растущим усилием (раздел 3.7). Сжимает сам по себе – `/DATA_FORMAT=COMPRESSED` подразумевается; с `/DATA_FORMAT=UNCOMPRESSED` отвергается (CONFQUAL), значение вне 1..9 тоже (IVQUAL). `/LIST` выводит кодек: `Data format: compressed (LZ4)`, `(Deflate)` или `(LZMA)`. Каждая запись после сжатия проверяется (ZCHECK). Работает с `/PHYSICAL`, `/IMAGE`, `/ENCRYPT`, `/PARITY`. Мелкие файлы идут в записи SOLID, если не задан `/NOSOLID` (см. `/SOLID`).

Пример

```
$ vbackup /home /mnt/usb/home.bck /LEVEL=6
```

/LIST

Квалификатор команды.

Формат

`/LIST[=file]`

Описание

Выводит сводку saveset-а — имя, число томов, продукт, узел, пользователя, команду, дату, базы, вид и фильтр, размеры блока, группы и тома, формат данных, комментарий, шифрование, а для saveset-ов `/PHYSICAL` и `/IMAGE` – устройство или файловую систему, — а затем его файлы в форме, выбранной `/BRIEF`, `/FULL` или `/FORMAT`. Файлы читаются из каталога; без каталога (NOTRAILER) читается весь saveset (NOCATALOG). Присутствующие (PRESENT) элементы инкрементного saveset-а не выводятся. С именем файла листинг пишется в этот файл; существующий файл перезаписывается.

Заданный при сохранении, `/LIST` выводит листинг нового saveset-а (если тот идёт на стандартный вывод или на другой узел — отвергается). Листинг saveset-а на другом узле, *узел*::*файл*, строится по присылаемому им потоку (раздел 3.10). С `/JOURNAL` и без параметров выводит листинг журнала. `/SELECT` и `/EXCLUDE` ограничивают листинг saveset-а так же, как восстановление.

Пример

```
$ vbackup /backup/rrl.bck /LIST=/tmp/rrl.lis /FULL
```

/LOG

Квалификатор команды.

Формат

`/LOG` `/NOLOG` (по умолчанию)

Описание

Сообщает о каждом сохранённом (SAVED), восстановленном (RESTORED), сравнённом (COMPARED), скопированном (COPIED) или удалённом (SRCDELETED, DELETED) файле, о каждом созданном томе (CREATED; без `/LOG` — только о томах со второго), о файлах, пропущенных из-за флага nodump (SKIPPED), о шифровании (ENCRYPTED) и о команде mkfs (IMGCMD). Без `/LOG` VBACKUP сообщает о начале, итогах и завершении операции.

Пример

```
$ vbackup rrl.bck /tmp/out /LOG
%VBACKUP-I-STARTED, Operation: restore, Input: rrl.bck, Output: /tmp/out - started
%VBACKUP-I-RESTORED, File: /tmp/out/rrl - restored
%VBACKUP-I-RESTORED, File: /tmp/out/rrl/a.txt - restored
...
%VBACKUP-I-RESTSUMM, Files: 6, Bytes: 108902 - restored
%VBACKUP-I-COMPLETED, Operation: restore, Seconds: 0.00 - completed
```

/MODIFIED

Квалификатор выбора входных файлов.

Формат

```
/MODIFIED
```

Описание

Заставляет `/SINCE` и `/BEFORE` рассматривать время изменения (mtime) файла. Это умолчание. Задать можно только один из `/MODIFIED`, `/CREATED` и `/CHANGED`.

Пример

```
$ vbackup /home home.bck /SINCE=TODAY /MODIFIED
```

/ORIGINAL

Квалификатор выходного файла.

Формат

```
saveset /ORIGINAL
```

Описание

Восстанавливает каждый файл в каталог, из которого он был сохранён; выходной параметр не задаётся (иначе QUALUSE). Места — абсолютные базы, записываемые в сводку начиная с X01-02; saveset, созданный раньше, их не знает (ORIGNOBASE). Перед записью VBACKUP сообщает, куда пойдут файлы (ORIGTARGET). Saveset из нескольких входных спецификаций возвращает каждый файл под его собственную базу. Существующие файлы сохраняются, если не задан `/REPLACE`. `/ORIGINAL` нельзя сочетать с `/PHYSICAL` и `/IMAGE`; с `/INCREMENTAL` saveset должен иметь одну базу.

Осторожно с saveset-ами, созданными не вами: `/ORIGINAL` пишет туда, куда указывает saveset.

Пример

```
# vbackup /mnt/usb/home.bck /ORIGINAL /REPLACE
```

/OWNER

Квалификатор выходного файла.

Формат

```
/OWNER=ORIGINAL /OWNER=DEFAULT /OWNER=user
```

Описание

Задаёт владельца восстановленных или скопированных файлов.

Вариант	Смысл
`ORIGINAL`	Сохранённые владелец и группа: по имени, если имя известно в этой системе, иначе по номеру. Умолчание для root.
`DEFAULT`	Файлы принадлежат пользователю, запустившему VBACKUP. Умолчание для остальных пользователей, которые всё равно не могут отдавать файлы.
user	Все файлы принадлежат этому пользователю и его основной группе.

`ORIGINAL` и `DEFAULT` можно сокращать. О неизвестном пользователе сообщается IVQUAL; о владельце, которого не удалось установить, — ATTRERR.

Пример

```
# vbackup /backup/rrl.bck /home /OWNER=rrl
```

/PARITY

Квалификатор выходного saveset-а.

Формат

```
/PARITY=m
```

Описание

Задаёт число блоков чётности, которыми закрывается каждая группа, от 1 до 8. При `/PARITY=1`, по умолчанию, группа кончается XOR-блоком, и восстанавливается один плохой блок группы. При *m* от 2 за XOR-блоком идут *m* - 1 блоков PARITY (Reed-Solomon над GF(2^8), format.md 4.1): любые *m* плохих блоков группы — в том числе соседние, данные или чётность — восстанавливаются при чтении saveset-а (BLKFIXED). Saveset растёт на *m*/*n* объёма данных, где *n* — размер группы. Когда хороших блоков чётности больше, чем нужно, лишние строки проверяют починку; группа, чётность которой не сходится, не восстанавливается (PARITYERR).

Saveset с *m* от 2 имеет формат версии 2 в каждом блоке (3, если в нём могут быть записи SOLID, см. `/SOLID`). VBACKUP и vbkx до X01-14 его отвергают (NOTSAVESET); vbkx-go, vbkx-rs, vbkx-pl и модуль WCX версии X01-14 его читают. `/PARITY` нужны группы: с `/GROUP_SIZE=0` он недопустим.

Пример

```
$ vbackup /home /mnt/tape/home.bck /GROUP_SIZE=20 /PARITY=4
```

Saveset больше своих данных на 20% и переживает четыре плохих блока подряд в каждой группе из двадцати.

/PHYSICAL

Квалификатор команды.

Формат

```
device|image-file saveset /PHYSICAL saveset device|image-file /PHYSICAL [/REPLACE]
```

Описание

При сохранении поблочно копирует одно блочное устройство или файл образа (см. раздел 3.6). Вход должен быть блочным устройством или обычным файлом (PHYSNOTDEV). Устройство, смонтированное на чтение и запись, или раздел которого смонтирован так, отвергается (PHYSMOUNTED); так же — занятое устройство

(PHYSHELD). Если устройство меняет размер во время чтения, сообщается PHYSSIZE. Итоги сообщает PHYSSUMM.

При восстановлении saveset должен быть создан с `/PHYSICAL` (PHYSNOTPHYS). На блочное устройство требуется `/REPLACE` (PHYSREPLACE); смонтированное (PHYSMOUNTED), занятое (PHYSHELD) устройство и устройство меньше сохранённого (PHYSSMALL) отвергаются; на терминале спрашивается `YES`, как для `/IMAGE` (PHYSABORT). На устройство большего размера сообщается PHYSLARGER. Новый файл образа создаётся; существующий перезаписывается только с `/REPLACE`. Потерянный блок saveset-а — потерянная часть устройства (FILDAMAGED). После восстановления на устройство PHYSUUID напоминает, что копия несёт UUID оригинала.

`/PHYSICAL` принимает один вход при сохранении и один saveset при восстановлении. Отвергает `/SELECT`, `/EXCLUDE`, `/SINCE`, `/BEFORE`, `/BY_OWNER`, `/RECORD`, `/INCREMENTAL`, `/IMAGE`, `/ORIGINAL` и `/DELETE`. `/DATA_FORMAT=COMPRESSED` и `/ENCRYPT` работают с ним.

Пример

```
# vbackup /dev/sdb1 /mnt/usb/sdb1.bck /PHYSICAL
# vbackup /mnt/usb/sdb1.bck /dev/sdc1 /PHYSICAL /REPLACE
```

/RECORD

Квалификатор команды.

Формат

`/RECORD /NORECORD` (по умолчанию)

Описание

При сохранении записывает saveset в журнал после того, как он записан и, при `/VERIFY`, проверен без различий; а для каждого файла, сохранённого в состоянии OK, — его абсолютное имя, inode, размер, время изменения и время смены атрибутов (RECORDED). Файл, изменившийся во время сохранения или не прочитанный, не записывается, поэтому следующее `/SINCE=BACKUP` сохранит его снова. Saveset, записанный в `-`, записывается как `(standard output)`, на другой узел — как *узел*`::`*файл*.

С saveset-ами и без выходного параметра перестраивает журнал по их каталогам: каждый элемент в состоянии OK с данными inode (X01-02 и позднее) даёт состояние файла, более поздний saveset побеждает. Saveset без TRAILER пропускается (NOTRAILER); элементы без данных inode подсчитываются (NOINODE).

`/RECORD` нельзя сочетать с `/PHYSICAL`.

Пример

```
$ vbackup /home full.bck /RECORD /JOURNAL=/backup/home.jnl
$ vbackup full.bck,mon.bck /RECORD /JOURNAL=/backup/home.jnl
%VBACKUP-I-STARTED, Operation: rebuild of the journal, Input: full.bck,mon.bck - started
%VBACKUP-I-RECORDED, Files: 6, Journal: /backup/home.jnl - recorded
%VBACKUP-I-COMPLETED, Operation: rebuild of the journal, Seconds: 0.00 - completed
```

/REPLACE

Квалификатор выходного файла; квалификатор выходного saveset-а.

Формат

`/REPLACE /NOREPLACE` (по умолчанию)

Описание

Операция	Действие
Сохранение	Существующий saveset перезаписывается. Без /REPLACE сохранение останавливается с OPENOUT (errno 17, File exists). С saveset-ом на другом узле /REPLACE передаётся VBACKUP-у там.
Копирование saveset-а	Существующие тома выхода перезаписываются. Без /REPLACE копирование останавливается на первом существующем томе (OPENOUT, errno 17).
Восстановление, копирование	Существующий файл удаляется и восстанавливается заново. Без /REPLACE он сохраняется, и сообщается FILEEXISTS.
Извлечение	Существующий выходной файл перезаписывается.
Восстановление /PHYSICAL, /IMAGE	Выходное устройство перезаписывается; обязателен.

/INCREMENTAL подразумевает /REPLACE.

Пример

```
$ vbackup /backup/rrl.bck /home '/SELECT=*.c' /REPLACE
```

/SAVE_SET

Квалификатор выходного saveset-а.

Формат

```
/SAVE_SET
```

Описание

Указывает, что выходной параметр — saveset, каким бы ни было его имя. Не нужен, если имя оканчивается на .bck или .sav или равно -. Без него выход с другим именем при входе, не являющемся saveset-ом, означает копирование. При входе, являющемся saveset-ом, он делает команду копированием saveset-а (/TRANSFER). На входной стороне /SAVE_SET ни на что не влияет: saveset распознаётся по содержимому.

Пример

```
$ vbackup /etc /backup/etc.2026-10-05 /SAVE_SET
```

/SELECT

Квалификатор выбора входных файлов.

Формат

```
/SELECT=(pattern[,...])
```

Описание

Берёт только файлы, хранимое имя которых соответствует одному из шаблонов. В шаблоне * соответствует любым символам, включая /, а % и ? — одному символу. Хранимое имя задаётся относительно базы, например rrl/src/a.c; хранимые имена показывает /LIST. Не более 64 шаблонов.

При сохранении и копировании шаблоны выбирают файлы; каталоги берутся всегда. При восстановлении каталог восстанавливается, только если он сам соответствует шаблону (родительские каталоги выбранного файла создаются в любом случае); при сравнении несоответствующие файлы пропускаются. С `/JOURNAL /LIST /FULL` шаблоны сопоставляются с абсолютными именами журнала. В листинге saveset-а выводятся только подходящие записи. Его нельзя сочетать с `/INCREMENTAL`, `/PHYSICAL` и `/IMAGE`.

Пример

```
$ vbackup home.bck /tmp/r '/SELECT=(*.c,*.h)'
```

/SINCE

Квалификатор выбора входных файлов.

Формат

`/SINCE=time` `/SINCE=BACKUP`

Описание

Выбирает для сохранения только файлы, время которых не раньше заданного (формы записи времени см. в `/BEFORE`). Рассматривается время изменения, если не задан `/CREATED` или `/CHANGED`. Каталоги сохраняются всегда.

`/SINCE=BACKUP` выбирает файлы, изменившиеся с момента их последнего сохранения с `/RECORD`, как это знает журнал (раздел 3.5). Файл, неизвестный журналу, сохраняется. Без журнала сохраняется всё, как в первый раз, и сообщается NOJOURNAL.

Любой `/SINCE` делает saveset инкрементным; сводка хранит фильтр в том виде, в каком он задан (`Filter:` в листинге), а INCRSUMM подсчитывает файлы, перечисленные как присутствующие. `/SINCE` нельзя сочетать с `/PHYSICAL` и `/DELETE`.

Пример

```
$ vbackup /home /mnt/usb/home.bck /SINCE=TODAY /VOLUME_SIZE=4G
$ vbackup /home mon.bck /SINCE=BACKUP /RECORD
```

/SOLID

Квалификатор выходного saveset-а.

Формат

`/SOLID` (по умолчанию, когда сохранение сжимает) `/NOSOLID`

Описание

С `/DATA_FORMAT=COMPRESSED` или `/LEVEL` мелкие обычные файлы — каждый не больше 256 КБ и не второе имя жёсткой ссылки — собираются в группы до 256 КБ (в X01-21 — до 1 МБ), в порядке обхода, и каждая группа сжимается одной записью SOLID: записи FILE, DATA и FEND её файлов вместе (format.md 6.12). Словарь общий, метаданные тоже сжимаются, поэтому дерево мелких файлов становится намного меньше: `/usr/include` на `/LEVEL=6` — 55.6 МБ с `/NOSOLID` и 37 МБ с `/SOLID` (раздел 3.7). Большие файлы, каталоги и ссылки пишутся как раньше; каждый из них сначала закрывает открытую группу.

`/SOLID` действует по умолчанию всякий раз, когда сохранение сжимает; `/SOLID` сам по себе не сжимает. `/NOSOLID` пишет каждый файл своими записями, как до X01-21. С `/PHYSICAL` он ничего не меняет; работает с `/IMAGE`, `/ENCRYPT` и `/PARITY`. Каждая запись SOLID распаковывается и сравнивается до записи; группа, не вернувшаяся такой же (ZCHECK) или не уменьшившаяся, пишется обычными записями.

Saveset, в котором могут быть записи SOLID, имеет формат версии 3 в каждом блоке. VBACKUP, vbkx и модули до X01-21 его отвергают (NOTSAVESET) и никогда не читают неверно. `/TRANSFER` копирует его блок за блоком без изменений. Восстановление, `/LIST`,

/COMPARE, /EXTRACT, /SELECT, /VERIFY, /DELETE, vbkx и модули файловых менеджеров читают записи SOLID сами; отдельный файл достаётся через каталог по его записи SOLID, которая распаковывается, файлы перед ним пропускаются.

Цена: блоки, потерянные без возможности починки — больше *m* плохих блоков в одной группе, где *m* — значение /PARITY, — уносят файлы разрезанной ими записи SOLID начиная с места повреждения и все файлы записи SOLID, которая в них начинается, а не только тот файл, в котором были: от нескольких до пары десятков мелких файлов (раздел 3.7). Файлы записи SOLID до разреза восстанавливаются, каждый проверяется по CRC своей записи FEND (начиная с X01-22, и для saveset-ов X01-21); файл на разрезе называется в FILDAMAGED, файлы после него — в FILLOST. При чётности по умолчанию один плохой блок группы по-прежнему восстанавливается. Для saveset-а, который должен пережить сильные повреждения, задайте /PARITY=2 или больше либо /NOSOLID.

Примеры

```
$ vbackup /home /mnt/usb/home.bck /LEVEL=6
$ vbackup /home /mnt/usb/home.bck /LEVEL=6 /PARITY=2
$ vbackup /home /mnt/usb/old.bck /LEVEL=6 /NOSOLID
```

Первый saveset хранит мелкие файлы /home в записях SOLID. Второй тоже и переживает два плохих блока в каждой группе. Третий — для читающего до X01-21: каждый файл своими записями, формат версии 1.

/TRANSFER

Квалификатор команды.

Формат

```
input-saveset output-saveset /TRANSFER
```

Описание

Копирует saveset в saveset блок за блоком. Блоки копируются как есть – никогда не записи: копия байт в байт совпадает с оригиналом, том в том, а зашифрованный saveset копируется без пароля. Вход — один saveset, заданный первым томом, или -; выход — имя saveset-а или -:

Команда	Действие
`vbackup x.bck y.bck`	Копия том за томом: y.bck, y.bck.002, …
`vbackup x.bck -`	Тома подряд — на стандартный вывод.
`vbackup - y.bck`	Поток стандартного ввода раскладывается по y.bck, y.bck.002, … на каждом VHDR.

Каждый блок проверяется по пути. Плохой блок копируется как есть, и о нём сообщается (BLKCOPIED): копия хранит то, что ей дали, а восстановление чинит блок по его группе. Об отсутствующем томе входа сообщается (MISSVOL), остальные всё равно копируются; о входе, кончившемся до своего TRAILER, сообщается (NOTRAILER). Вход, начинающийся не с VHDR тома 1, не принимается (NOTSAVESET). Размер блока и тома входа сохраняются. Итоги даёт XFRSUMM; /LOG сообщает и о томе 1 (CREATED); /REPLACE перезаписывает существующие тома.

Вход-saveset и выход с именем saveset-а означают это копирование и без квалификатора (раздел 5.1). /TRANSFER требует его, что бы ни значили имена: vbackup x.bck dir /TRANSFER пишет saveset dir, dir.002, … и ничего не восстанавливает. При любом другом входе или выходе он отвергается (QUALUSE: the input must be one saveset (or -), and there must be an output). Его несут команды, которые VBACKUP отдаёт другому узлу, чтобы VBACKUP там версии до X01-11 отказал, а не стал восстанавливать (раздел 1.10).

Пример

```
$ vbackup /backup/home.bck /mnt/usb/home.bck /LOG
%VBACKUP-I-STARTED, Operation: copy of a saveset, Input: /backup/home.bck, Output: /mnt/usb/home.bck -
started
%VBACKUP-I-CREATED, Volume: /mnt/usb/home.bck - created
%VBACKUP-I-CREATED, Volume: /mnt/usb/home.bck.002 - created
...
%VBACKUP-I-CREATED, Volume: /mnt/usb/home.bck.005 - created
%VBACKUP-I-XFRSUMM, Blocks: 72, Volumes: 5, Bad: 0 - copied
%VBACKUP-I-COMPLETED, Operation: copy of a saveset, Seconds: 0.02 - completed
$ vbackup /backup/home.bck /TRANSFER
%VBACKUP-E-QUALUSE, Qualifier: /TRANSFER - the input must be one saveset (or -
), and there must be an output
```

/VERIFY

Квалификатор команды.

Формат

`/VERIFY` `/NOVERIFY` (по умолчанию)

Описание

При сохранении, после записи saveset-а, читает его обратно и сравнивает каждый файл с диском (VERIFYING, затем COMPARERR при различии и CMPSUMM с итогами). Файл каталога, которого поток не дал, — потерянный в плохих блоках или которого там никогда не было, — считается различием (начиная с X01-22), так что `/VERIFY` никогда не говорит `Differences: 0` над недостающим файлом. С saveset-ом, записываемым на стандартный вывод, отвергается (QUALUSE). Saveset на другом узле читается оттуда обратно, когда другая сторона завершилась, и сравнивается с диском здесь (раздел 3.10). `/DELETE` и, при `/VERIFY`, `/RECORD` действуют, только если проверка не нашла различий.

При копировании читает каждый обычный файл обратно с обеих сторон сразу после копирования и сравнивает их; о различии сообщается COMPARERR, а CMPSUMM даёт итоги, если различия найдены.

Пример

```
$ vbackup /home/rrl /backup/rrl.bck /VERIFY
%VBACKUP-I-STARTED, Operation: save, Input: /home/rrl, Output: /backup/rrl.bck - started
%VBACKUP-I-SAVESUMM, Files: 6, Bytes: 108902, Blocks: 5, Volumes: 1 - saved
%VBACKUP-I-VERIFYING, Saveset: /backup/rrl.bck - verifying
%VBACKUP-I-CMPSUMM, Files: 6, Differences: 0 - compared
%VBACKUP-I-COMPLETED, Operation: save, Seconds: 0.00 - completed
```

/VOLUME_SIZE

Квалификатор выходного saveset-а.

Формат

`/VOLUME_SIZE=size`

Описание

Разрезает saveset на тома заданного размера в байтах, за которым может следовать K, M, G или T (двоичные множители) и необязательная B. Размер округляется вниз до кратного размеру блока и должен быть не меньше (размер группы + 3) x размер блока: 851968 байт при умолчаниях. Тома называются *имя*, *имя*`.002`, *имя*`.003` и так далее (раздел 3.3); о каждом томе после первого сообщается (CREATED). Храните тома вместе в одном каталоге.

Если saveset идёт на стандартный вывод, тома идут в него подряд, каждый со своим VHDR (раздел 3.9); приёмник (`vbackup -`*saveset*) или VBACKUP на другом узле (раздел 3.10) снова раскладывает их по файлам томов. Копирование saveset-а сохраняет тома своего входа, что бы ни говорил `/VOLUME_SIZE`.

Пример

```
$ vbackup /home home.bck /VOLUME_SIZE=4G
$ vbackup /home home.bck /VOLUME_SIZE=100K
%VBACKUP-E-IVQUAL, Value: 100K, Qualifier: /VOLUME_SIZE - is not valid
```

/XATTRS

Квалификатор входного файла; квалификатор выходного файла.

Формат

`/XATTRS` (по умолчанию) `/NOXATTRS`

Описание

Сохраняет, восстанавливает и копирует расширенные атрибуты файлов: пользовательские атрибуты, списки POSIX ACL (`system.posix_acl_access`, `system.posix_acl_default`), возможности файлов и метку SELinux. Некоторые из них может восстановить только root; о тех, что не удалось установить, сообщается ATTRERR. `/NOXATTRS` их не сохраняет и не восстанавливает.

Пример

```
$ vbackup /home/rrl rrl.bck /NOXATTRS
```

Переменные окружения

На VBACKUP влияют следующие переменные окружения. Переменные с пометкой *vbkx* учитывает и vbkx; с пометкой *распаковщики* — vbkx-go, vbkx-rs и vbkx-pl.

Переменная	Смысл
`VBACKUP_KEY_FILE`	Файл ключа зашифрованного saveset-а, если не задан `/KEY_FILE` (*vbkx*, *распаковщики*, модули).
`VBACKUP_NOPROMPT`	1 — никогда не спрашивать пароль на терминале; без файла ключа результат — NOKEY. Устанавливается модулями файловых менеджеров (*vbkx*).
`VBACKUP_KDFITER`	Число итераций PBKDF2 для сохранения `/ENCRYPT`, не менее 1000; меньшие значения игнорируются. Только для тестов — не уменьшайте его для настоящих saveset-ов. Число хранится в saveset-е.
`VBACKUP_NOSIMD`	1 — вычислять чётность `/PARITY` переносимым кодом даже там, где у процессора есть векторные инструкции (AVX2, SSSE3 на x86; NEON на aarch64); для поиска неисправностей.
`VBACKUP_NOHWCRYPTO`	1 — вычислять SHA-256 переносимым кодом даже там, где у процессора есть для этого инструкции (aarch64); для поиска неисправностей.

Переменная	Смысл
VBACKUP_CTHREADS	Потоки шифрования и распаковки при восстановлении, /COMPARE, /VERIFY (X01-23), включая вызывающий: по умолчанию по числу процессоров, не более 8; 1 — нет.
VBACKUP_ZTHREADS	Потоки сжатия при сохранении /DATA_FORMAT=COMPRESSED: по умолчанию по числу процессоров, не более 8; 1 и меньше — нет.
VBACKUP_PIPELINE	0 — saveset пишется без потока записи, для поиска неисправностей. Подсказки кэшу страниц остаются; saveset тот же.
VBACKUP_PREFETCH	Потоки опережающего чтения следующих файлов при сохранении или копировании: 8 по умолчанию, не более 64; 0 — нет. Больше потоков может помочь на NFS или медленном сетевом диске, меньше — на одном медленном жёстком диске.
VBACKUP_RSH	Команда, которая вместо ssh запускает VBACKUP на другом узле для *узел*::*файл* (раздел 3.10). Вызывается как *команда узел удалённая-команда*, так же как ssh.
VBACKUP_HELPLIB	Библиотека справки для /HELP.
HOME	Каталог журнала по умолчанию для пользователя, не являющегося root.

Глава 6. Сообщения

6.1 Формат сообщений

VBACKUP пишет сообщения на стандартный вывод ошибок, по одной строке:

```
dd-mm-yyyy hh:mm:ss.cc pid %VBACKUP-s-IDENT, text
```

Поле	Смысл
dd-mm-yyyy hh:mm:ss.cc	Дата и время с точностью до сотых долей секунды
pid	Номер процесса
`VBACKUP`	Имя средства (facility)
s	Уровень серьёзности: S, I, W, E или F
`IDENT`	Идентификатор сообщения, по которому сообщения перечислены ниже
text	Текст сообщения

Текст везде имеет одну форму: сначала то, о чём сообщение, в виде пар `Метка: значение` — File, Saveset, Device, Volume, Block, Files, Bytes, errno, — затем дефис и слова, так что имена и числа видны с первого взгляда:

```
%VBACKUP-I-BLKFIXED, Block: 3, Volume: 1 - was bad, rebuilt from its group
```

За значением *errno* в скобках следует его текст из библиотеки C.

6.2 Уровни серьёзности и код завершения

Уровень	Смысл	Влияние на код завершения
S — успех	Операция выполнена успешно.	Нет (0).
I — информация	Сведения; всё в порядке.	Нет.
W — предупреждение	Что-то было не так, как должно, но операция выполнена; результат может требовать внимания.	Не менее 1.
E — ошибка	Что-то не сделано: файл, блок, сравнение или сама команда.	2.
F — фатальная ошибка	Операция не может продолжаться.	2.

Код завершения VBACKUP равен 2, если было выдано хотя бы одно сообщение E или F, иначе 1, если было выдано хотя бы одно сообщение W, иначе 0.

Ошибки, обнаруженные подпрограммами командного языка StarLet ещё до того, как VBACKUP рассматривает команду, выводятся с префиксом `%CLI_RTNS`, а не `%VBACKUP`, — например неоднозначное сокращение или недопустимое ключевое слово; код завершения в этом случае 2:

```
%CLI_RTNS-F: Ambiguous qualifier 'RE'
%CLI_RTNS-E: Illegal or unrecognized keyword 'ZIP'
```

Слово, начинающееся с косой черты, но не называющее квалификатор, — имя файла, а не ошибка само по себе; обычно оно приводит к MAXPARM или IVOP.

vbkx и распаковщики последней надежды этим средством сообщений не пользуются; их диагностика описана в приложении C.

6.3 Сообщения

ATTRERR, File: *file* - *what* not restored, errno: *n* (*reason*)

Средство: VBACKUP. **Уровень:** предупреждение (W).

Пояснение: Атрибут восстановленного или скопированного файла не удалось установить. *what* — `owner`, `protection`, `times`, `flags` (флаги chattr), `attributes` (каталога, устанавливаемые в конце восстановления), имя расширенного атрибута или `the removal` — файл, который `/INCREMENTAL` должен был удалить, удалить не удалось. Данные файла восстановлены.

Действие пользователя: Чаще всего восстановление выполняется не от root. Восстановите от root или задайте `/OWNER=DEFAULT`; задайте `/NOXATTRS`, если расширенные атрибуты не нужны. Некоторые атрибуты целевая файловая система не может хранить (например, ACL на vfat).

BADREC, Block: *block*, Volume: *volume* - an invalid record, skipped

Средство: VBACKUP. **Уровень:** предупреждение (W).

Пояснение: В потоке записей, в указанном блоке или после него, обнаружена запись с бессмысленным заголовком или телом; она пропущена. Окружающие блоки имели верные контрольные суммы: запись была записана неправильно, либо saveset создан не VBACKUP. Если данные потеряны, файл, к которому относится запись, сообщается как повреждённый.

Запись SOLID (раздел 3.7), которая не распаковывается или содержит бессмысленную запись, сообщается с блоком, где она начинается. Она пропускается вся, как потерянные блоки: ни один её файл из неё не восстанавливается, и каждый называется в FILLOST — неверными данными не восстанавливается никогда. Запись SOLID, лишь обрезанная потерянными блоками или концом saveset-а, сохранение которого было прервано, здесь не сообщается: её часть до разреза восстанавливается (см. FILLOST).

Действие пользователя: Проверьте файлы, названные в FILDAMAGED и FILLOST. Если saveset создан VBACKUP, сообщите о проблеме, приложив копию saveset-а.

BLKCOPIED, Block: *block*, Volume: *volume* - is bad, copied as it is: a restore repairs it from its group

Средство: VBACKUP. **Уровень:** предупреждение (W).

Пояснение: Копирование saveset-а (`/TRANSFER`, приёмник потока, VBACKUP на другом узле, принимающий сохранение) встретило блок с неверной контрольной суммой. Он копируется как есть: копия хранит то, что ей дали, а восстановление из копии чинит блок по его группе так же, как из оригинала. *block* — место блока в копии, считая с 0, *volume* — том выхода, который пишется; если до плохого блока не пропущен ни один том, это его номер блока и номер тома в saveset-е.

Действие пользователя: Для копии — никаких, пока в той же группе нет другого плохого блока. Если блок испортился по дороге (сеть, канал), скопируйте saveset ещё раз; если он плох в оригинале, носитель может выходить из строя: скопируйте saveset на другой носитель и проверьте его восстановлением или `/COMPARE`.

BLKFIXED, Block: *block*, Volume: *volume* - was bad, rebuilt from its group

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: Блок saveset-а имел неверную контрольную сумму (или, в зашифрованном saveset-е, неверный тег) и был единственным плохим блоком своей группы; он восстановлен по остальным блокам группы и её блоку XOR. Ничего не потеряно.

Действие пользователя: Для файлов — никаких. Носитель может выходить из строя: скопируйте saveset на другой носитель.

BLKFORGED, Block: *block*, Volume: *volume* - is not what was written: its CRC is right, its authentication fails

Средство: VBACKUP. **Уровень:** предупреждение (W).

Пояснение: Блок зашифрованного saveset-а имеет верную контрольную сумму, но его тег аутентификации не совпадает: блок изменён умышленно (с исправлением CRC) либо носитель ведёт себя очень странно. Блок считается плохим: если его удалось восстановить по группе, далее следует BLKFIXED, иначе BLKLOST.

Действие пользователя: Если следует BLKFIXED, файлы верны. Выясните, кто мог писать в saveset.

BLKLOST, Block: *block*, Volume: *volume* - is bad and cannot be rebuilt

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Блок saveset-а испорчен, а в его группе есть ещё один плохой блок (или в saveset-е нет блоков XOR), поэтому восстановить его нельзя. Чтение продолжается со следующего исправного блока. Файлы, данные которых лежали в потерянных блоках, сообщаются FILDAMAGED или FILLOST; все остальные файлы верны.

Действие пользователя: Оставьте восстановленные файлы; повреждённые восстановите из другого saveset-а. В следующий раз задайте меньший `/GROUP_SIZE` и храните две копии важных saveset-ов.

CMPSUMM, Files: *n*, Differences: *n* - compared

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: Итоги сравнения (`/COMPARE`, `/VERIFY`): число сравнённых файлов и число файлов с различиями — среди них файлы каталога, которых поток записей не дал (COMPARERR `not compared`). Выводится без `/LOG`. После копирования с `/VERIFY` выводится, только если найдены различия.

Действие пользователя: Никаких. При различиях см. COMPARERR.

COMPARED, File: *file* - compared

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: С `/LOG`: файл сравнён с диском, различий не найдено.

Действие пользователя: Никаких.

COMPARERR, File: *file* - *difference*

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: `/COMPARE` или `/VERIFY` сохранения или копирования нашли различие между saveset-ом (или копией) и диском. *difference* — одно из: `not found on the disk`, `not a regular file on the disk`, `not a directory on the disk`, `not a symbolic link on the disk`, `the link points elsewhere`, `not a FIFO on the disk`, `not a device on the disk`, `the size differs`, `the file on the disk is shorter`, `the contents differ at octet n`, а для копирования — `cannot be read back` или `the copy differs from the original`. Сообщается только первое различие файла.

Начиная с X01-22 каталог сверяется с прочитанными записями, и файл каталога, которого поток записей не дал, — тоже различие: `not compared: its records were lost in bad blocks` после потерянных блоков (см. FILLOST), `not compared: the catalog has it, the record stream does not`, если не был потерян ни один блок (см. NOTINSTREAM). Поэтому `/VERIFY` не может сказать `Differences: 0` над файлами, потерянными в плохих блоках.

Действие пользователя: Если файл после сохранения изменён намеренно – никаких. Иначе сохраните его снова или выясните, что его изменило. Для `not compared` см. FILLOST или NOTINSTREAM. При `/VERIFY` различие не даёт `/DELETE` ничего удалить, а `/RECORD` — записать сохранение.

COMPLETED, Operation: *operation*, Seconds: *s.cc* - *outcome*

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: Операция (save, restore, compare, extract, copy, rebuild of the journal, copy of a saveset) завершилась за указанное время. *outcome* — `completed`, `completed with warnings` или `completed with errors`, в согласии с кодом завершения. Выводится без `/LOG`; листинг его не выводит.

Действие пользователя: Никаких; если итог не `completed`, смотрите предупреждения и ошибки выше.

CONFQUAL, Qualifiers: /*qualifier*, /*qualifier* - cannot be given together

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Заданы два взаимоисключающих квалификатора. Такие пары: `/INCREMENTAL` с `/SELECT` или `/EXCLUDE`; более одного из `/MODIFIED`, `/CREATED`, `/CHANGED`; `/FULL` с `/BRIEF`; `/PHYSICAL` с `/SINCE`, `/BEFORE`, `/RECORD`, `/INCREMENTAL`, `/BY_OWNER`, `/SELECT` или `/EXCLUDE`; `/IMAGE` с `/PHYSICAL`, `/INCREMENTAL`, `/BY_OWNER`, `/SELECT` или `/EXCLUDE`; `/ORIGINAL` с `/PHYSICAL` или `/IMAGE`; `/DELETE` с `/PHYSICAL`, `/IMAGE`, `/SINCE` или `/BEFORE`; `/IMAGE` с `/CONFIRM` при восстановлении. Ничего не сделано.

Действие пользователя: Уберите один из квалификаторов.

COPIED, File: *file* - copied

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: С `/LOG`: файл скопирован операцией копирования.

Действие пользователя: Никаких.

CPYSUMM, Files: *n*, Bytes: *n* - copied

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: Итоги копирования: число созданных файлов и записанных байтов данных. Выводится без `/LOG`.

Действие пользователя: Никаких.

CRCERR, File: *file* - checksum mismatch: the data differ from what was saved

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Данные восстановленного или извлечённого файла не совпадают с CRC, записанной при сохранении, хотя все блоки прочитаны верно. Файл записан, но его содержимое — не то, что сохранялось. Это же сообщение даёт saveset со сжатыми данными, читаемый VBACKUP версии до X01-04.

Действие пользователя: Не доверяйте файлу; восстановите его из другого saveset-а. Используйте текущую версию VBACKUP. Если saveset создан и прочитан одной и той же версией VBACKUP, сообщите о проблеме.

CREATED, Volume: *volume* - created

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: Создан том saveset-а. О томе 1 сообщается только при `/LOG`; о каждом следующем — всегда. О следующих томах saveset-а, записываемого на стандартный вывод, сообщается как `Volume: (standard output)`: в потоке они идут друг за другом.

Действие пользователя: Никаких. Если тома идут на сменные носители, храните их вместе.

DELETED, File: *file* - deleted: it is not in the incremental saveset

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: Восстановление `/INCREMENTAL` удалило файл из каталога saveset-а, потому что каталог saveset-а его не перечисляет: файл был удалён до этого сохранения. Выводится при `/LOG`.

Действие пользователя: Никаких.

DELSUMM, Deleted: *n*, Kept: *n*

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: Итоги `/DELETE`: число файлов, удалённых после сохранения и проверки, и число оставленных.

Действие пользователя: Если файлы оставлены, см. SRCKEPT.

ENCRYPTED, Saveset: *saveset*, Iterations: *n* - encrypted: ChaCha20, HMAC-SHA256, PBKDF2

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: С `/LOG`: saveset записывается зашифрованным; *n* — число итераций PBKDF2 для его ключей.

Действие пользователя: Никаких. Если *n* меньше 600000, задана `VBACKUP_KDFITER`: уберите её для настоящих saveset-ов.

FATALSAVE, Saveset: *saveset* - could not be completed

Средство: VBACKUP. **Уровень:** фатальная ошибка (F).

Пояснение: Saveset не удалось записать до конца: не записался том (перед этим — WRITERR), не записался каталог или кончилась память. Saveset оставлен как есть, без TRAILER; он не проверяется, и ничего не записывается в журнал и не удаляется.

Действие пользователя: Устраните сообщённую ранее причину (чаще всего – переполненный диск) и сохраните снова. Файлы, записанные до обрыва, можно восстановить из неполного saveset-а.

FILCHANGED, File: *file* - changed while it was being saved

Средство: VBACKUP. **Уровень:** предупреждение (W).

Пояснение: Размер, время изменения или время смены атрибутов файла после чтения отличались от прежних: в файл писали во время сохранения, и копия может оказаться смесью старого и нового. Файл сохраняется в состоянии CHANGED и не записывается в журнал, поэтому следующее `/SINCE=BACKUP` сохранит его снова. Копирование сообщает об этом так же.

Действие пользователя: Сохраните файл снова, когда в него никто не пишет, или сохраняйте со снимка (LVM, btrfs).

FILDAMAGED, File: *file* - is incomplete: its data was lost in bad blocks

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Часть данных файла лежала в потерянных блоках (BLKLOST, MISSVOL); файл восстановлен, сравнён или извлечён настолько, насколько его данные удалось прочитать. При восстановлении `/PHYSICAL` у устройства потеряны части. Начиная с X01-22 оно называет и файл, на котором потерянные блоки или конец прерванного saveset-а разрезали запись SOLID (раздел 3.7): файлы перед ним в записи SOLID восстановлены целиком, сам он — настолько, насколько дошёл, файлы после него — FILLOST. Это же сообщение даёт saveset со сжатыми данными, читаемый VBACKUP версии до X01-04.

Действие пользователя: Восстановите файл из другого saveset-а. Все файлы, не названные в сообщениях, верны.

FILEEXISTS, File: *file* - already exists, not restored

Средство: VBACKUP. **Уровень:** предупреждение (W).

Пояснение: Восстанавливаемый, копируемый или извлекаемый файл существует, а `/REPLACE` не задан; существующий файл оставлен.

Действие пользователя: Задайте `/REPLACE`, чтобы перезаписать его, или восстанавливайте в другой каталог.

FILLOST, File: *file* - not restored: its records were lost in bad blocks

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Каталог перечисляет файл, который так и не появился в потоке: его запись FILE была в потерянных блоках. Файла нет в выходе.

Много FILLOST сразу после одного плохого места (BLKLOST или BADREC) означают, что задета запись SOLID (раздел 3.7): мелкие файлы сжатого сохранения хранятся группами до 256 КБ (в X01-21 — до 1 МБ), и потерянные блоки уносят файлы группы начиная с места повреждения и все файлы группы, которая в них начинается, — от нескольких до пары десятков, а не один. Начиная с X01-22 файлы группы до повреждения восстанавливаются, каждый проверяется по CRC своей записи FEND, и для saveset-ов X01-21 тоже; файл на разрезе называется в FILDAMAGED. Здесь называется каждый потерянный; остальные файлы восстановлены.

`/COMPARE` и `/VERIFY` сообщают такой файл в COMPARERR `not compared: its records were lost in bad blocks` и считают его различием.

Действие пользователя: Восстановите файлы из другого saveset-а. Для saveset-ов, которые должны пережить сильные повреждения, в следующий раз сохраняйте с `/PARITY=2` или больше либо с `/NOSOLID`.

GLUED, Parameter: *word* - the qualifiers glued to it are taken as qualifiers

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: К параметру были приклеены квалификаторы, как в `x.sav/sav/log`; VBACKUP разделил их, как это делает DCL. Сообщение выводится, чтобы опечатка в имени файла не была незаметно принята за квалификатор.

Действие пользователя: Никаких, если так и задумано. Если слово было именем файла, задайте его как есть (имя существующего файла никогда не разрезается) или отделите квалификаторы пробелами.

IMGCMD, Command: *command*

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: С `/LOG`: восстановление `/IMAGE` сейчас запустит команду, создающую новую файловую систему.

Действие пользователя: Никаких.

IMGMKFS, Command: *command* - failed: *reason*

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Команда, создающая новую файловую систему при восстановлении `/IMAGE`, не выполнилась. *reason* — `the program is not installed`, последняя выведенная программой строка, `it failed` или причина, по которой программу не удалось запустить. Ничего не восстановлено.

Действие пользователя: Установите программу (e2fsprogs, xfsprogs, btrfs-progs, dosfstools) или устраните сообщённую ею причину и восстановите снова.

IMGMOUNT, Type: *type*, Device: *device*, errno: *n* - the new file system cannot be mounted (*reason*)

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Только что созданную на устройстве при восстановлении `/IMAGE` файловую систему не удалось смонтировать во временный каталог. Ничего не восстановлено.

Действие пользователя: Убедитесь, что ядро поддерживает этот тип файловой системы и что VBACKUP запущен от root.

IMGNOID, Volume: *mount-point* - the *UUID* of its file system is not known: the new one gets a new one

Средство: VBACKUP. **Уровень:** предупреждение (W).

Пояснение: Сохранение `/IMAGE` не нашло UUID файловой системы (ни в `/dev/disk/by-uuid`, ни, для файловых систем ext, в суперблоке). Saveset создаётся; восстановление `/IMAGE` создаст файловую систему с новым UUID.

Действие пользователя: Если UUID важен (например, на него ссылается `/etc/fstab`), установите его на новой файловой системе после восстановления.

IMGNOTIMG, Saveset: *saveset* - was not made with /IMAGE: restore it without /IMAGE

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Восстановлению `/IMAGE` задан saveset без идентичности файловой системы.

Действие пользователя: Восстановите saveset без `/IMAGE`, в каталог.

IMGNOTMNT, Device: *device* - is not mounted: mount it (read-only is enough) and give the mount point or the device

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Вход сохранения `/IMAGE` — несмонтированное блочное устройство; VBACKUP читает файлы через смонтированную файловую систему.

Действие пользователя: Смонтируйте устройство, можно только на чтение, и сохраните снова.

IMGNOTVOL, File: *file* - is neither the mount point of a file system nor a device: /IMAGE saves a whole volume

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Вход сохранения `/IMAGE` — не точка монтирования файловой системы (например, подкаталог) и не смонтированное блочное устройство.

Действие пользователя: Задайте точку монтирования или сохраните каталог без `/IMAGE`.

IMGSMALL, Device: *device*, Bytes: *n*, Needed: *n* - too small, nothing written

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Выходное устройство восстановления `/IMAGE` меньше места, занятого сохранённой файловой системой, плюс 5%, плюс 16 МБ на метаданные новой файловой системы.

Действие пользователя: Используйте устройство большего размера.

IMGSUMM, Device: *device*, Type: *type*, Files: *n*, Bytes: *n* - file system made, files restored

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: Итоги восстановления `/IMAGE`: файловая система создана на устройстве, файлы восстановлены в неё.

Действие пользователя: Никаких.

IMGUNSUPP, Saveset: *saveset*, Type: *type* - VBACKUP does not make such a file system: use /PHYSICAL for it

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Saveset содержит файловую систему типа, для которого VBACKUP не умеет создавать новую файловую систему (создаются типы ext2, ext3, ext4, xfs, btrfs, vfat, msdos).

Действие пользователя: Восстановите файлы в каталог без `/IMAGE` или сохраняйте такие тома с `/PHYSICAL`.

INCRSUMM, Files: *n* - unchanged, listed as present, not saved

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: Инкрементное сохранение (задан фильтр времени) перечислило в каталоге *n* охваченных файлов как присутствующие, не сохраняя их.

Действие пользователя: Никаких.

IVOP, cannot tell what to do: *reason*

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Операцию нельзя определить по параметрам: вход не существует (а значит, не является saveset-ом), а выход не является именем saveset-а.

Действие пользователя: Проверьте входную спецификацию. Для сохранения закончите имя выхода на `.bck` или `.sav` либо задайте `/SAVE_SET`.

IVQUAL, Value: *value*, Qualifier: /*qualifier* - is not valid

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Недопустимое значение квалификатора: `/BLOCK_SIZE` вне пределов 8192..1048576 или не кратно 512; `/GROUP_SIZE` больше 100; `/VOLUME_SIZE` — не размер или меньше (размер группы + 3) x размер блока; `/BY_OWNER` или `/OWNER` называет неизвестного пользователя; `/DATA_FORMAT` с неизвестным ключевым словом.

Действие пользователя: Исправьте значение.

IVTIME, Time: *value*, Qualifier: /*qualifier* - is not a valid time

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Значение `/SINCE` или `/BEFORE` — не время, понятное VBACKUP.

Действие пользователя: Запишите время, как сказано в описании `/BEFORE` (раздел 5.2).

JNLERR, Journal: *journal* - *what*, errno: *n* (*reason*)

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Журналом нельзя воспользоваться. *what* — одно из: `no journal given, and no home directory`; `cannot make its directory`; `cannot be opened` (журнал или его файл блокировки); `cannot be locked`; `cannot be read, or is too short`; `is no journal, or is damaged`; `cannot be created`, `cannot be written` (временный файл); `cannot be replaced`. Сохранение с `/SINCE=BACKUP` или `/RECORD`, не сумевшее открыть журнал, не выполняется.

Действие пользователя: Устраните причину, указанную в *reason*. Испорченный журнал можно перестроить: удалите его и задайте saveset-ы с `/RECORD`.

KEYFILE, Key file: *file* - *reason*

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Пароль нельзя взять из файла ключа или с терминала (тогда *file* — `/dev/tty`). *reason* — причина, по которой файл не удалось открыть или прочитать; `not a regular file`; `others may read or change it - chmod 600 it`; `its first line is longer than 1024 bytes`; `its first line is empty`; для терминала — `an empty passphrase` или `longer than 1024 bytes, or no line`. Ничего не прочитано и не записано.

В Windows права файла ключа не проверяются: кто может его читать, решает его ACL.

Действие пользователя: Сделайте файл ключа обычным файлом, доступным только владельцу (`chmod 600 file`), с паролем в первой строке.

KEYMATCH, the two passphrases differ: nothing saved

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Сохранение `/ENCRYPT` дважды запросило пароль на терминале, и ответы не совпали. Saveset не создан.

Действие пользователя: Сохраните снова и дважды введите один и тот же пароль.

MAXPARM, Parameter: *word* - one too many: only an input and an output are taken; a qualifier begins with /

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: В команде больше двух параметров. Чаще всего квалификатор набран без косой черты (`.log` вместо `/LOG`) или список входов содержит пробелы. Ничего не сделано.

Действие пользователя: Исправьте команду; разделяйте несколько входов запятыми без пробелов.

MISSING, File: *file* - should be there from an earlier saveset, and is not

Средство: VBACKUP. **Уровень:** предупреждение (W).

Пояснение: При восстановлении `/INCREMENTAL` каталог перечисляет файл как присутствующий (не изменившийся с прежнего сохранения), но ни один предыдущий saveset цепочки не поместил его в выход.

Действие пользователя: Чаще всего пропущен saveset цепочки. Задайте их все: сначала полный, затем инкрементные от старейшего.

MISSVOL, Volume: *n*, Saveset: *saveset* - is missing

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Том saveset-а не найден рядом с первым, или поток из нескольких томов перешёл от одного тома к более позднему (saveset тогда называется -). Файлы, целиком лежащие в остальных томах, всё равно восстанавливаются; о прочих сообщается. Копирование saveset-а сообщает о томе, которого не нашло, и копирует остальные.

Действие пользователя: Поместите все тома в один каталог, не меняя их имён, и повторите операцию. Для потока выясните, почему отправитель пропустил том (его собственный MISSVOL).

NOCATALOG, Saveset: *saveset* - has no catalog: it is listed by reading it whole

Средство: VBACKUP. **Уровень:** предупреждение (W).

Пояснение: `/LIST` не нашёл пригодного каталога (у saveset-а нет TRAILER или его каталог не читается) и выводит вместо этого записи FILE потока, что дольше. Для saveset-а из канала не выводится.

Действие пользователя: Для листинга — никаких. См. NOTRAILER.

NOFILES, Pattern: *specification* - no file matches it

Средство: VBACKUP. **Уровень:** предупреждение (W).

Пояснение: Входная спецификация сохранения или копирования не выбрала ни одного файла, либо её базу не удалось открыть (перед этим — OPENIN).

Действие пользователя: Проверьте спецификацию и её шаблоны; заключите шаблоны в кавычки, чтобы их не раскрыл командный интерпретатор.

NOINODE, Saveset: *saveset*, Files: *n* - not recorded: the catalog has no inode data (written before X01-02)

Средство: VBACKUP. **Уровень:** предупреждение (W).

Пояснение: При перестройке журнала найдено *n* элементов каталога без inode и времени смены атрибутов, нужных журналу; saveset записан до X01-02. Эти файлы не записываются.

Действие пользователя: Никаких; следующее `/SINCE=BACKUP` сохранит эти файлы снова.

NOJOURNAL, Journal: *journal* - is not there yet: /SINCE=BACKUP saves everything

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: Сохранение `/SINCE=BACKUP` не нашло журнала; сохраняется каждый охваченный файл, как в первый раз.

Действие пользователя: Никаких, если это первое сохранение плана. Иначе проверьте `/JOURNAL` и пользователя: у каждого пользователя свой журнал.

NOKEY, Saveset: *saveset* - needs a passphrase, and there is no terminal to ask it on: give /KEY_FILE=file or VBACKUP_KEY_FILE

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Saveset зашифрован (или должен быть зашифрован), файл ключа не задан, а спросить негде (cron, канал, файловый менеджер), либо `VBACKUP_NOPROMPT=1` запрещает спрашивать.

Действие пользователя: Задайте `/KEY_FILE=file` или установите `VBACKUP_KEY_FILE`.

NOMEM, errno: *n* - cannot allocate memory (*reason*)

Средство: VBACKUP. **Уровень:** фатальная ошибка (F).

Пояснение: Не удалось выделить память. Операция прекращается; идущее сохранение завершается с FATALSAVE.

Действие пользователя: Освободите память или увеличьте ограничения процесса и повторите операцию.

NOPARAM, Parameter: *parameter* - is missing

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Не задан обязательный параметр: входная спецификация, выходная спецификация, выходной каталог восстановления или выходное устройство восстановления `/PHYSICAL`.

Действие пользователя: Задайте параметр. Чтобы восстановить saveset туда, откуда пришли его файлы, задайте `/ORIGINAL`.

NORMAL, normal successful completion

Средство: VBACKUP. **Уровень:** успех (S).

Пояснение: Условие успеха средства. VBACKUP его не выводит; об успехе говорят COMPLETED и код завершения 0.

Действие пользователя: Никаких.

NOTFOUND, File: *name* - is not in the saveset

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: `/EXTRACT` называет файл, которого нет в saveset-е. Имя должно быть точным, как его показывает `/LIST`, без шаблонов.

Действие пользователя: Выведите листинг saveset-а и задайте хранимое имя.

NOTINCR, Saveset: *saveset* - *reason*: nothing restored with /INCREMENTAL

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Восстановление `/INCREMENTAL` отвергло saveset, потому что не может доверять его списку файлов: `it does not say whether it is full or incremental (written before X01-02)`, `it has no catalog`, `its catalog cannot be read` или `its catalog is damaged`. Из него ничего не восстанавливается, и на нём цепочка обрывается.

Действие пользователя: Восстановите этот saveset без `/INCREMENTAL`. Saveset-ы, созданные X01-02 и позднее, содержат всё, что нужно `/INCREMENTAL`.

NOTINSTREAM, File: *file* - not restored: the catalog has it, the record stream does not, though no block was lost

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: После чтения потока записей каталог сверен со встреченными записями FILE (начиная с X01-22 — всегда, а не только после потерянных блоков), и в нём назван файл, записи которого так и не пришли, — хотя ни один блок не потерян. Файла нет в выходе. С saveset-ом, который VBACKUP записал правильно и никто не менял, этого не бывает: в потоке есть запись, которую этот читающий не знает (например, запись более новой VBACKUP в saveset-е старой версии формата), saveset правили или подделали, либо он сделан неправильно. Из канала записи каталога берутся по мере поступления. Завершение — как при ошибке, как и с FILLOST. `/COMPARE` и `/VERIFY` сообщают тот же файл в COMPARERR `not compared: the catalog has it, the record stream does not`; vbkx говорит `File: `*file*` - not extracted: the catalog has it, the record stream does not`.

Действие пользователя: Попробуйте более новую VBACKUP. Иначе восстановите файл из другого saveset-а и сообщите о проблеме, приложив saveset.

NOTOPIC, Topic: *topic* - sorry, no documentation on it

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: `/HELP` называет тему, которой нет в библиотеке справки.

Действие пользователя: Введите `vbackup /HELP`, чтобы увидеть темы.

NOTRAILER, Saveset: *saveset* - has no trailer: the save did not complete, or its last volume is missing

Средство: VBACKUP. **Уровень:** предупреждение (W).

Пояснение: Последний блок saveset-а — не TRAILER: сохранение было прервано (диск переполнился, программу остановили), отсутствует последний том или канал оборвался раньше конца saveset-а — для потока из нескольких томов раньше, чем пришёл его последний том. Файлы до места обрыва можно восстановить; saveset читается последовательно. Перестройка журнала такой saveset пропускает. Копирование saveset-а сообщает об этом, скопировав то, что пришло, и у его копии тоже нет TRAILER.

Действие пользователя: Найдите недостающий том, если он есть. Иначе восстановите то, что можно, и создайте новый saveset.

NOTSAVESET, File: *file* - is not a saveset

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: `/LIST`, `/COMPARE` или `/EXTRACT` задан вход, не являющийся saveset-ом, или более одного входа; либо файл, читаемый как saveset, им не является; либо вход копирования saveset-а начинается не с заголовка тома 1. VBACKUP, vbkx или модуль до X01-21 говорит это о saveset-е формата версии 3 — записанном со сжатием версией X01-21 или новее без `/NOSOLID` (см. `/SOLID`); до X01-14 — о saveset-е версии 2 (`/PARITY=2` или больше). С *узел*`::`*файл* файл — это `-`: VBACKUP там ничего не прислал, и его сообщения выше говорят почему.

Действие пользователя: Задайте первый том одного saveset-а. Для saveset-а версии 3 или 2 возьмите VBACKUP X01-21 или новее; для старого читающего сохраняйте с `/NOSOLID` (и без `/PARITY` от 2).

OPENDIR, Directory: *directory*, errno: *n* - cannot be read (*reason*)

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Каталог, встреченный при сохранении или копировании, не удалось открыть или прочитать; его содержимое не сохраняется.

Действие пользователя: Выполните сохранение от пользователя, который может читать каталог (root), или исключите каталог.

OPENIN, File: *file*, errno: *n* - cannot be opened as input (*reason*)

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Файл не удалось открыть на чтение: файл или база спецификации при сохранении или копировании, saveset, устройство сохранения `/PHYSICAL`, вход сохранения `/IMAGE`, сравниваемый файл; либо имя слишком длинное (errno 36). Если вход команды не существует и выход не задан, OPENIN сообщает об этом с errno 2.

Действие пользователя: Проверьте имя и права доступа.

OPENOUT, File: *file*, errno: *n* - cannot be created as output (*reason*)

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Файл не удалось создать: том saveset-а (errno 17, File exists, если он существует и `/REPLACE` не задан), восстанавливаемый, копируемый или извлекаемый файл, файл листинга `/LIST=file`, временный спул каталога, выход восстановления `/PHYSICAL` или `/IMAGE`. С errno 22 и причиной `a name that leads out of the output directory` saveset содержит имя с `..` или ведущей `/`; такой файл никогда не записывается. Имя, путь к которому проходит через символическую ссылку, отвергается так же. В Windows errno 22 с `not a valid name on Windows` — имя, которого Windows не может хранить (раздел 3.17), а errno 17 с `a name that differs in case only is there` — второе имя одного и того же файла.

Действие пользователя: Проверьте права доступа и свободное место; задайте `/REPLACE`, чтобы перезаписать saveset. Имя, ведущее за пределы выходного каталога, означает, что saveset создан не VBACKUP или создан во вред.

ORIGNOBASE, Saveset: *saveset* - does not say where its files came from (made before X01-02): give an output directory

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Восстановлению `/ORIGINAL` задан saveset, не хранящий свои базы в виде абсолютных имён.

Действие пользователя: Восстановите saveset в выходной каталог.

ORIGTARGET, Saveset: *saveset*, Target: *directory* - its files go back there

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: Восстановление `/ORIGINAL` сейчас вернёт файлы saveset-а в указанный каталог; по сообщению на каждую базу.

Действие пользователя: Никаких; если это не то место, немедленно нажмите Ctrl/C.

PARITYERR, Block: *n*, Volume: *n* - the group beginning here does not agree with its parity: a block of it with a right CRC holds other bytes, nothing of it is rebuilt

Средство: VBACKUP. **Уровень:** предупреждение (W).

Пояснение: Saveset сделан с `/PARITY`: блоки группы, которая начинается с названного блока, были плохими, и любой способ восстановить их по блокам чётности оставлял

строку, которая не сходится, — какой-то блок группы с верным CRC содержит не те байты, что были записаны. Ничего из группы не восстанавливается; её плохие блоки потеряны (BLKLOST), а файлы в них названы (FILDAMAGED).

Действие пользователя: Сохраните saveset: остальные группы не затронуты. Выясните, как на носителе оказался такой блок.

PHYSABORT, Device: *device* - not overwritten: the answer was not YES

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Восстановление `/PHYSICAL` или `/IMAGE` на устройство спросило на терминале `YES`, а ответ был иным. Ничего не записано.

Действие пользователя: Никаких, если устройство было не то. Иначе повторите и ответьте `YES` прописными буквами.

PHYSHELD, Device: *device* - is in use (*holder*): free it first, or save what uses it

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Устройство сохранения `/PHYSICAL` или выходное устройство восстановления `/PHYSICAL` или `/IMAGE` занято: `held by` устройство device-mapper или RAID (физический том LVM, член RAID, контейнер dm-crypt), используется как `swap`, либо `the kernel says it is busy`.

Действие пользователя: Освободите устройство (деактивируйте группу томов, остановите массив, закройте контейнер, `swapoff`) или сохраните устройство, которое его использует.

PHYSLARGER, Device: *device*, Bytes: *n*, Saved: *n* - larger: the rest stays as it is, the file system keeps its old size

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: Выходное устройство восстановления `/PHYSICAL` больше сохранённого. Образ записывается в его начальную часть; остальное не трогается.

Действие пользователя: Если нужно место, увеличьте файловую систему (`resize2fs`, `xfs_growfs`, …).

PHYSMOUNTED, Device: *device* - is mounted[read-write] on *mount-point*: *advice*

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Сохранению `/PHYSICAL` задано устройство, смонтированное на чтение и запись (*advice*: `unmount it, mount it read-only, or save a snapshot`), или восстановлению `/PHYSICAL` либо `/IMAGE` задано смонтированное выходное устройство (*advice*: `nothing is written to a mounted device, unmount it`). *device* может содержать `(a partition of it)`. Устройство, в которое пишут во время копирования, даёт испорченную копию, выглядящую исправной, пока она не понадобится.

Действие пользователя: Последуйте совету.

PHYSNOTDEV, File: *file* - is neither a block device nor a file

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Вход сохранения `/PHYSICAL` или выход восстановления `/PHYSICAL` — ни блочное устройство, ни обычный файл; либо выход восстановления `/IMAGE` — не блочное устройство.

Действие пользователя: Задайте блочное устройство или файл образа.

PHYSNOTPHYS, Saveset: *saveset* - was not made with /PHYSICAL: restore it without /PHYSICAL

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Восстановлению `/PHYSICAL` задан saveset, не содержащий устройства.

Действие пользователя: Восстановите его без `/PHYSICAL`.

PHYSREPLACE, Device: *device* - everything on it would be overwritten: give /REPLACE to do so

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Восстановление `/PHYSICAL` или `/IMAGE` на блочное устройство задано без `/REPLACE`. Ничего не записано.

Действие пользователя: Убедитесь, что устройство то самое, и задайте `/REPLACE`.

PHYSSIZE, Device: *device*, Bytes: *n*, then: *n* - changed its size while it was read: the copy is not to be trusted

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Размер устройства сохранения `/PHYSICAL` в конце копирования отличался от размера в начале.

Действие пользователя: Выясните, что изменило устройство (изменение размера, горячее подключение), и сохраните его снова.

PHYSSMALL, Device: *device*, Bytes: *n*, Saved: *n* - too small, nothing written

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Выходное устройство восстановления `/PHYSICAL` меньше сохранённого.

Действие пользователя: Используйте устройство не меньшего размера; либо восстановите в файл образа или, если saveset таков, с `/IMAGE`.

PHYSSUMM, Device: *device*, Bytes: *n*, Data: *n* - the rest zeros

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: Итоги сохранения или восстановления `/PHYSICAL`: размер устройства и число байтов данных, не являющихся нулями.

Действие пользователя: Никаких.

PHYSUUID, Device: *device* - now carries the labels and UUIDs of the device saved: never mount it beside the original

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: Восстановление `/PHYSICAL` на устройство или восстановление `/IMAGE`, сохранившее UUID, создало устройство, которое для системы выглядит как оригинал.

Действие пользователя: Не монтируйте копию и оригинал одновременно: xfs это отвергает, btrfs может испортить обе. Если нужны обе, смените UUID у одной из них.

QUALUSE, Qualifier: /*qualifier* - *reason*

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Квалификатор задан там, где его нельзя использовать; *reason* говорит почему: `/DELETE` без `/VERIFY`; `/ENCRYPT` при операции, отличной от сохранения; `/VERIFY` или `/LIST` со стандартным выводом в качестве saveset-а (`a saveset written to the standard output is gone once written: it cannot be read back here`); `/DELETE` или `/LIST` с saveset-ом на другом узле (`not with a saveset made on another node`); `/TRANSFER`, когда команда — не копирование saveset-а (`the input must be one saveset (or -), and there must be an output`); `/PHYSICAL` или `/IMAGE` с более чем одним входом (`one input - one device or volume - a saveset`) или более чем одним saveset-ом (`one saveset at a time`); `/ORIGINAL` с выходным параметром или с `/INCREMENTAL` и saveset-ом из нескольких баз.

В Windows `not on Windows` говорится о `/PHYSICAL` и `/IMAGE`: в `vbackup.exe` их нет (раздел 3.17).

Действие пользователя: Исправьте команду, как говорит причина.

READERR, File: *file*, errno: *n* - cannot be read (*reason*)

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Ошибка чтения: сохраняемого файла (он сохраняется в состоянии READERR до места ошибки и не записывается в журнал), копируемого или сравниваемого файла либо временного спула каталога при сохранении.

Действие пользователя: Проверьте устройство и файл; сохраните файл снова.

RECORDED, Files: *n*, Journal: *journal* - recorded

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: Журнал обновлён сохранением `/RECORD` (записано *n* файлов) или перестроен по каталогам.

Действие пользователя: Никаких.

REMOTE, Node: *node*, errno: *n* - the pipe to VBACKUP there cannot be made (*reason*)

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Для saveset-а на другом узле, *узел*::*файл*, VBACKUP не смог создать канал к VBACKUP там или запустить процесс для ssh (или для команды `VBACKUP_RSH`). Ничего не отправлено и не прочитано. Команда, которую нельзя запустить, или недоступный узел сообщаются не здесь, а REMOTEERR.

В Windows — errno 40 и `not on Windows yet`: в этой версии `vbackup.exe` не работает с saveset-ами на других узлах.

Действие пользователя: Чаще всего системе не хватает процессов или дескрипторов файлов: смотрите *reason* и повторите операцию.

REMOTEERR, Node: *node*, Exit: *n* - VBACKUP there did not complete: see its messages above

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: VBACKUP на другом узле или запускающий его ssh завершился с кодом *n* (2 — VBACKUP там выдал ошибку; 127 — команду `VBACKUP_RSH` нельзя запустить здесь или `vbackup` нет в PATH там; 255 — отказал ssh: узел недоступен или не пускает этот), либо был завершён сигналом (*n* = 128 + сигнал). Код 1 (предупреждения) не сообщается: он делает код завершения не меньше 1. Если другая сторона остановилась во время сохранения, этому сообщению предшествуют WRITERR (errno 32, Broken pipe) и FATALSAVE.

Действие пользователя: Прочитайте сообщения другой стороны выше. Проверьте, что `ssh`*узел*`vbackup` выводит краткую справку, ничего не спрашивая, что VBACKUP там — X01-11 или новее (более ранний отвергает `/TRANSFER`) и что saveset там можно создать или прочитать.

RESTORED, File: *file* - restored

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: С `/LOG`: файл восстановлен.

Действие пользователя: Никаких.

RESTSUMM, Files: *n*, Bytes: *n* - restored

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: Итоги восстановления: число созданных файлов и каталогов и записанных байтов данных по всем saveset-ам команды. Выводится без `/LOG`.

Действие пользователя: Никаких.

SAVED, File: *stored-name* - saved

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: С `/LOG`: файл сохранён.

Действие пользователя: Никаких.

SAVESUMM, Files: *n*, Bytes: *n*, Blocks: *n*, Volumes: *n* - saved

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: Итоги сохранения: сохранённые файлы, сохранённые байты данных, записанные блоки всех типов, тома. Выводится без `/LOG`.

Действие пользователя: Никаких.

SKIPPED, File: *stored-name* - skipped: *reason*

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: С `/LOG`: файл или каталог не включён в сохранение или копирование; *reason* — `nodump flag set`.

Действие пользователя: Никаких. Чтобы сохранить такие файлы, задайте `/IGNORE=NOBACKUP`.

SRCDELETED, File: *file* - deleted: it is in the saveset and verified

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: С `/DELETE /LOG`: файл удалён после сохранения и проверки.

Действие пользователя: Никаких.

SRCKEPT, File: *file* - not deleted: *reason*

Средство: VBACKUP. **Уровень:** предупреждение (W).

Пояснение: `/DELETE` оставил файл. *reason* — `it changed after it was saved`, причина неудачи удаления либо — при *file*, равном `every file`, – `the saveset did not verify`.

Действие пользователя: Сохраните файл снова, если он изменился; при необходимости удалите его вручную.

STARTED, Operation: *operation*, Input: *input*[, Output: *output*] - started

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: Операция начинается. *operation* — `save`, `restore`, `compare`, `extract`, `copy`, `rebuild of the journal` или `copy of a saveset`. Стандартный ввод и вывод показываются как `(standard input)` и `(standard output)`; выход восстановления `/ORIGINAL` — как `(where its files came from)`; saveset на другом узле — как *узел*`::`*файл*. При нескольких входных спецификациях показываются все, как они заданы. Выводится без `/LOG`; листинг его не выводит. VBACKUP на другом узле выводит свой STARTED, с операцией `copy of a saveset`.

Действие пользователя: Никаких.

TOODEEP, Directory: *directory*, Levels: *n* - nested deeper, not descended

Средство: VBACKUP. **Уровень:** предупреждение (W).

Пояснение: Каталог, встреченный при сохранении или копировании, вложен глубже 256 уровней ниже базы; его содержимое не сохраняется.

Действие пользователя: Сохраните этот каталог отдельной спецификацией.

TOOMANY, Too many: *what*, Taken: *n* - the rest is ignored

Средство: VBACKUP. **Уровень:** предупреждение (W).

Пояснение: Задано более 64 входных спецификаций или более 64 шаблонов `/SELECT` или `/EXCLUDE`; берутся первые 64.

Действие пользователя: Используйте меньше спецификаций или шаблонов: шаблон, общий каталог, более широкий шаблон.

UNNAMED, Saveset: *saveset* - blocks were lost and *reason*: files missing from the restore cannot all be named

Средство: VBACKUP. **Уровень:** предупреждение (W).

Пояснение: При восстановлении потеряны блоки, а у saveset-а нет каталога, его каталог не читается или испорчен, поэтому файлы, записи которых потеряны, нельзя все назвать сообщениями FILLOST. Некоторых файлов в выходе может не быть.

Действие пользователя: Сравните выход с источником или с листингом более старого saveset-а.

UNSUPP, File: *file* - *what* not restored, errno: *n* (*reason*)

Средство: VBACKUP. **Уровень:** предупреждение (W).

Пояснение: Файл типа, который здесь нельзя создать, не восстановлен: `an unknown file type`; `a special file`, который не удалось создать (файл устройства, восстанавливаемый не от root, сокет); или для `/EXTRACT` – `anything but a regular file`.

В Windows так называется каждый FIFO, сокет и устройство из Linux-saveset-а (errno 40, Function not implemented): таких файлов в Windows нет.

Действие пользователя: Восстанавливайте файлы устройств от root. Извлекать можно только обычные файлы.

VERIFYING, Saveset: *saveset* - verifying

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: `/VERIFY`: только что записанный saveset читается обратно и сравнивается с диском.

Действие пользователя: Никаких.

VMSNOCRC, Saveset: *saveset* - written /NOCRC: its blocks carry no CRC, damage in them cannot be seen

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: Saveset OpenVMS BACKUP записан с `/NOCRC`: у его блоков нет CRC, и испорченный блок как таковой не распознаётся. XOR-блоки, если они есть, всё равно читаются.

Действие пользователя: Никаких. Где это важно, проверьте восстановленные файлы другими средствами.

VMSRAW, File: *file*, Organization: *org*, Record format: *rfm* - restored as it is on the VMS disk: its records are not converted

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: Файл saveset-а OpenVMS BACKUP восстановлен образом файла RMS: индексный или относительный файл, или файл с записями переменной длины без управления кареткой. Его записи без RMS не достать.

Действие пользователя: Скопируйте файл обратно на OpenVMS в двоичном режиме и восстановите там его атрибуты, или перед сохранением сделайте на OpenVMS `CONVERT` в последовательный файл.

VMSSAVESET, Saveset: *saveset*, Block size: *n*, Group size: *n* - an OpenVMS BACKUP saveset

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: На входе saveset, записанный OpenVMS BACKUP; он читается так, как описано в разделе 3.16.

Действие пользователя: Никаких.

WRITERR, File: *file*, errno: *n* - cannot be written (*reason*)

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Ошибка записи: тома saveset-а (сохранение затем завершается FATALSAVE), спула каталога, восстанавливаемого или извлекаемого файла, стандартного вывода, файла листинга или выходного устройства восстановления `/PHYSICAL`.

Действие пользователя: Чаще всего выход переполнен (errno 28): освободите место или разрежьте saveset на тома и повторите операцию.

WRONGKEY, Saveset: *saveset* - the passphrase does not open it

Средство: VBACKUP. **Уровень:** ошибка (E).

Пояснение: Заданный пароль не соответствует контрольному значению ключа зашифрованного saveset-а. Ничего не прочитано и не записано. Пароль не сохраняется: для следующего saveset-а команды он будет запрошен снова.

Действие пользователя: Проверьте файл ключа: учитывается только первая строка, регистр букв важен. Saveset, пароль которого утерян, не откроет никто.

WRONGVOL, Volume: *n*, Saveset: *saveset* - belongs to another saveset, ignored

Средство: VBACKUP. **Уровень:** предупреждение (W).

Пояснение: Файл с именем тома saveset-а принадлежит другому saveset-у (отличается идентификатор saveset-а) или вообще не является томом. Он пропускается; обычно далее следует MISSVOL для этого тома.

Действие пользователя: Поместите на его место нужный том. Не смешивайте тома saveset-ов, сохранённых под одним именем.

XATTRSKIP, File: *file* - an extended attribute not saved: it cannot be read, or is longer than 64 KB (on Windows: a stream)

Средство: VBACKUP. **Уровень:** предупреждение (W).

Пояснение: Файл сохранён, но один из его расширенных атрибутов — нет: его не удалось прочитать, или его значение длиннее 64 КБ — наибольшего значения расширенного атрибута Linux. В Windows расширенные атрибуты файла – его альтернативные потоки данных (`user.<поток>`), и поток длиннее 64 КБ не сохраняется.

Действие пользователя: Если поток важен, скопируйте его в отдельный файл (в Windows: `more < файл:поток > файл.поток`) или задайте `/NOXATTRS`, чтобы сохранять без расширенных атрибутов и без этого сообщения.

XFRSUMM, Blocks: *n*, Volumes: *n*, Bad: *n* - copied

Средство: VBACKUP. **Уровень:** информация (I).

Пояснение: Итоги копирования saveset-а: скопированные блоки, тома и плохие блоки среди них (о каждом сообщено BLKCOPIED). Выводится без `/LOG`, и при неудаче копирования — с тем, что успело скопироваться. VBACKUP на другом узле выводит его за свою сторону сохранения или восстановления через *узел*::*файл*.

Действие пользователя: Никаких, если Bad равно 0. Иначе см. BLKCOPIED.

ZCHECK, Offset: *n* - a record compressed did not decompress to the same octets: it is stored uncompressed (a fault of VBACKUP: report it)

Средство: VBACKUP. **Уровень:** предупреждение (W).

Пояснение: Каждая запись, которую VBACKUP сжал, тут же распаковывается и сравнивается с данными, из которых сделана (раздел 3.7). Эта не вернулась такой же; она записана несжатой, записью DATA. Saveset верен и восстанавливается; смещение — смещение записи в её файле. Для группы мелких файлов (записи SOLID, см. `/SOLID`) смещение 0, и группа пишется обычными записями, несжатой.

Действие пользователя: Сообщите об этом, назвав файл, `/LEVEL` и версию VBACKUP; больше всего поможет сам файл. Пока что `/LEVEL=1`, `/NOSOLID` (для группы) или `/DATA_FORMAT=UNCOMPRESSED` обходят этот кодек.

Глава 7. Устранение неполадок

Глава начинает с того, что видно, — признака или сообщения — и говорит, на что смотреть и что делать. Названные здесь сообщения полностью описаны в главе 6.

7.1 Сверка падает на занятых файлах

Признак. Сохранение с `/VERIFY` заканчивается кодом 2, и единственные ошибки — COMPARERR, часто после FILCHANGED.

Причина. Файлы изменились между сохранением и сверкой: база данных, журнал, дерево git работающей программы. В saveset-е то, чем файлы были в момент чтения; saveset не виноват.

Действие. Сохраняйте снимок файловой системы (LVM, btrfs, ZFS) вместо живой, или запускайте сохранение, когда с этими файлами никто не работает, или исключите их через `/EXCLUDE`. Командная процедура может повторить сохранение, если COMPARERR — единственная ошибка.

7.2 Не восстанавливаются владельцы и атрибуты

Признак. ATTRERR на многих файлах; файлы восстановлены и принадлежат тому, кто восстанавливал.

Причина. Восстановление запущено не от root, или файловая система вывода не может хранить атрибут (например, ACL на vfat).

Действие. Восстанавливайте от root; или задайте `/OWNER=DEFAULT`, чтобы взять файлы себе, и `/NOXATTRS`, если расширенные атрибуты не нужны (раздел 5.1, «Ограничения»).

7.3 Saveset испорчен

Признак. CRCERR, BLKFIXED, BLKLOST, FILDAMAGED или FILLOST при чтении saveset-а.

На что смотреть. Только BLKFIXED — испорченный блок восстановлен из своей группы: восстановлено всё. BLKLOST — испорченных блоков в группе больше, чем группа может восстановить; файлы, лежавшие в них, названы в FILDAMAGED или FILLOST, все остальные восстановлены. Файл никогда не восстанавливается с неверными данными без сообщения.

Действие.

1. Сначала снимите копию saveset-а с отказывающего носителя, если нужно — `dd conv=noerror,sync`.
2. Прочитайте копию снова: `vbackup copy.bck /LIST`, затем восстановите её.
3. Попробуйте автономный распаковщик и распаковщик последней надежды (приложение C): каждый читает блоки сам.
4. Для следующих saveset-ов задайте больший `/PARITY` и меньший `/GROUP_SIZE` (раздел 3.3).

7.4 Тома нет или он от другого saveset-а

Признак. MISSVOL, WRONGVOL, NOTRAILER.

Причина. Тома saveset-а ищутся рядом с первым, по имени (раздел 3.3). Том перенесли, переименовали или скопировали из другого saveset-а с тем же именем; или сохранение не завершилось.

Действие. Сложите все тома в один каталог под их исходными именами. Файлы, целиком лежащие в имеющихся томах, восстанавливаются в любом случае. У saveset-а незавершённого сохранения нет ни трейлера, ни каталога: он читается целиком, как поток (NOCATALOG).

7.5 Пароль спрашивается, не подходит или отсутствует

Признак. NOKEY в пакетном задании; WRONGKEY; KEYMATCH.

Действие.

- NOKEY: спросить негде — нет терминала. Задайте `/KEY_FILE=`*файл* или переменную `VBACKUP_KEY_FILE`; файл должен читаться только владельцем (`chmod 600`).
- WRONGKEY: в файле ключа значима только первая строка, и регистр букв важен. Проверьте, что это тот файл ключа, с которым делался saveset.
- KEYMATCH: два ответа на терминале различаются; сохраните снова.

Потерянный пароль не восстановить: другого пути в зашифрованный saveset нет. Храните файл ключа не там, где лежат saveset-ы.

7.6 Saveset на другом узле недоступен

Признак. REMOTE или REMOTEERR с кодом завершения.

На что смотреть. Код REMOTEERR: 2 — VBACKUP там сообщил об ошибке (его сообщения выведены выше); 127 — `vbackup` там не в PATH, или команду `VBACKUP_RSH` здесь нельзя запустить; 255 — ssh не удался: узел недоступен или не пускает этот по ключам.

Действие. Проверьте вручную `ssh` *узел* `vbackup /HELP`: команда должна выполниться без пароля. Другому узлу нужен VBACKUP X01-11 или новее (раздел 3.10).

7.7 Инкрементное восстановление отвергнуто

Признак. NOTINCR, ORIGNOBASE, MISSING.

Действие. Для `/INCREMENTAL` нужен saveset с каталогом, сделанный X01-02 или новее; восстановите сначала полный saveset, затем каждый инкрементный в порядке их создания, от старых к новым, — а не от новых к старым, как OpenVMS BACKUP (раздел 3.5). ORIGNOBASE: saveset, сделанный до X01-02, не говорит, откуда его файлы; задайте каталог вывода вместо `/ORIGINAL`. MISSING называет файл, который должен был дать более ранний saveset цепочки: один saveset цепочки пропущен.

7.8 Устройство отвергнуто

Признак. PHYSMOUNTED, PHYSHELD, PHYSSMALL.

Действие. Сохранение `/PHYSICAL` берёт устройство, которое не смонтировано или смонтировано только для чтения; у смонтированного сохраняйте снимок. Восстановление никогда не пишет на смонтированное устройство или занятое другой программой: сначала отмонтируйте его. Устройство вывода должно быть не меньше сохранённого (раздел 3.6).

7.9 HELP показывает только сводку

Признак. `vbackup /HELP` показывает короткую сводку и никаких тем.

Причина. VBACKUP собран без продукта HELP (раздел 2.1).

Действие. Установите HELP, соберите и установите VBACKUP заново; или читайте страницу руководства `vbackup(1)` и это руководство.

7.10 Файловый менеджер не открывает saveset

Действие. Модули ставятся в Midnight Commander и far2l, только если те найдены во время установки; поставьте сначала файловый менеджер, потом VBACKUP, или разложите файлы из `share/vbackup/plugins` вручную (приложение D). Total Commander берёт модуль из его архива (`vbackup_wcx.zip`).

Приложение A. Файлы

A.1 Устанавливаемые файлы

Пути даны относительно префикса, по умолчанию `/usr/local`.

Файл	Содержимое
`bin/vbackup`	Утилита
`bin/vbkx`	Автономный распаковщик
`share/man/man1/vbackup.1`, `vbkx.1`	Страницы руководства
`share/help/vbackup.hlb`, `share/help/ru/vbackup_ru.hlb`	Библиотеки справки, английская и русская (с HELP)
`share/doc/vbackup/`	`README.md`, `LICENSE`, `NOTICE`, `format.md`, `vmsbackup.md`, `vbackuplib.md`, `vbackuplib_ru.md`, это руководство (`vbackup_users_guide.md`, `vbackup_users_guide_ru.md` и их PDF)
`share/vbackup/plugins/mc/`	`uvbk` и `mc.ext.ini.vbackup` — Midnight Commander
`share/vbackup/plugins/far/vbackup.ini`	far2l и Far Manager (MultiArc)
`share/vbackup/plugins/wcx/`	`vbackup.wcx` для Double Commander, `pluginst.inf`; `win/` — модуль Total Commander, если он собирался
`share/vbackup/plugins/magic/vbackup.magic`	Сигнатура saveset-а для `file(1)`

Если найдены Midnight Commander и far2l, установка также кладёт `uvbk` в каталог extfs Midnight Commander, а его разделы, между метками, — в `mc.ext.ini` и в `custom.ini` модуля MultiArc.

A.2 Используемые файлы

Файл	Назначение
Журнал `/JOURNAL=`*файл*	Учёт saveset-ов и их файлов — для `/SINCE=BACKUP` и `/LIST /JOURNAL` (раздел 3.5)
Файл ключа `/KEY_FILE` или `VBACKUP_KEY_FILE`	Пароль, в первой строке (раздел 3.8)
`/tmp`	Буфер каталога сохранения: безымянный временный файл, исчезает в конце сохранения
`~/.ssh`	Ключи, по которым VBACKUP заходит на другой узел (раздел 3.10)

Переменные окружения перечислены в разделе 5.2.

Приложение B. Сводка формата saveset-a

Это приложение кратко излагает формат saveset-a версии 1. Определяющее описание — `doc/format.md`; читающая программа, написанная только по этому документу, должна уметь выводить листинг и восстанавливать любой saveset. Начиная с X01-08 saveset можно читать и со стандартного ввода, а начиная с X01-11 поток несёт несколько томов (см. раздел 3.9; `format.md`, разделы 2 и 8). Версия 2 (начиная с X01-14) добавляет чётность `/PARITY`, версия 3 (начиная с X01-21) — запись SOLID.

B.1 Соглашения

- Все целые — little-endian, беззнаковые, если не оговорено иное (u8, u16, u32, u64; i64 — в дополнительном коде). Поля кодируются побайтно.
- TIME — 12 байт: i64 секунд с 1970-01-01 00:00:00 UTC, u32 наносекунд.
- Строки и пути — сырые байты без завершающего NUL; кодировка не предполагается.
- CRC — CRC-32/IEEE (отражённый полином 0xEDB88320, начальное значение и итоговое XOR 0xFFFFFFFF; CRC(“123456789”) = 0xCBF43926), не CRC-32C.
- UUID — 16 случайных байт, версия 4.

B.2 Тома и блоки

```
volume 1 : VHDR  G G G ... G
volume 2 : VHDR  G G ... G
...
volume K : VHDR  G G ... G  TRAILER

G (group) = DATA x n, then XOR        (1 <= n <= N; N = 0: no XOR blocks)
```

Том $k >= 2$ называется *имя*.k, не менее трёх цифр. Группа никогда не пересекает границу тома. Размер блока B — от 8192 до 1048576, кратен 512; полезная нагрузка блока — P = B - 64 байта. Блоки нумеруются с 0 через весь saveset.

Поток (канал, начиная с X01-11) несёт тома подряд, каждый со своим VHDR; номера блоков идут как обычно. Читатель потока принимает VHDR следующего тома как конец текущего; приёмник раскладывает поток по файлам томов на каждом VHDR. Читатель потока читает только вперёд: TRAILER заканчивает группы там, где пришёл, а каталог в конце потока для поиска не используется.

B.3 Заголовок блока (64 байта)

Смещение	Размер	Поле	Смысл
0	4	magic	`V B K B`
4	2	hdrlen	64
6	2	version	1; 2 при `/PARITY` от 2; 3, если в saveset-е могут быть записи SOLID (сжатое сохранение без `/NOSOLID`), при любом `/PARITY`
8	4	bsize	B
12	1	type	1 DATA, 2 XOR, 3 VHDR, 4 TRAILER, 5 EDATA, 6 ETRAILER, 7 PARITY
13	1	flags	бит 0 LASTINVOL, бит 1 LASTINSET
14	2	gindex	DATA: позиция в группе; XOR: n; PARITY: n + 256 * строка; иначе 0
16	16	ssuuid	UUID saveset-a
32	8	blkno	номер блока
40	4	volno	номер тома, с 1
44	4	recoff	DATA: смещение первого заголовка записи, начинающейся в блоке; 0xFFFFFFFF, если такой нет
48	4	paylen	DATA: занятые байты нагрузки; VHDR, TRAILER: длина тела TLV; XOR: P
52	4	prvrecoff	recoff предыдущего блока DATA группы, 0xFFFFFFFF для первого
56	4	prvpaylen	paylen предыдущего блока DATA группы, 0 для первого
60	4	crc	CRC заголовка (с этим полем, равным 0) и всей области нагрузки

Нагрузка блока XOR — XOR областей нагрузки по P байт n блоков DATA его группы. Один плохой блок DATA восстанавливается по ней; его `recoff` и `paylen` берутся из `prvrecoff` и `prvpaylen` следующего за ним блока.

B.4 Типы блоков

Тип	Имя	Содержимое
1	DATA	Часть потока записей
2	XOR	Чётность блоков DATA своей группы (никогда не шифруется)
7	PARITY	Строка 1 .. m - 1 чётности Reed-Solomon своей группы, версия 2 (`/PARITY`)
3	VHDR	Первый блок каждого тома: полная запись SUMMARY (в зашифрованном saveset-е — краткая открытая SUMMARY)
4	TRAILER	Последний блок saveset-a: одно тело TLV с итогами и местом каталога
5	EDATA	Блок DATA зашифрованного saveset-a
6	ETRAILER	TRAILER зашифрованного saveset-a

B.5 Записи

Заголовок записи, 8 байт: u16 тип, u16 флаги (0), u32 длина тела. Запись может пересекать границы блоков и томов; её заголовок — никогда.

Тип	Запись	Тело
1	SUMMARY	элементы TLV
2	FILE	элементы TLV (теги файла)
3	DATA	u32 fileno, u32 0, u64 смещение, данные (не более 1048576 байт)
4	FEND	элементы TLV: FILENO, SIZE, CRC, STATUS
5	CATALOG	элементы: u32 длина элемента, элементы TLV
6	END	элементы TLV: NFILES, NBYTES, NERRORS
7	DATAZ	u32 fileno, u32 кодек (1 = блок LZ4), u64 смещение, u32 rawlen, сжатые байты
8	SOLID	u32 кодек (как в DATAZ), u32 rawlen (не больше 1114112), u32 count, сжатые байты: записи FILE, DATA и FEND нескольких мелких файлов (только версия 3; format.md 6.12)

Порядок в потоке:

```
SUMMARY
  ( FILE [DATA|DATAZ ...] FEND ) ...   one group per file; version 3: any
                                       run of them may be one SOLID instead
  CATALOG ...                          at most 1 MiB body each
END
```

Запись неизвестного типа и элемент TLV с неизвестным тегом читающий пропускает по их длине. Распакованная запись SOLID — последовательность записей FILE, DATA и FEND с их 8-байтными заголовками, каждая целиком внутри, последняя кончается на `rawlen`; читающий версии 3 отдаёт их так, как если бы они стояли в потоке. Запись SOLID, которая не открывается, – пробел, как потерянные блоки. Запись SOLID, разрезанная потерянными блоками или концом обрезанного saveset-a, теряется не вся (VBACKUP начиная с X01-22): сжатые байты до разреза распаковываются настолько, насколько это в точности верно, — LZ4 и Deflate до последней целой последовательности или кода, LZMA до последнего символа, декодированного до конца входа, никогда не угаданный октет, — и записи, целиком лежащие в этом, отдаются, затем пробел. Пишущий закрывает группу, прежде чем её записи превысили бы 262144 байта (1048576 в X01-21).

После последовательного прохода каталог, если он есть, сверяется со встреченными записями FILE — всегда, а не только после потерь (VBACKUP начиная с X01-22). Файл каталога (STATUS 0..2), запись FILE которого так и не пришла, называется: потерянным вместе с блоками, если блоки были потеряны, иначе — “the catalog has it, the stream does not” (NOTINSTREAM). Поток делает это со своими записями CATALOG по мере их поступления (`format.md`, раздел 8).

B.6 Теги

Элемент TLV — u16 тег, u32 длина, значение. Для всех записей одно пространство тегов.

Теги	Группа	Имена
1-20	Теги файла (FILE, CATALOG)	FILENO, PATH, FTYPE (1 REG, 2 DIR, 3 SYMLINK, 4 HARDLINK, 5 CHR, 6 BLK, 7 FIFO, 8 SOCK), MODE, UID, GID, UNAME, GNAME, SIZE, MTIME, ATIME, CTIME, BTIME, RDEV, LINK, XATTR, FSFLAGS, DEVINO, BASEIDX, NLINK
21-22	FILE (21 и в CATALOG)	WINATTR (u32, FILE_ATTRIBUTE_* Windows), NTSD (дескриптор безопасности, self-relative) — файлы, сохранённые в Windows, с X01-18
23	CATALOG	SOLID (u8): 1 — запись FILE лежит внутри записи SOLID по LOCVOL, LOCBLK, LOCOFF
32-36	FEND, CATALOG	CRC, STATUS (0 OK, 1 CHANGED, 2 READERR, 3 PRESENT — только в каталоге), LOCVOL, LOCBLK, LOCOFF
64-77	SUMMARY	PRODUCT, HOST, USER, CMDLINE, CREATED, BASE, BLOCKSIZE, GROUPSIZE, VOLSIZE, COMMENT, SYSTEM, KIND (0 FULL, 1 INCREMENTAL), FILTER, COMPRESS
78-80	SUMMARY, /PHYSICAL	PHYSICAL, DEVSIZE, SECTORSIZE
81-87	SUMMARY, /IMAGE	IMAGE, FSTYPE, FSLABEL, FSUUID, FSUSED, ROOTATTR, MOUNTOPTS
88-92	VHDR и SUMMARY, шифрование	CIPHER (1), KDF (1), KDFITER, SALT, KEYCHECK
96-104	END, TRAILER	NFILES, NBYTES, NERRORS, NBLOCKS, CATVOL, CATBLK, CATOFF, NVOLS, NENTRIES
128-130	Журнал	SSUUID, SPEC, RECORDED

B.7 Шифрование (format.md, 6.10)

- Примитивы: ChaCha20 (RFC 8439) и SHA-256 (FIPS 180-4), с построенными на нём HMAC-SHA256 и PBKDF2-HMAC-SHA256.
- Ключи:

```
MK    = PBKDF2-HMAC-SHA256(passphrase, SALT, KDFITER, 32 bytes)
KENC  = HMAC-SHA256(MK, "VBACKUP ENC")
KMAC  = HMAC-SHA256(MK, "VBACKUP MAC")
CHECK = HMAC-SHA256(MK, "VBACKUP CHECK")     stored as KEYCHECK
```

- Область нагрузки EDATA и ETRAILER: шифротекст (`paylen` байт), нули и 32-байтный TAG по смещению P - 32; `paylen` не больше P - 32.
- Шифрование: шифротекст = открытый текст XOR ChaCha20(KENC, nonce = u32 0 || u64 blkno, счётчик с 0).
- TAG = HMAC-SHA256(KMAC, ssuuid || bsize || blkno || volno || type || gindex || recoff || paylen || шифротекст). Блок с верным CRC и неверным TAG – плохой блок. TAG проверяется до расшифровки.
- VHDR несёт в открытом виде только PRODUCT, BLOCKSIZE, GROUPSIZE, VOLSIZE, CIPHER, KDF, KDFITER, SALT и KEYCHECK; полная SUMMARY — первая запись зашифрованного потока.
- CRC и блоки XOR вычисляются по шифротексту: проверке и починке пароль не нужен.

B.8 Журнал

Отдельный файл: 16-байтный заголовок (`V B K J`, u16 версия 1, u16 резерв, u64 число записей), записи в формате записей saveset-a — SSET (тип 16) для saveset-a, FSTATE (тип 17) для файла — и завершающий u32 CRC всего предыдущего. Ключ FSTATE — абсолютное имя файла. Журнал переписывается целиком через `журнал.tmp` под исключительной блокировкой `журнал.lock`.

B.9 Saveset-ы OpenVMS BACKUP

Читаются, не пишутся (раздел 3.16; `doc/vmsbackup.md`): блоки одного размера с заголовком 256 байт (BBH: size 256, уровень структуры 0x0101, applic 1 у данных и 2 у XOR, номер блока с 1, CRC-32/IEEE блока с нулями в полях CRC и checksum, флаг 1 — `/NOCRC`), записи с заголовком 16 байт (BRH: размер, тип 1 SUMMARY, 3 FILE, 4 VBN, адрес VBN), атрибуты SUMMARY и FILE — TLV со счётчиком, данные файла — его VBN по 512 байт.

Приложение C. Автономный распаковщик vbkx

C.1 Описание

vbkx читает saveset-ы, созданные VBACKUP, на машине, где VBACKUP не установлен: в системе восстановления, в другом дистрибутиве, в файловом менеджере. Он собран только из ядра чтения VBACKUP — без командного языка, библиотеки справки и средства сообщений — и скомпонован статически там, где система это позволяет: скопируйте один файл и запустите.

vbkx версии X01-21 читает saveset-ы формата версии 3 — записи SOLID сжатого сохранения (раздел 3.7, `/SOLID`), — как и версий 1 и 2, во всех своих командах: `l`, `x`, `p`, `t`. Файл, названный в `x` или `p`, достаётся через каталог по его записи SOLID, которая распаковывается, файлы перед ним пропускаются. vbkx до X01-21 такой saveset отвергает (`is not a saveset`). vbkx версии X01-22 восстанавливает файлы записи SOLID до места, где её разрезали потерянные блоки, как VBACKUP (раздел 1.7), а в `x` и `t` называет каждый файл каталога, которого поток не дал.

C.2 Формат

```
vbkx l saveset [-m] [-k keyfile] [-n]
vbkx x saveset [-C dir] [-f] [-j] [-k keyfile] [-n] [name ...]
vbkx p saveset name [-k keyfile] [-n]
vbkx t saveset [-k keyfile] [-n]
```

Сначала идёт буква команды, затем saveset (первый том или - для стандартного ввода), затем ключи и имена в любом порядке.

C.3 Команды

Команда	Действие
`l`	Выводит файлы, по строке на файл, из каталога (или из потока, если каталога нет).
`x`	Извлекает все файлы за один проход или только заданные имена, каждое через каталог. Имя каталога берёт всё, что под ним.
`p`	Пишет один файл на стандартный вывод.
`t`	Читает весь saveset и проверяет все контрольные суммы; при успехе выводит *saveset*: `all files read, all checksums match`. Файл каталога, которого поток не дал, называется, и проверка не проходит.

Листинг имеет вид (время местное):

```
2026-10-05 14:19:25         4096 d0755 rrl
2026-10-05 14:19:25            6 -0644 rrl/a.txt
2026-10-05 14:19:25            5 l0777 rrl/link -> a.txt
```

Буквы типов — как у `ls`: `- d l h` (жёсткая ссылка, за которой следует `link to` *имя*) `c b p s`, затем режим в восьмеричном виде.

C.4 Ключи

Ключ	Команды	Смысл
`-C dir`	x	Извлекать в *dir*, создаваемый при отсутствии; по умолчанию — текущий каталог.
`-f`	x	Перезаписывать существующие файлы; без него они оставляются, и об этом сообщается.
`-j`	x	Извлекать файлы по последнему компоненту имени, без каталогов («извлечь без пути» MultiArc).
`-m`	l	Листинг без `-> цель`, для программ.
`-k keyfile`	все	Пароль зашифрованного saveset-а: первая строка *keyfile*, который может читать только владелец (режим 600).
`-n`	все	Никогда не спрашивать пароль (для программ); то же делает `VBACKUP_NOPROMPT=1`.

Без `-k` используется файл ключа, названный `VBACKUP_KEY_FILE`; без того и другого пароль запрашивается на терминале (в Windows — на консоли).

C.5 Что восстанавливается

vbkx восстанавливает данные и дыры, режим, времена изменения и доступа, каталоги, символические и жёсткие ссылки и FIFO; под root — также владельца (по номеру) и файлы устройств. ACL, расширенные атрибуты и флаги chattr он не восстанавливает; когда они нужны, восстанавливайте VBACKUP.

Именам saveset-а не доверяют: имя с `..`, ведущей `/` или путём через символическую ссылку отвергается, каждый каталог на пути открывается с O_NOFOLLOW. Повреждения чинятся, насколько позволяют блоки XOR; каждый обрезанный файл называется `is incomplete`, каждый файл каталога, до которого не удалось добраться, — `not extracted` (после того как потерянные блоки разрезали запись SOLID, — её файлы начиная с места повреждения; файлы до него извлекаются). Каталог всегда сверяется с потоком (начиная с X01-22): файл каталога, записи которого так и не пришли, хотя ни один блок не потерян, называется `not extracted: the catalog has it, the record stream does not` (как NOTINSTREAM у VBACKUP). Из канала (`-`) saveset выводится и извлекается по мере чтения; имена отвергаются, потому что его каталог идёт последним; его записи каталога сверяются с потоком по мере поступления.

C.6 Диагностика

vbkx пишет диагностику на стандартный вывод ошибок с префиксом `vbkx:` в форме сообщений VBACKUP — `Метка: значение - слова`, — но без даты, номера процесса и идентификатора сообщения:

```
vbkx: Block: 3, Volume: 1 - is bad and cannot be rebuilt
vbkx: File: rrl/big.bin - checksum mismatch: the data differ from what was saved
vbkx: File: rrl/big.bin - is incomplete: its data was lost in bad blocks
vbkx: File: rrl/c.txt - not extracted: its records were lost in bad blocks
vbkx: File: rrl/d.txt - not extracted: the catalog has it, the record stream does not
vbkx: Saveset: e.bck - the passphrase does not open it
```

C.7 Коды завершения

Код	Смысл
0	Сделано.
1	Что-то повреждено или не сделано (файл неполон, не извлечён, существует, атрибут не установлен).
2	Команду или saveset нельзя использовать (неверная команда, не saveset, не открывается, неверный или отсутствующий пароль, выходной каталог не создаётся).

C.8 vbkx.exe в Windows

vbkx.exe принимает те же команды. Имена, в saveset-е в UTF-8, становятся UTF-16 под \\?\, поэтому длинные пути работают. Восстанавливаются данные, размеры, времена, атрибут «только чтение» (нет бита записи у владельца), каталоги и жёсткие ссылки; символические ссылки — там, где Windows это позволяет (режим разработчика или право их создавать), иначе об этом сообщается. FIFO, файлы устройств, владельцы и режимы сверх «только чтение» не создаются. Имя, недопустимое в Windows, — компонент с <>:"\|?* или управляющим символом, имя устройства (CON, PRN, AUX, NUL, COM1-9, LPT1-9, в том числе с расширением), имя, оканчивающееся точкой или пробелом, — не извлекается, и об этом сообщается; точка повторной обработки (reparse point) на пути отвергается. Данные и листинг выводятся байт в байт, как в Linux.

Для vbkx.exe не нужны ни StarLet, ни CMake. С MinGW-w64, из корня дерева исходных текстов:

```
$ x86_64-w64-mingw32-gcc -O2 -Ilib -o vbkx.exe tools/vbkx.c \
    lib/vbkfmt.c lib/vbkrd.c lib/vbklz4.c lib/vbkdfl.c lib/vbklzm.c lib/vbkcrp.c lib/vbkvms.c lib/vbkrs.c
static -lshell32
$ make -f tools/Makefile.win
```

C.9 Распаковщики последней надежды: vbkx-go, vbkx-rs, vbkx-pl

Ещё три распаковщика хранятся на тот день, когда всё остальное умерло:

Программа	Исходный текст	Сборка
vbkx-go	`tools/go/main.go`	`go build -o vbkx-go main.go` (Go 1.19 или новее)
vbkx-rs	`tools/rust/src/main.rs`	`rustc -O -C strip=symbols -o vbkx-rs src/main.rs` (Rust 1.63 или новее) или `cargo build --release`
vbkx-pl	`tools/perl/vbkx.pl`	Собирать нечего: `perl vbkx.pl ...` (Perl 5.10 или новее, только модули `perl-base`)

`make` в `tools/go` и `tools/rust` делает то же; сборка CMake собирает vbkx-go и vbkx-rs, если найдены Go или Rust. Каждый — один исходный файл, использующий только стандартную библиотеку, нарочно простой, чтобы его можно было прочесть и исправить по `format.md`; заголовок каждого исходного текста — его руководство.

Формат

```
vbkx-go l saveset [-k file]
vbkx-go x saveset [-C dir] [-k file]
vbkx-go t saveset [-k file]
vbkx-go selftest
```

(то же для `vbkx-rs` и `perl vbkx.pl`)

Команда	Действие
`l`	Выводит файлы по записям FILE потока (каталог не нужен), время в UTC.
`x`	Извлекает все файлы в *dir* (по умолчанию — текущий каталог). Существующий файл никогда не перезаписывается.
`t`	Читает весь saveset и проверяет контрольные суммы.
`selftest`	Проверяет SHA-256, HMAC-SHA256, PBKDF2 и ChaCha20 по векторам их стандартов.

Листинг у всех трёх одинаков байт в байт:

```
2026-10-03 20:40:12         1234 -0644 tree/a.txt
2026-10-03 20:40:12            5 l0777 tree/link -> a.txt
2026-10-03 20:40:12            6 h0644 tree/hard link to tree/a.txt
```

Они восстанавливают данные (дыры остаются дырами), режимы с битами setuid, setgid и sticky, времена изменения и доступа, каталоги, символические ссылки, жёсткие ссылки и FIFO; не восстанавливают владельцев, ACL, расширенные атрибуты, флаги chattr, файлы устройств и сокеты. Записи DATAZ читаются собственным проверяющим декодером LZ4, зашифрованные saveset-ы – с `-k keyfile`, иначе `VBACKUP_KEY_FILE`, иначе с паролем, запрашиваемым на терминале без эха; каждый блок проверяется по тегу до расшифровки, а подделанный

блок восстанавливается по группе, как плохой. Они чинят один плохой блок на группу, после потери продолжают со следующего исправного блока, пропускают отсутствующий или обрезанный том, находят размер блока перебором, если потерян первый блок тома 1, и называют каждый повреждённый (`is incomplete`) или недостающий (`was not extracted`) файл. Saveset `/PHYSICAL` выходит как файл образа устройства, saveset `/IMAGE` – как дерево его файлов. Все три версии X01-21 читают saveset-ы версии 3: каждый открывает запись SOLID своим декодером и берёт её записи так, как если бы они стояли в потоке; запись SOLID, которая не открывается, – пробел, и её файлы называются недостающими. Запись SOLID, разрезанную потерянными блоками, они не спасают, как VBACKUP и vbkx начиная с X01-22: они теряют всю запись SOLID и всё равно называют каждый её файл.

vbkx-pl использует Digest::SHA, если он установлен, — только ради скорости; без него собственный SHA-256 примерно в сто раз медленнее (открытие зашифрованного saveset-а тогда занимает минуты). `VBKXPL_PURE=1` заставляет использовать собственный код, даже если Digest::SHA есть. Скорость чтения – несколько мегабайт в секунду.

Коды завершения всех трёх: 0 — всё сделано; 1 — что-то повреждено или не сделано; 2 — команду или saveset нельзя использовать.

Приложение D. Модули файловых менеджеров

D.1 Общие сведения

Saveset открывается как папка в перечисленных ниже файловых менеджерах: листинг, просмотр, копирование наружу, проверка. Все модули работают только на чтение: в saveset никогда ничего не пишется. Листинг берётся из каталога, поэтому saveset в сотни гигабайт открывается сразу, а файл копируется наружу по своему месту в каталоге — мелкий файл сжатого сохранения через свою запись SOLID (раздел 3.7). Модули X01-21 читают saveset-ы формата версии 3; старые их не открывают. Модули пользуются читающим VBACKUP: начиная с X01-22 и они восстанавливают файлы записи SOLID до места, где её разрезали потерянные блоки.

Файловый менеджер не может спросить пароль. Чтобы открыть зашифрованный saveset, поместите пароль в файл ключа (`chmod 600`) и назовите этот файл в переменной окружения `VBACKUP_KEY_FILE` до запуска файлового менеджера (в Windows — в системных настройках переменных окружения). Без неё saveset не открывается; ничто не ждёт ответа на экране. Модули устанавливают `VBACKUP_NOPROMPT=1` или передают vbkx ключ `-n`.

D.2 Midnight Commander

Элемент	Значение
Файлы	`plugins/mc/uvbk` (сценарий extfs), `plugins/mc/mc.ext.ini.vbackup` (раздел `mc.ext.ini`)
Работает поверх	`vbackup` (переменная `VBACKUP` может назвать другой образ)
Установка	`cmake --install` помещает `uvbk` в каталог extfs MC, а раздел — в `mc.ext.ini`, если MC установлен, и убирает их при удалении. Вручную: скопируйте `share/vbackup/plugins/mc/uvbk` в `~/.local/share/mc/extfs.d` (или `/usr/lib/mc/extfs.d`), а раздел — в `mc.ext.ini` перед `[Default]`.
Использование	Enter на файле `.bck`; F3 показывает полный листинг.

Раздел:

```
[vbackup]
Shell=.bck
ShellIgnoreCase=true
Open=%cd %p/uvbk://
View=%view{ascii} vbackup %f /SAVE_SET /LIST /FULL
```

`uvbk list` выдаёт каталог строками `ls -l` (`vbackup ARCHIVE /LIST /FORMAT=LS`); `uvbk copyout` извлекает один файл (`vbackup ARCHIVE TARGET /EXTRACT=NAME`). Копирование внутрь, удаление и создание каталогов отвергаются. Saveset из нескольких томов открывается только с локального диска: внутри другой VFS (архив, FTP) MC передаёт сценарию временную копию тома 1, и остальных томов рядом с ней нет.

D.3 far2l и Far Manager 3

Элемент	Значение
Файл	`plugins/far/vbackup.ini`, описание формата для модуля MultiArc
Работает поверх	vbkx (`vbkx.exe` в Windows, ищется по PATH)
Установка	far2l: дописывается в `~/.config/far2l/plugins/multiarc/custom.ini` (это делает установка). Far3: допишите его в `Plugins\MultiArc\Formats\custom.ini`.
Использование	Enter или Ctrl/PgDn на файле `.bck`; F5 копирует файлы наружу.

Команды, которые запускает MultiArc:

```
List=vbkx l %%AQ -m -n
Extract=vbkx x %%AQ -f -n %%FSQ
ExtractWithoutPath=vbkx x %%AQ -f -j -n %%FSQ
Test=vbkx t %%AQ -n
```

Saveset распознаётся по первым четырём байтам `VBKB` (`ID=56 42 4B 42`) и по расширению `bck`.

D.4 Total Commander и Double Commander

Элемент	Значение
Исходный текст	`plugins/wcx/vbkwcx.c`, модуль упаковщика WCX поверх ядра чтения
Файлы	`vbackup.wcx64` (Total Commander, 64 бита), `vbackup.wcx` (Total Commander, 32 бита), `vbackup.wcx` — разделяемый объект — для Double Commander в Linux
Сборка	`make -f plugins/wcx/Makefile linux win64 win32` (в `out/linux`, `out/win64`, `out/win32`); сборка CMake всегда собирает модуль для Linux (устанавливается в `share/vbackup/plugins/wcx`) и модули для Windows, если найден MinGW-w64
Установка в TC	Откройте архив модуля (`make -f plugins/wcx/Makefile zip`, с `pluginst.inf`) в TC и подтвердите; или Configuration -> Options -> Packer -> Configure packer extension WCXs, расширение `bck`.
Установка в DC	Options -> Plugins -> Packer plugins (WCX) -> Add, расширение `bck`.

Модуль строит листинг по каталогу (без каталога или с повреждённым каталогом — читает saveset целиком), копирует файл наружу по его месту в каталоге, проверяя FILENO и CRC, читает записи DATA и DATAZ и дыры, чинит то, что позволяют блоки XOR, и сообщает о файле, не извлечённом целиком (E_BAD_DATA). Том, которого нет рядом с томом 1, запрашивается через обратный вызов ChangeVol. Имя, ведущее за пределы целевого каталога (`..`, ведущая `/`, обратная косая черта; в Windows также `c:`, `con`, `x?`), в листинг не попадает. За пределы целевого каталога ничего не пишется.

Ограничения: зашифрованный saveset открывается только с `VBACKUP_KEY_FILE` (без неё — E_EOPEN, с неверным паролем — E_BAD_ARCHIVE); владельцы, ACL, расширенные атрибуты и флаги chattr не восстанавливаются; файлы устройств, FIFO и сокеты показываются в листинге, но не создаются; символические ссылки создаются в Linux и пропускаются в Windows; времена в листинге – времена DOS (местные, с шагом две секунды), а извлечённый файл получает свои времена полностью.